

1. (202111.2) Edgcore SONiC User Guide	3
1.1 (202111.2) SONiC - Getting Started	4
1.1.1 (202111.2) Installation of Edgcore SONiC Software	5
1.1.2 (202111.2) Managing SONiC Configuration Files	7
1.1.3 (202111.2) Zero Touch Provisioning (ZTP)	11
1.1.4 (202111.2) Displaying Feature States	14
1.1.5 (202111.2) New Default Configuration	18
1.1.6 (202111.2) Configure Management Interface	23
1.2 (202111.2) System Management Configuration	26
1.2.1 (202111.2) Specifying a Device Name	27
1.2.2 (202111.2) CPU and Memory Utilization	28
1.2.3 (202111.2) Docker Warm Restart	30
1.2.4 (202111.2) Initiating a Warm Reboot	31
1.2.5 (202111.2) Using Container Auto-Restart	32
1.2.6 (202111.2) Network Time Protocol (NTP)	33
1.2.7 (202111.2) System Log (syslog)	34
1.2.8 (202111.2) Troubleshooting Information	35
1.2.9 (202111.2) Reloading Configuration	36
1.2.10 (202111.2) Management Framework	38
1.2.11 (202111.2) MIB & Trap Support List	42
1.3 (202111.2) Interface Configuration	44
1.3.1 (202111.2) Shutting Down and Starting Up Interfaces	45
1.3.2 (202111.2) Configuring Interface IP Addresses	47
1.3.3 (202111.2) Link Aggregation Control Protocol (LACP)	48
1.3.4 (202111.2) Configuring Port Mirroring	49
1.3.5 (202111.2) Configure Interface Speed, Breakout Mode and Auto-Negotiation	50
1.3.6 (202111.2) Link-Training	53
1.3.7 (202111.2) Configure Laser Frequency and TX Power for ZR Transceiver	56
1.3.8 (202111.2) Auto-Negotiation	59
1.3.8.1 (202111.2) AutoNego User Guide for AS4630-54NPE/AS4630-54PE/AS4630-54TE	61
1.3.8.2 (202111.2) AutoNego User Guide for AS5812-54X	62
1.3.8.3 (202111.2) AutoNego User Guide for AS5835-54X/AS5835-54T	63
1.3.8.4 (202111.2) AutoNego User Guide for AS7326-56X	64
1.3.8.5 (202111.2) AutoNego User Guide for AS7712-32X	65
1.3.8.6 (202111.2) AutoNego User Guide for AS7726-32X	66
1.3.8.7 (202111.2) AutoNego User Guide for AS9716-32D	67
1.3.8.8 (202111.2) AutoNego User Guide for AS9726-32D	69
1.3.8.9 (202111.2) AutoNego User Guide for AS9736-64D	70
1.4 (202111.2) IP Configuration	71
1.4.1 (202111.2) IPv4/IPv6 Interfaces and Routes	72
1.4.2 (202111.2) Neighbor Discovery Protocol (NDP)	74
1.4.3 (202111.2) Static Anycast Gateway (SAG)	75
1.5 (202111.2) Layer 2	76
1.5.1 (202111.2) Configuring DHCP Relay	77
1.5.2 (202111.2) Configuring interface TPID	79
1.5.3 (202111.2) Control Plane Policing (CoPP)	80
1.5.4 (202111.2) Creating VLANs	82
1.5.5 (202111.2) Forwarding Database (FDB)	83
1.5.6 (202111.2) Link Aggregation (LAG)	84
1.5.7 (202111.2) Link Layer Discovery Protocol (LLDP)	85
1.5.8 (202111.2) Multi-Chassis Link Aggregation Group (MC-LAG)	86
1.5.9 (202111.2) PVST and STP	91
1.5.10 (202111.2) Virtual Extensible LAN (VxLAN)	92
1.6 (202111.2) Layer 3	94
1.6.1 (202111.2) SONiC Routing Stack	95
1.6.2 (202111.2) Border Gateway Protocol (BGP)	97
1.6.3 (202111.2) Bidirectional Forwarding Detection (BFD)	100
1.6.4 (202111.2) Virtual Routing and Forwarding (VRF)	101
1.6.5 (202111.2) Management VRF Configuration	102
1.6.6 (202111.2) Leaking VRF Routes	103
1.6.7 (202111.2) Network Address Translation (NAT)	105
1.6.8 (202111.2) Displaying ARP Entries	107
1.6.9 (202111.2) EVPN/VxLAN	108
1.6.10 (202111.2) EVPN-MH	116
1.6.11 (202111.2) DHCP Relay on EVPN/VxLAN	119
1.6.12 (202111.2) In Band Management VRF	126
1.6.13 (202111.2) BGP Unnumbered	127
1.7 (202111.2) Network Monitoring	128
1.7.1 (202111.2) Critical Resource Monitoring (CRM)	129
1.7.2 (202111.2) Flow Sampling (sFlow)	132
1.7.3 (202111.2) Configuring Drop Counters	133
1.7.4 (202111.2) Setting the Watermark Telemetry Interval	135
1.8 (202111.2) QoS Configuration	136
1.8.1 (202111.2) Class of Service (CoS)	137
1.8.2 (202111.2) QoS Marking	140
1.8.3 (202111.2) Explicit Congestion Notification (ECN)	143
1.8.4 (202111.2) Priority Flow Control (PFC)	146
1.8.5 (202111.2) Starting the PFC Watchdog	151
1.8.6 (202111.2) Displaying Queue and Priority-Group Counters	155
1.8.7 (202111.2) Queue Scheduling	157
1.8.8 (202111.2) Weighted Random Early Detection (WRED)	159
1.8.9 (202111.2) RDMA over Converged Ethernet (RoCE)	161
1.8.10 (202111.2) Port and Queue Shaping	165
1.8.11 (202111.2) Port Rate Limit (Traffic Policing)	167

1.8.12 (202111.2) Shared Buffer Configuration	169
1.8.13 (202111.2) ACL-based CoS	172
1.9 (202111.2) Security	176
1.9.1 (202111.2) AAA and TACACS+	177
1.9.2 (202111.2) Configuring TACACS+ Server Parameters	178
1.9.3 (202111.2) Configuring RADIUS Server Parameters	179
1.9.4 (202111.2) Access Control Lists (ACL)	181
1.9.5 (202111.2) Configuring a Control Plane ACL	185
1.9.6 (202111.2) ACL Rule Configuration File	187
1.10 (202111.2) PINS	191

(202111.2) Edgecore SONiC User Guide

(202111.2) SONiC - Getting Started

(202111.2) Installation of Edgecore SONiC Software

Edgecore SONiC (Enterprise SONiC Distribution by Edgecore Networks) is a network operating system based on the [SONiC sub-project \(OCP Networking Project Group\)](#) and runs on switches from Edgecore. The SONiC software needs to be loaded onto the supported switch platforms using ONIE, as described in [\(202111\) Install, Upgrade, and Remove Edgecore SONiC](#).

After installing SONiC on a switch, you can connect to the console port of the device and configure it through one of the following methods:

- Command Line Interface
- `config_db.json` configuration file
- `minigraph.xml` configuration file

See the [Managing SONiC Configuration Files](#) section for more information on JSON and XML configuration files.

SONiC also includes a command-line tool for installing the SONiC software as an alternative image in the partition. When a switch is already running a SONiC, you can use the `sonic-installer install` command to install another new image using the CLI.

Example

```
admin@sonic:~$ sudo sonic_installer install http://192.168.1.3/download/SONiC-OS-Edgecore-SONiC_20200827_110345_ec201911_2020-aug_enhanced_178.bin
New image will be installed, continue? [y/N]: y
Downloading image...
...100%, 480 MB, 3357 KB/s, 146 seconds passed
Command: /tmp/sonic_image
Verifying image checksum ... OK.
Preparing image archive ... OK.
ONIE Installer: platform: xxxx
onie_platform:
Installing SONiC in SONiC
Installing SONiC to /host/image-xxxx
Directory /host/image-xxxx/ already exists. Cleaning up...
Archive: fs.zip
creating: /host/image-xxxx/boot/
inflating: /host/image-xxxx/boot/vmlinuz-3.16.0-4-amd64
inflating: /host/image-xxxx/boot/config-3.16.0-4-amd64
inflating: /host/image-xxxx/boot/System.map-3.16.0-4-amd64
inflating: /host/image-xxxx/boot/initrd.img-3.16.0-4-amd64
creating: /host/image-xxxx/platform/
extracting: /host/image-xxxx/platform/firsttime
inflating: /host/image-xxxx/fs.squashfs
inflating: /host/image-xxxx/dockerfs.tar.gz
Log file system already exists. Size: 4096MB
Installed SONiC base image SONiC-OS successfully
Command: cp /etc/sonic/minigraph.xml /host/
Command: grub-set-default --boot-directory=/host 0
Done
```

You can set the new image to boot by default using the `sonic_installer set_default` command, which will load the image at the next reboot and all subsequent reboots.

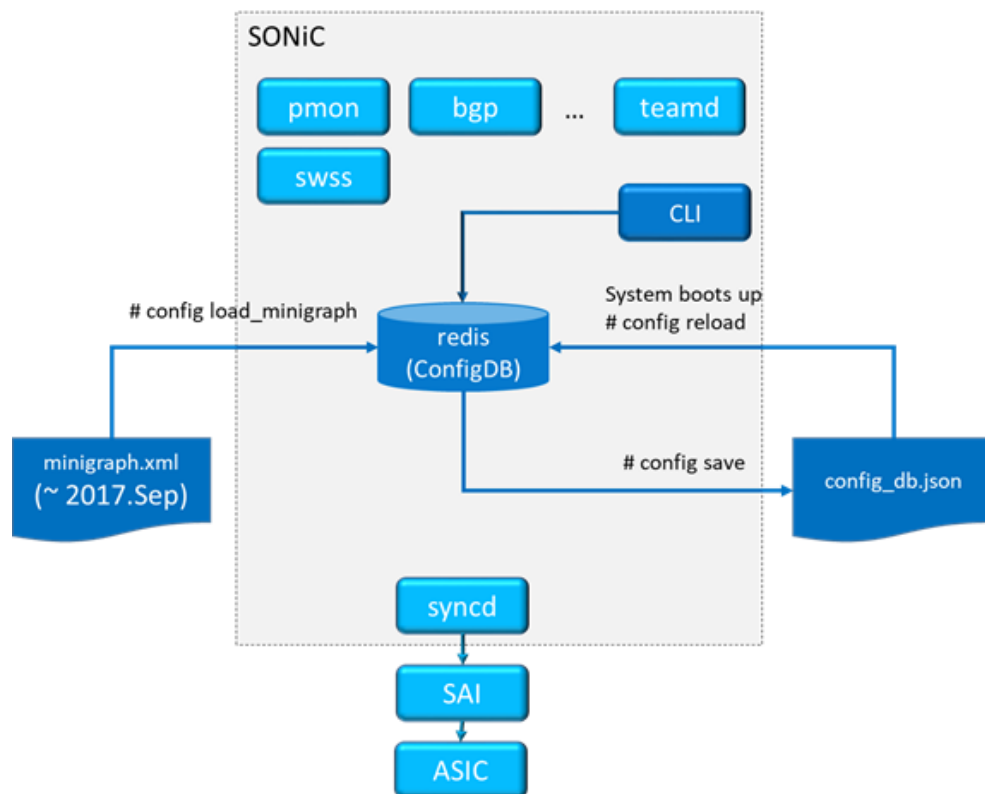
Example

```
admin@sonic:~$ sudo sonic_installer set_default SONiC-OS-HEAD.XXXX
```

Other commands related to software installation and management are documented in section [\(202111\) Software Installation and Management](#) of the CLI guide.

(202111.2) Managing SONiC Configuration Files

The SONiC configuration is dependent on two primary databases. A Redis database that can be considered as the switch “running configuration,” and a `config_db.json` file that functions like a “startup configuration” file.



Redis is an open source in-memory data structure store that maintains configuration changes from the various software features. The `config_db.json` file is text-readable in the JavaScript Object Notation (JSON) format. The `config_db.json` file configuration is imported into the Redis database at startup, but the Redis configuration is not written back to the `config_db.json` file unless a save command is manually executed from the CLI.



Note

SONiC also supports loading a configuration from an XML formatted file called `minigraph.xml` stored in the Linux file system `/etc/sonic/` location. The `minigraph.xml` file is a legacy method used before September 2017.

For further information on SONiC configuration files, see the section [Configuration File Management](#) in the CLI command reference.

- [Loading and Saving a JSON Configuration File](#)
- [Loading the SONiC Configuration from a minigraph.xml File](#)
- [Reloading the SONiC Configuration](#)
- [Loading the Management Interface Configuration](#)
- [Displaying the Running Configuration](#)

Loading and Saving a JSON Configuration File

You can load a switch configuration from a JSON file using the `config load` command. Either the default `/etc/sonic/config_db.json` file is loaded, or another file name can be specified.

The configuration in the file is loaded into the running Redis database. This command does not flush out the Redis database before loading the new configuration, the input either modifies or is added to the configuration.

The following example loads the configuration from the default `config_db.json` file.

Example

```
admin@sonic:~$ sudo config load
Load config from the file /etc/sonic/config_db.json? [y/N]: y
Running command: /usr/local/bin/sonic-cfggen -j /etc/sonic/config_db.json --write-to-db
```

Use the **config save** command from the CLI to save the running Redis database (the running configuration) to the default **/etc/sonic/config_db.json** file or another user-specified file. This will save the current configuration after a reboot.

Example

```
admin@sonic:~$ sudo config save -y /etc/sonic/config2.json
```

Loading the SONiC Configuration from a minigraph.xml File

Use the **config load_minigraph** command to load the configuration from the **/etc/sonic/minigraph.xml** file. The XML file must be copied to the **/etc/sonic/** location before executing the command. This command restarts various services running on the device and takes some time to complete.

Example

```
admin@sonic:~$ sudo config load_minigraph
Reload config from minigraph? [y/N]: y
Running command: /usr/local/bin/sonic-cfggen -j /etc/sonic/config_db.json --write-to-db
```

Reloading the SONiC Configuration

Along with loading a configuration file into the running Redis database, the **config reload** command also flushes out the current configuration before importing a new configuration. This command loads the default **/etc/sonic/config_db.json** file or another specified JSON-formatted file.

The **config reload** command takes some time to complete since it stops and restarts all services running on the device. In addition, this command can disrupt management connections to the device and they may require reconfiguration. In general, executing this command from the console port is recommended.

Example

```
admin@sonic:~$ sudo config reload
Clear current config and reload config from the file /etc/sonic/config_db.json? [y/N]: y
Running command: systemctl stop dhcp_relay
Running command: systemctl stop swss
Running command: systemctl stop snmp
Warning: Stopping snmp.service, but it can still be activated by:
  snmp.timer
Running command: systemctl stop lldp
Running command: systemctl stop pmon
Running command: systemctl stop bgp
Running command: systemctl stop teamd
Running command: /usr/local/bin/sonic-cfggen -H -k Force10-Z9100-C32 --write-to-db
Running command: /usr/local/bin/sonic-cfggen -j /etc/sonic/config_db.json --write-to-db
Running command: systemctl restart hostname-config
Running command: systemctl restart interfaces-config
Timeout, server 10.11.162.42 not responding.
```

Loading the Management Interface Configuration

To reconfigure a device's management interface and hostname, you can use the **config load_mgmt_config** command. This command loads the management configuration from the **/etc/sonic/device_desc.xml** file or another specified file.

Example

```
admin@sonic:~$ sudo config load_mgmt_config
Reload config from minigraph? [y/N]: y
Running command: /usr/local/bin/sonic-cfggen -M /etc/sonic/device_desc.xml --write-to-db
```



Note

device_desc.xml example file can be as below

```
<DeviceMiniGraph xmlns="Microsoft.Search.Autopilot.Evolution" xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Hostname>SONIC-X</Hostname>
  <HwSku>Accton-HW-Y</HwSku>
  <Address xmlns:a="Microsoft.Search.Autopilot.NetMux">
    <a:IPPrefix>127.0.0.1/16</a:IPPrefix>
  </Address>
  <ManagementAddress xmlns:a="Microsoft.Search.Autopilot.NetMux">
    <a:IPPrefix>192.168.200.10/24</a:IPPrefix>
  </ManagementAddress>
</DeviceMiniGraph>
```

Displaying the Running Configuration

Use the **show runningconfiguration all** command to display the current running configuration for all modules.

Example

```
admin@sonic:~$ show runningconfiguration all
```

Alternatively, you can also specify one of the following modules: `bgp`, `interfaces`, `ntp`, `snmp`, `acl`, `ports`, or `syslog` along with **`show runningconfiguration`** command to display the respective module details. See the [Startup & Running Configuration](#) section in the CLI reference guide for more details.

(202111.2) Zero Touch Provisioning (ZTP)

SONiC supports Zero Touch Provisioning (ZTP), which enables a large number of switches to be configured quickly and automatically. ZTP loads a JSON file at boot up that is essentially a script that performs the configuration steps needed to set up a switch on the network.

The ZTP JSON file can be installed on a device or on a provisioning server. When the ZTP JSON file is on a server, DHCP Option 67 can be used to obtain the URL of ZTP JSON file and enable it to be downloaded.

By default, the ZTP service is enabled in SONiC. The service can be disabled from the CLI using the **config ztp disable** command. Once the ZTP service has been disabled, it must be manually enabled again using the **config ztp enable** command.

Example

```
root@sonic:/home/admin# config ztp disable
Active ZTP session will be stopped and disabled, continue? [y/N]: y
Running command: ztp disable -y

root@sonic:/home/admin# config ztp enable
Running command: ztp enable
```

In addition, the **config ztp run** command can be used to start a new ZTP session. This command deletes the existing `/etc/sonic/config_db.json` file and erases the previous ZTP session data before loading a new ZTP JSON file.

Example

```
root@sonic:/home/admin# config ztp run
ZTP will be restarted. You may lose switch data and connectivity, continue? [y/N]: y
Running command: ztp run -y
Go Back To [<u>Beginning of the
document</span></span> or [<u>Beginning
of this section</span></span>
```

To check the status of the current ZTP session, use the **show ztp status** command. This command displays information related to all configuration sections as defined in the switch provisioning information discovered in a particular ZTP session.

For more information see the [ZTP](#) section in the CLI reference guide.

Example

```
root@B1-SP1-7712:/home/admin# show ztp status
ZTP Admin Mode : True
ZTP Service : Inactive
ZTP Status : SUCCESS
ZTP Source : dhcp-opt67 (eth0)
Runtime : 05m 31s
Timestamp : 2019-09-11 19:12:24 UTC
ZTP Service is not running
01-configdb-json: SUCCESS
02-connectivity-check: SUCCESS
Use the verbose option to display more detailed information.
root@B1-SP1-7712:/home/admin# show ztp status --verbose
Command: ztp status --verbose
=====
ZTP
=====
ZTP Admin Mode : True
ZTP Service : Inactive
ZTP Status : SUCCESS
ZTP Source : dhcp-opt67 (eth0)
Runtime : 05m 31s
Timestamp : 2019-09-11 19:12:16 UTC
ZTP JSON Version : 1.0
ZTP Service is not running
-----
01-configdb-json
-----
Status : SUCCESS
Runtime : 02m 48s
Timestamp : 2019-09-11 19:11:55 UTC
Exit Code : 0
Ignore Result : False
-----
02-connectivity-check
-----
Status : SUCCESS
Runtime : 04s
Timestamp : 2019-09-11 19:12:16 UTC
Exit Code : 0
Ignore Result : False
```


(202111.2) Displaying Feature States

- [Introduction](#)
- [Feature List](#)

Introduction

SONiC includes a capability in which a feature state can be enabled or disabled, which will make the corresponding feature docker container start or stop. In Edgecore SONiC, the [Feature List](#) describes the default service container states, the default admin state features, and the commands to check its states.



Note

After the 202006 branch, a new set of commands were introduced to check the container states. See - <https://github.com/Azure/sonic-utilities/blob/master/doc/Command-Reference.md#feature> for more information.

Until the **show feature** command is implemented, you can use the [Feature List](#) table to check the service states of SONiC features.

Refer to [Feature List](#) to find the command's service state and admin's (configuration) state in the table.

For example, for the LACP feature, the default service state of the container is **Enable**, and the default admin state is **Disable**. You can use the below commands to check the states.

command to show service state

```
systemctl status teamd.service
docker ps | grep teamd
docker exec -i teamd ps -x
```

These commands can be issued under the SONiC Linux shell.

The **systemctl** command can display **teamd** (LACP service container) status related information, and the other two docker commands are an alternative method to check the container states.

Use the **docker ps** command to check the running state of the **teamd** container. For experienced users, who want more information on the docker status, the **docker exec -i teamd ps -x** command can display the processes running in the **teamd** container.

command to show admin state

```
show interfaces portchannel
```

The **show interfaces** command is the SONiC CLI command that you can use to check the admin or operation states of the LACP feature.

Most of the commands used to check the admin states are SONiC CLI commands. In special cases, you may need to use other methodologies (e.g., other Linux shell commands or checking the config_db.json file) to check the feature states. For those cases, there will be a remark in the description.

Feature List

Category	Feature Name	Default Service Container	Default Admin State (Enable/Disable)
L2	LAG (LACP)	Enable systemctl status teamd.service docker ps grep teamd docker exec -i teamd ps -x	Disable admin@sonic~\$ show interfaces portchannel

	LLDP	Enable <code>systemctl status lldp.service</code> <code>docker ps grep lldp</code> <code>docker exec -i lldp ps -x</code>	Enable <code>admin@sonic~\$ show lldp table</code> <code>admin@sonic~\$ show lldp neighbors</code>
	MCLAG	Enable <code>systemctl status iccpd.service</code> <code>docker ps grep iccpd</code> <code>docker exec -i iccpd ps -x</code>	Disable <code>admin@sonic~\$ show mclag brief</code>
	QinQ	Enable <code>systemctl status swss.service</code>	Disable Check config_db.json for VLAN stacking configuration.
	Storm Control	Enable <code>systemctl status swss.service</code>	Disable <code>admin@sonic~\$ show storm-control all</code>
	VLAN	Enable <code>systemctl status swss.service</code> <code>docker exec -i swss ps -x grep vlanmgrd</code>	Disable <code>admin@sonic~\$ show vlan brief</code>
	VLAN Translation	Enable <code>systemctl status swss.service</code>	Disable Check config_db.json for VLAN stacking configuration.
L3	BGP	Enable <code>systemctl status bgp.service</code> <code>docker ps grep bgp</code> <code>docker exec -i bgp ps -x</code>	Disable <code>admin@sonic~\$ vtysh</code> Hello, this is FRRouting (version 7.2.1-sonic). Copyright 1996-2005 Kunihiro Ishiguro, et al. <code>sonic# show ip bgp summary</code> % BGP instance not found
	EVPN/VxLAN	Enable <code>systemctl status bgp.service</code> <code>show vxlan interface</code>	Disable Refer to EVPN HLD
	IS-IS	Enable <code>systemctl status bgp.service</code> <code>docker ps grep bgp</code>	Disable <code>admin@as7816-64x::~\$ vtysh</code> Hello, this is FRRouting (version 7.2.1-sonic). Copyright 1996-2005 Kunihiro Ishiguro, et al. <code>as7816-64x# show isis summary</code> (Number of area 0 means there is no active ISIS area)
	OSPF v2	Enable <code>systemctl status bgp.service</code> <code>docker ps grep bgp</code>	Disable <code>admin@sonic::~\$ vtysh</code> Hello, this is FRRouting (version 7.2.1-sonic). Copyright 1996-2005 Kunihiro Ishiguro, et al. <code>as7816-64x# show ip ospf</code> % OSPF instance not found
	Proxy ARP	Enable <code>systemctl status swss.service</code>	Disable <code>root@sonic::~# cat /proc/sys/net/ipv4/conf/[interface]/proxy_arp_pvlan</code> ('0' is disabled, '1' is enabled) This command is issued under the Linux shell.
	SAG (Static Anycast Gateway)	Enable <code>systemctl status swss.service</code>	Disable <code>admin@sonic::~\$ show sag</code>

	VRF	Enable systemctl status swss.service	Disable admin@sonic:~\$ show vrf
QoS	CoS	Enable systemctl status swss.service	Enable CoS mapping configuration uses the chip default configuration. Check config_db.json for DOT1P_TO_TC_MAP configuration if you change the configuration.
	DSCP	Enable systemctl status swss.service	Enable DSCP mapping configuration uses the chip default configuration. Check config_db.json for DSCP_TO_TC_MAP configuration if the user changes the configuration.
	ECN-WRED	Enable systemctl status swss.service	Disable admin@sonic:~\$ show ecn
	PFC (Priority Flow Control)/Asym PFC	Enable systemctl status swss.service	Disable admin@sonic:~\$ show pfc priority
	PFC Watchdog (WD)	Enable systemctl status swss.service	Disable admin@sonic:~\$ show pfcwd config
	PFC Watermark (WM)	Enable systemctl status swss.service	Enable admin@sonic:~\$ show counterpoll show
	Port Rate Limit	Enable systemctl status swss.service	Disable Check config_db.json for SCHEDULER and PORT_QOS_MAP configuration.
Security	AAA	Enable systemctl status swss.service	Enable admin@sonic:~\$ show aaa
	CoPP	Enable systemctl status swss.service	Enable docker exec swss cat /etc/swss/config.d/00-copp.config.json This command is issued under the Linux shell.
	IPv4 ACL	Enable systemctl status swss.service	Disable admin@sonic:~\$ show acl table
	IPv6 ACL	Enable systemctl status swss.service	Disable admin@sonic:~\$ show acl table
	LDAP	Enable systemctl status swss.service	Disable admin@sonic:~\$ show aaa
	TACACS+	Enable systemctl status swss.service	Disable. admin@sonic:~\$ show aaa
Management & Monitor	Configurable drop counter	Enable systemctl status swss.service	Disable admin@sonic:~\$ show dropcounters configuration
	Critical Resource Monitoring (CRM)	Enable systemctl status swss.service	Enable admin@sonic:~\$ crm show resources
	DHCP Relay Agent	Enable systemctl status dhcp_relay.service docker exec -i dhcp_relay ps -x	Disable admin@sonic:~\$ show vlan brief Check DHCP Helper Address Field
	DNS Client	Enable systemctl status systemd-resolved.service	Disable

	Management Framework	Enable <code>systemctl status mgmt-framework.service</code> <code>docker ps grep mgmt-framework</code> <code>docker exec -i mgmt-framework ps -x</code>	Enable
	Mgmt VRF	Enable <code>systemctl status swss.service</code>	Disable <code>admin@sonic:~# show mgmt-vrf</code>
	Mirror	Enable <code>systemctl status swss.service</code>	Disable <code>admin@sonic:~# show mirror_session</code>
	NAT	Enable <code>systemctl status swss.service</code>	Disable <code>admin@sonic:~# show nat config</code>
	NTP	Enable <code>systemctl status ntp.service</code>	Enable <code>admin@sonic:~# show ntp</code>
	Platform Monitor	Enable <code>systemctl status pmon.service</code> <code>systemctl status *-platform-monitor.service</code>	Enable <code>admin@sonic:~# show platform syseeprom</code> <code>admin@sonic:~# show platform psustatus</code> <code>admin@sonic:~# show interfaces transceiver</code>
	sFlow	Enable <code>systemctl status sflow.service</code>	Disable <code>admin@sonic:~# show sflow</code>
	SNMP	Enable <code>systemctl status snmp.service</code>	Enable
	Syslog	Enable <code>systemctl status rsyslog.service</code>	Enable
	Telemetry	Enable <code>systemctl status telemetry.service</code> <code>docker ps grep telemetry</code>	Enable
	ZTP	Enable <code>systemctl status ztp.service</code>	Enable <code>admin@sonic:~# sudo show ztp status</code>

(202111.2) New Default Configuration

- What is "default configuration"?
- Why there is an Edgecore distribution "default configuration"?
 - FRR configuration
 - REST Server configuration
 - Remove configurations
- How do you reset to "default configuration"?
 - Configuration manipulation
 - Command Line (Supports 202012.3 or later versions)

What is "default configuration"?

When a device boots up for the first time, the default configuration is automatically generated and written to **config_db.json**, in case the configuration file **config_db.json** is missing. In this instance, according to the generic procedure, SONiC refers to the minigraph template, platform, and port configuration to generate the "Default Startup Configuration" described in the [SONiC User Manual](#).

Why there is an Edgecore distribution "default configuration"?

In the Edgecore distribution, default configuration allows users to easily generate the configuration file as needed rather than adding numerous needless configurations to the file. Traditional switch users can expect only the most basic configuration, such as the port configuration, can be found in the default configuration file, which they can configure as required.

The ACL, IP, and QOS configurations in the SONiC Community "Default Startup Configuration" have increased the effort in the deployment (planning) stage. In response to this feedback, in Enterprise SONiC Distribution by Edgecore (or EC SONiC), we added an option called "factory" which generates configurations as simply as possible in the absence of a SONiC configuration file (config db.json). Even without the "factory" option, users can still use Community's "Default Startup Configuration" to generate configurations as they want.

To make the file easier to deploy for the users, The following changes are made to the "EC default configurations", compared to the Community's SONiC "Default Startup Configuration" file.

FRR configuration

The EC default configuration allows the FRR to use its original configuration and activate ISIS/FRR functionalities, enabling users to fully control the FRR via its VTYSH.

- **docker_routing_config_mode**: This configuration can be set to "unified", or "split", or "separated" as per Community's definition. In EC default configuration, default changes to **"split"**, to allow the user to fully control the FRR and store the configuration in the device.

In the 202006 version of Edgecore distribution, the default configuration mode is set to "split", while in the 202012 & 202111 versions, it is "separated".

To make the transition to a new version smoothly, it is recommended that if the user wants to use the original config_db.json file in the 202012 & 202111 versions, they should add **docker_routing_config_mode** in the **DEVICE_METADATA** section and set it to **"split"** manually.

- **frr_mgmt_framework_config**: This configuration can be set to "true", or "false" as per Community's definition. However, In EC default configuration, the default changes to **"true"** to enable OSPF/ISIS features in the FRR.

In the 202012 & 202111 versions, it is essential to add this configuration in **DEVICE_METADATA** to enable OSPF and ISIS features in FRR.

To make the transition to a new version smoothly, it is recommended that if the user wants to use the original config_db.json file in the 202012 & 202111 versions, they should add **frr_mgmt_framework_config** in the **DEVICE_METADATA** section and set it to **"true"**.

The following is an example to show the changes in the configurations of **DEVICE_METADATA**.

Example

```
"DEVICE_METADATA": {
  "localhost": {
    "buffer_model": "traditional",
    "default_bgp_status": "up",
    "default_pfcwd_status": "disable",
    "docker_routing_config_mode": "split",
    "frr_mgmt_framework_config": "true",
    "hostname": "leaf-3",
    "hwsku": "Accton-AS7326-56X",
    "mac": "04:f8:f8:6a:fa:91",
    "platform": "x86_64-accton_as7326_56x-r0",
    "type": "LeafRouter"
  }
},
```

REST Server configuration

In the Community SONiC's "Default Startup Configuration", the REST server does not enable an authentication mechanism. As a result, there will be an increase in risk and security issues. In Edgecore distribution, by default, the REST server enables the authentication to lower the security risk.

Example

```
"REST_SERVER": {
  "default": {
    "client_auth": "user"
  }
},
```

In the 202012 & 202111 versions, it is strongly recommended to enable authentication on the REST server to prevent access from anonymous.

This also applies to users who use existing configuration files on the 202012 & 202111 versions. It is strongly recommended to add this configuration to the existing configuration file to provide stronger protection for your device access.

Remove configurations

Users cannot use certain default Community SONiC configuration settings if the deployment environment is different. To address this and to maintain a cleaner basis for customers to quickly deploy them, these types of configurations are removed from the Edgecore distribution.

In the EC default configuration, the "factory" setup will remove the below configurations.

- DEVICE_METADATA : "bgp_asn"
- LOOPBACK_INTERFACE
- BGP_NEIGHBOR
- DEVICE_NEIGHBOR
- INTERFACE from config_db.json

- **DEVICE_METADATA** ["localhost"]["bgp_asn"] - This is only used when **docker_routing_config_mode** is set to "separated", to allow the user to have full control of FRR. The "bgp_asn" setting is ineffective if the **docker_routing_config_mode** is set to "split" and will be removed as shown in the below example. This original configuration is used to set AS number for BGP. In Edgecore Distribution, the user should specify the BGP AS number according to the deployment plan.

Example

```
"DEVICE_METADATA": {
  "localhost": {
    "bgp_asn": "65100",          <-- removed
    "hostname": "sonic",
    "hwsku": "Accton-AS7326-56X",
    "mac": "04:f8:f8:6a:fa:91",
    "platform": "x86_64-accton_as7326_56x-r0",
    "type": "LeafRouter"
  }
},
```

- **LOOPBACK_INTERFACE** - Loopback interface configuration should be decided by the user.

Example

```
"LOOPBACK_INTERFACE": {
  "Loopback0": {},
  "Loopback0|10.1.0.1/32": {}
},
```

- **BGP_NEIGHBOR** - The BGP neighbor configuration should be decided by the user according to the deployment plan.

Example

```
"BGP_NEIGHBOR": {
  "10.0.0.1": {
    "asn": "65200",
    "holdtime": "180",
    "keepalive": "60",
    "local_addr": "10.0.0.0",
    "name": "ARISTA01T2",
    "nhopself": "0",
    "rrclient": "0"
  },
  "10.0.0.3": {
    "asn": "65200",
    "holdtime": "180",
    "keepalive": "60",
    "local_addr": "10.0.0.2",
    "name": "ARISTA02T2",
    "nhopself": "0",
    "rrclient": "0"
  },
  "10.0.0.5": {
    "asn": "65200",
    "holdtime": "180",
    "keepalive": "60",
    "local_addr": "10.0.0.4",
    "name": "ARISTA03T2",
    "nhopself": "0",
    "rrclient": "0"
  },
  .....
},
```

- **DEVICE_NEIGHBOR** - The Community default configuration on this is empty. Remove it to make the configuration clean.

Example

```
"DEVICE_NEIGHBOR": {},
```

- **INTERFACE** - In the Community default configuration, every port is assigned an IP address. The IP address on the interface should be planned and configured by the user according to the deployment plan.

Example

```
"INTERFACE": {
  "Ethernet0|10.0.0.0/31": {},
  "Ethernet1|10.0.0.2/31": {},
  "Ethernet2|10.0.0.4/31": {},
  "Ethernet3|10.0.0.6/31": {},
  "Ethernet4|10.0.0.8/31": {},
  "Ethernet5|10.0.0.10/31": {},
  .....
  .....
  "Ethernet51|10.0.0.102/31": {},
  "Ethernet52|10.0.0.104/31": {},
  "Ethernet56|10.0.0.106/31": {}
}
```

How do you reset to "default configuration"?

The default configuration will only be generated on the first boot after installing the image file to the device through ONIE mode.

The better approach is to reset the device to the default configuration and then re-apply the essential configuration.

Configuration manipulation

The following example shows how to reset the device to the default configuration.

Step 1: Install the image from ONIE.

Step 2: Disable ZTP.

```
admin@sonic:~$ sudo config ztp disable
```

Step 3: Delete the old configuration file.

```
admin@sonic:~$ sudo rm /etc/sonic/config_db.json
```

Step 4: Reboot the device.

```
admin@sonic:~$ sudo reboot
```

After reboot, the new default configuration will be generated.

Command Line (Supports 202012.3 or later versions)

The **config-setup factory** command helps the user quickly reset the device to the default configuration.

Default configuration

```
admin@sonic:~$ sudo rm /etc/sonic/config_db.json
admin@sonic:~$ sudo config-setup factory
```

(202111.2) Configure Management Interface

- [management interface](#)
- [static route on management interface](#)

management interface

The management interface in SONiC is eth0.

- to configure management interface ip on eth0 and remember to save it

```
sudo config interface ip add eth0 192.168.1.1/24 192.168.1.254
sudo config save -y
```

- to remove ip from management interface eth0

```
sudo config interface ip remove eth0 192.168.1.1/24
```

- to show current ip and default gateway setting about management interface, use following command. notice that this command only shows what settings user applied

```
show management_interface address
```

```
admin@sonic:~$ show management_interface address
Management IP address = 192.168.8.112/20
Management Network Default Gateway = 192.168.0.254
Management IP address = 2001:db8:2de::e13/64
Management Network Default Gateway = 2001:db8:2de::e15
```

- later ec202012.3 (not included) , there is a sub "**running**" cmd , to show current ip and default gateway configured on linux. this command only for verification purpose. because "show management_interface address" only shows what settings user applied.

```
show management_interface address running
```

```
admin@sonic:~$ show management_interface address running
Management IP address = 192.168.8.112/20
Management Network Default Gateway = 192.168.0.254
Management IP address = 2001:db8:2de::e13/64
Management Network Default Gateway = 2001:db8:2de::e15
```

- to check **management** default gateway, use following command, it shows only route in the default table on kernel

```
ip route show table default
```

it's different from data plane (ASIC) routing table, to show data plane (ASIC) routing table please use:

```
show ip route
```

when mgmt vrf is enabled, to check default gateway please use following command. default gateway will be move to "5000" route table when mgmt vrf is enabled.

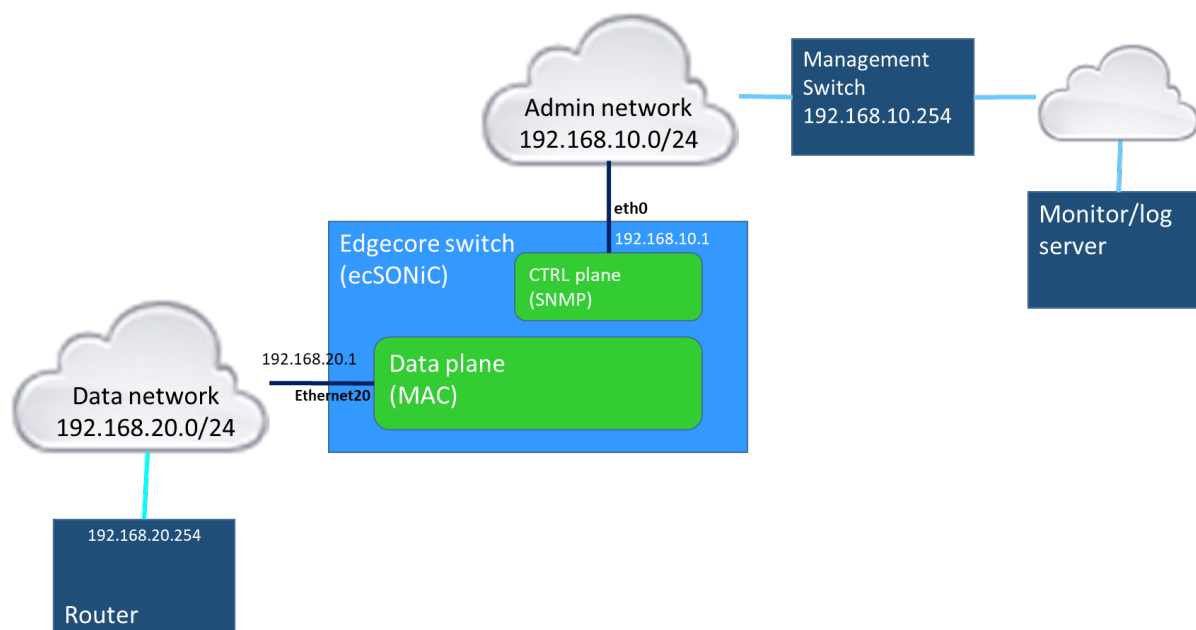
```
ip route show table 5000
```

- **Please notice**, user should **not** to use linux **ip** cmd to add default gateway, it might impact sonic and kernel and frr route entry synchronize scheme.

```
ip route add default via 192.168.1.254 (-->Not recommend)
```

static route on management interface

For some reason, if you want to add a static route on the management interface, you need to be carefully handling the routes on the switch which distinguish the default VRF and management VRF. For example, a network architecture as following: there are two routing interfaces, Ethernet20: 192.168.20.1/24 for data network and eth0: 192.168.10.1/24 for admin network. Due to management purpose, the trap server is located in remote monitor/log server which is behind the management switch 192.168.10.254.



In this application, you may use default gateway in adding IP address command, or use prefix option to specify the route. But please remember to enable management VRF before applying the "prefix" options.

configure default gateway of management interface

```
sudo config vrf add mgmt
sudo config interface ip add eth0 192.168.10.1/24 192.168.10.254
```

or

configure static route on management interface

```
sudo config vrf add mgmt
sudo config interface ip add eth0 192.168.10.1/24
sudo config route add prefix vrf mgmt 0.0.0.0/0 nexthop vrf mgmt 192.168.10.254
```

(202111.2) System Management Configuration

(202111.2) Specifying a Device Name

To specify or modify the name assigned to a switch system, use the **config hostname** command.

Example

```
admin@sonic:~$ sudo config hostname CSW06
```

For more information, see the [\(202111\) Device Hostname](#) section in the CLI reference guide.

(202111.2) CPU and Memory Utilization

SONiC can display CPU and memory utilization for the system as well as currently active processes.

The "show process cpu" command

The **show processes cpu** command displays information about the active processes in the switch, and their corresponding CPU utilization statistics. This command uses the Linux "top -bn 1 -o %CPU" command to display the output. The following is a sample output of the **show processes cpu** command:

Example

```
admin@sonic:~$ show processes cpu
top - 23:50:08 up 1:18, 1 user, load average: 0.25, 0.29, 0.25
Tasks: 161 total, 1 running, 160 sleeping, 0 stopped, 0 zombie
%Cpu(s): 3.8 us, 1.0 sy, 0.0 ni, 95.1 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 8181216 total, 1161060 used, 7020156 free, 105656 buffers
KiB Swap: 0 total, 0 used, 0 free. 557560 cached Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
2047 root 20 0 683772 109288 39652 S 23.8 1.3 7:44.79 syncd
1351 root 20 0 43360 5616 2844 S 11.9 0.1 1:41.56 redis-server
10093 root 20 0 21944 2476 2088 R 5.9 0.0 0:00.03 top
1 root 20 0 28992 5508 3236 S 0.0 0.1 0:06.42 system
2 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.56 ksoftirqd/0
5 root 0 -20 0 0 0 S 0.0 0.0 0:00.00 kworker/0:0H
```

The "show processes memory" command

The **show processes memory** command displays information about the active processes in the switch, and the corresponding memory used. This command uses Linux's "top -bn 1 -o %MEM" command to display the output. The following is a sample output of the **show processes memory** command:

Example

```
admin@sonic:~$ show processes memory
top - 23:41:24 up 7 days, 39 min, 2 users, load average: 1.21, 1.19, 1.18
Tasks: 191 total, 2 running, 189 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.8 us, 20.7 sy, 0.0 ni, 76.3 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
KiB Mem : 8162264 total, 5720412 free, 945516 used, 1496336 buff/cache
KiB Swap: 0 total, 0 free, 0 used. 6855632 avail Mem

  PID USER      PR  NI   VIRT   RES    SHR S  %CPU  %MEM    TIME+  COMMAND
18051 root        20   0  851540 274784   8344 S   0.0   3.4    0:02.77 syncd
17760 root        20   0 1293428 259212  58732 S   5.9   3.2   96:46.22 syncd
  508 root        20   0  725364  76244  38220 S   0.0   0.9    4:54.49 dockerd
30853 root        20   0   96348  56824   7880 S   0.0   0.7    0:00.98 show
17266 root        20   0  509876  49772  30640 S   0.0   0.6    0:06.36 docker
24891 admin       20   0  515864  49560  30644 S   0.0   0.6    0:05.54 docker
17643 admin       20   0  575668  49428  30628 S   0.0   0.6    0:06.29 docker
23885 admin       20   0  369552  49344  30840 S   0.0   0.6    0:05.57 docker
18055 root        20   0  509076  49260  30296 S   0.0   0.6    0:06.36 docker
17268 root        20   0  371120  49052  30372 S   0.0   0.6    0:06.45 docker
 1227 root        20   0  443284  48640  30100 S   0.0   0.6    0:41.91 docker
23785 admin       20   0  443796  48552  30128 S   0.0   0.6    0:05.58 docker
17820 admin       20   0  435088  48144  29480 S   0.0   0.6    0:06.33 docker
  506 root        20   0 1151040  43140  23964 S   0.0   0.5    8:51.08 containerd
18437 root        20   0   84852  26388   7380 S   0.0   0.3   65:59.76 python3.6
```

The following table lists and describes the fields in the **show processes cpu** and **show processes memory** output:

Field	Description
top	Display Linux processes.
load average	The system load over the last one, five, and 15 minutes respectively.
Tasks	Shows total number of processes, and the number of running, sleeping, stopped, and zombie processess.
%cpu(s)	Shows the CPU usage separated by types. The data are the values between screen refreshes. The values are: • us: user processes • sy: system processes • ni: nice user processes • id: the CPU's idle time; a high idle time means there's not a lot going on otherwise • wa: wait time, or time spent waiting for I/O completion • hi: time spent waiting for hardware interrupts • si: time spent waiting for software interrupts • st: time stolen from this VM by the hypervisor
KiB Mem	Memory size
KiB Swap	Swapped size
PID	Process ID
USER	User name of the owner of the task.
PR	The process' s priority. The lower the number, the higher the priority.
NI	Represents a nice value of task. A negative nice value implies higher priority, and a positive nice value means lower priority.
VIRT	Total virtual memory used by the task.
RES	How much physical RAM the process is using, measured in kilobytes.
SHR	Represents the shared memory size (kb) used by a task.
S	Process status. The status of the task which can be one of the following: • D = uninterruptible sleep • I = idle, R = running • S = sleeping • T = stopped by job control signal • t = stopped by debugger during trace • Z = zombie
%CPU	Represents the CPU usage
%MEM	Shows the memory usage of the task.
TIME+	CPU time, the same as "TIME" , but reflecting more granularity through hundredths of a second.
COMMAND	The name of the command that started the process.

For more information, see the [\(202111\) System State](#) section in the CLI reference guide.

(202111.2) Docker Warm Restart

In addition to a device-level warm reboot, SONiC also supports a Docker-based warm restart for the following Dockers: BGP, teamD, and SWSS. This feature enables you to restart a particular Docker with no interruption of packet forwarding and no effect on other services.

The **config warm_restart enable/disable** command is used to enable or disable the warm restart for a particular service that supports it.

When you restart a particular service using "systemctl restart <service_name>", the warm restart configuration is checked to see whether it is enabled or disabled. If the configuration is enabled for that service, a warm reboot is performed. Otherwise, a cold restart is performed for the service.

Example

```
admin@sonic:~$ sudo config warm_restart enable swss

admin@sonic:~$ sudo config warm_restart enable teamd
```

For more detailed information on performing warm restarts, setting timers, and displaying the status, see the [\(202111\) Warm Restart](#) section in the CLI reference guide.

(202111.2) Initiating a Warm Reboot

To initiate a warm reboot of the system in SONiC, use the **warm-reboot** command as shown below.

Example

```
admin@sonic:~$ sudo warm-reboot -v
Tue Oct 22 23:20:53 UTC 2019 Pausing orchagent ...
Tue Oct 22 23:20:53 UTC 2019 Stopping radv ...
Tue Oct 22 23:20:54 UTC 2019 Stopping bgp ...
Tue Oct 22 23:20:54 UTC 2019 Stopped bgp ...
Tue Oct 22 23:20:57 UTC 2019 Initialize pre-shutdown ...
Tue Oct 22 23:20:58 UTC 2019 Requesting pre-shutdown ...
Tue Oct 22 23:20:58 UTC 2019 Waiting for pre-shutdown ...
Tue Oct 22 23:20:59 UTC 2019 Pre-shutdown succeeded ...
Tue Oct 22 23:20:59 UTC 2019 Backing up database ...
Tue Oct 22 23:21:00 UTC 2019 Stopping teamd ...
Tue Oct 22 23:21:00 UTC 2019 Stopped teamd ...
Tue Oct 22 23:21:00 UTC 2019 Stopping syncd ...
Tue Oct 22 23:21:11 UTC 2019 Stopped syncd ...
Tue Oct 22 23:21:11 UTC 2019 Stopping all remaining containers ...
Tue Oct 22 23:21:13 UTC 2019 Stopped all remaining containers ...
Tue Oct 22 23:21:15 UTC 2019 Rebooting with /sbin/kexec -e to SONiC-OS-20191021.01 ...
```

For more information, see the [\(202111\) Warm Reboot](#) section in the CLI reference guide.

(202111.2) Using Container Auto-Restart

The SONiC software is built on a modular concept where features and network applications are installed as “Docker containers.” A container allows an application to run isolated from its environment, and Docker is a standard method for building containers. This enables applications to be added or replaced without impacting the whole SONiC system.

In addition, SONiC allows Docker containers to be automatically shut down and restarted when unexpected conditions occur. Restarting a container ensures that the configuration is reloaded and all processes are restarted and can run in a healthy state.

To enable or disable the auto-restart feature for a container, use the **config feature autorestart** command from the CLI and specify the container name.

Example

```
admin@sonic:~$ sudo config feature autorestart database disabled
```

To display the status of the auto-restart feature for all containers or a specific container, use the **show feature autorestart** command.

Example

```
admin@sonic:~$ show feature autorestart
Container Name  Status
-----
database       enabled
syncd          enabled
teamd          disabled
dhcp_relay     enabled
lldp           enabled
pmon           enabled
bgp            enabled
swss           disabled
telemetry      enabled
sflow          enabled
snmp           enabled
radv           disabled
```

For more information, see the [\(202111\) Container Auto-restart](#) section in the CLI reference guide.

(202111.2) Network Time Protocol (NTP)

The Network Time Protocol (NTP) enables a switch to synchronize its internal clock based on periodic updates from NTP time servers. This synchronized time is used to record accurate dates and times for events on the device.

The multiple NTP time servers can be specified that a switch will poll for regular time updates. All responses received from multiple NTP time servers are filtered and compared to determine the most reliable and accurate time update.

To add a server to a switch's NTP server list, use the **config ntp add** command and specify the IP address. To remove a server IP address from the list, use the **config ntp del** command.

Example

```
admin@sonic:~$ sudo config ntp add 9.9.9.9
NTP server 9.9.9.9 added to configuration
Restarting ntp-config service...

admin@sonic:~$ sudo config ntp del 9.9.9.9
NTP server 9.9.9.9 removed from configuration
Restarting ntp-config service...
```

Use the **show ntp** command to display the status of NTP time updates.

Example

```
admin@sonic:~$ show ntp
synchronised to NTP server (204.2.134.164) at stratum 3
time correct to within 326797 ms
polling server every 1024 s

  remote          refid          st t when poll reach  delay  offset jitter
=====
 23.92.29.245     .XFAC.           16 u   - 1024   0   0.000   0.000   0.000
*204.2.134.164    46.233.231.73    2 u  916 1024 377   3.079   0.394   0.128
```

For more information, see the [\(202111\) NTP](#) section in the CLI reference guide.

(202111.2) System Log (syslog)

Syslog

SONiC supports the logging of error messages from a switch and sending them to a remote System Log (syslog) server. You can specify multiple syslog servers using the **config syslog add** command to add server IP addresses to the syslog server list. To remove a syslog server from the list, use the **config syslog delete** command.

Example

```
admin@sonic:~$ sudo config syslog add 1.1.1.1
Syslog server 1.1.1.1 added to configuration
Restarting rsyslog-config service...

admin@sonic:~$ sudo config syslog del 1.1.1.1
Syslog server 1.1.1.1 removed from configuration
Restarting rsyslog-config service...
```

For more information, see the [\(202111\) Syslog](#) section in the CLI reference guide.

(202111.2) Troubleshooting Information

SONiC can collect relevant information about the device's status for troubleshooting and debugging purposes. This data includes syslog entries, database state, routing-stack state, e.t.c. The EC SONiC development team can review this data to troubleshoot, which has been compressed into an archive file. The syntax of the archived file generated is `"/var/dump/DEVICE HOST NAME> YYYYMMDD HHMMSS.tar.gz"`.

You can use the **show techsupport** command to create an archive file for troubleshooting.

Example

```
admin@sonic:~$ show techsupport --since=yesterday # Will collect
syslog and core files for the last 24 hours
```

```
admin@sonic:~$ show techsupport --since='hour ago' # Will
collect syslog and core files for the last one hour
```

For more information, see the [\(202111\) Troubleshooting](#) section in the CLI reference guide.

(202111.2) Reloading Configuration

config reload

This command is used to clear current configuration and import new configuration from the input file or from /etc/sonic/config_db.json. This command shall stop all services before clearing the configuration and it then restarts those services.

This command restarts various services running in the device and it takes some time to complete the command. NOTE: If the user had logged in using SSH, users **might get disconnected** depending upon the new management IP address. Users need to reconnect their SSH sessions. In general, it is recommended to execute this command from console port after disconnecting all SSH sessions to the device. When users do "config reload" the newly loaded config may have management IP address, or it may not have management IP address. If mgmtIP is there in the newly loaded config file, that mgmtIP might be same as previously configured value or it might be different. This difference in mgmtIP address values results in following possible behaviours.

- Case1: Previously configured mgmtIP is same as newly loaded mgmtIP. The SSH session may not be affected at all, but it's possible that there will be a brief interruption in the SSH session. But, assuming the client's timeout value isn't on the order of a couple of seconds, the session would most likely just resume again as soon as the interface is reconfigured and up with the same IP.
- Case2: Previously configured mgmtIP is different from newly loaded mgmtIP. Users will lose their SSH connections.
- Case3: Newly loaded config does not have any mgmtIP. Users will lose their SSH connections.

NOTE: Management interface IP address and default route (or specific route) may require reconfiguration in case if those parameters are not part of the minigraph.xml.

When user specifies the optional argument "-y" or "--yes", this command forces the loading without prompting the user for confirmation. If the argument is not specified, it prompts the user to confirm whether user really wants to load this configuration file.

When user specifies the optional argument "-n" or "--no-service-restart", this command clear and loads the configuration without restarting dependent services running on the device. One use case for this option is during boot time when config-setup service loads existing old configuration and there is no services running on the device.

When user specifies the optional argument "-f" or "--force", this command ignores the system sanity checks. By default a list of sanity checks are performed and if one of the checks fail, the command will not execute. The sanity checks include ensuring the system status is not starting, all the essential services are up and swss is in ready state.

Usage:

```
config reload [-y|--yes] [-l|--load-sysinfo] [<filename>] [-n|--no-service-restart] [-f|--force]
```

Example:

```
admin@sonic:~$ sudo config reload
Clear current config and reload config from the file /etc/sonic/config_db.json? [y/N]: y
Running command: systemctl stop dhcp_relay
Running command: systemctl stop swss
Running command: systemctl stop snmp
Warning: Stopping snmp.service, but it can still be activated by:
  snmp.timer
Running command: systemctl stop lldp
Running command: systemctl stop pmon
Running command: systemctl stop bgp
Running command: systemctl stop teamd
Running command: /usr/local/bin/sonic-cfggen -H -k Force10-Z9100-C32 --write-to-db
Running command: /usr/local/bin/sonic-cfggen -j /etc/sonic/config_db.json --write-to-db
Running command: systemctl restart hostname-config
Running command: systemctl restart interfaces-config
Timeout, server 10.11.162.42 not responding.
```

Error Condition:

```
admin@sonic:~$ sudo config reload -y
System is not up. Retry later or use -f to avoid system checks

admin@sonic:~$ sudo config reload -y
Relevant services are not up. Retry later or use -f to avoid system checks

admin@sonic:~$ sudo config reload -y
SwSS container is not ready. Retry later or use -f to avoid system checks
```

NOTE : Config reload has two system sanity checks. (1) Check the services which are grouped under delayed target up. (2) Check swss is running for at least 120 seconds. If you want to ignore these two sanity check, please use "config reload -f".

(202111.2) Management Framework

Management framework use REST server to manage configuration and status on SONiC switches.

REST server will provide Swagger UI based online documentation and test UI for all REST APIs it supports.

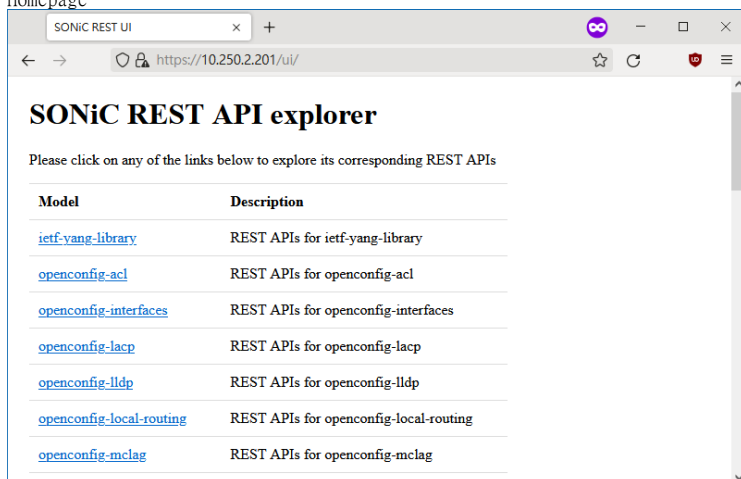
Documentation can be accessed by launching URL https://REST_SERVER_IP/ui in a browser.

This page will list all supported OpenAPI definition files (both YANG generated and manual) along with link to open Swagger UI for them.

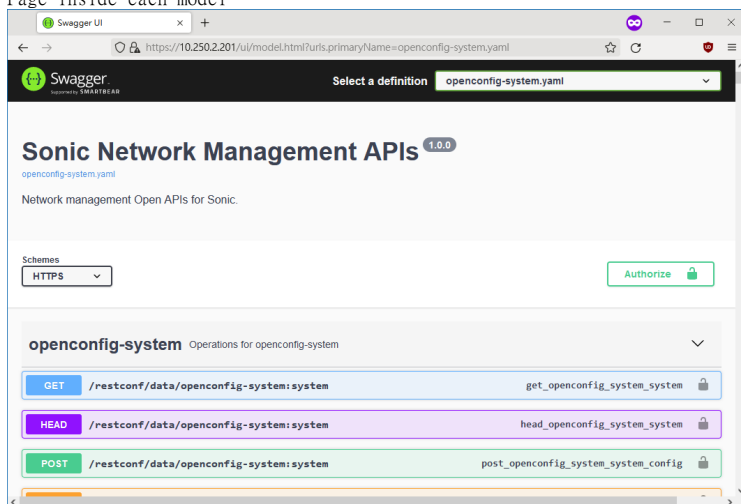
Swagger Web UI

View of web UI

- Homepage



- Page inside each model



Usage

In web UI, user can click item in the page, to expand the corresponding API operation block.

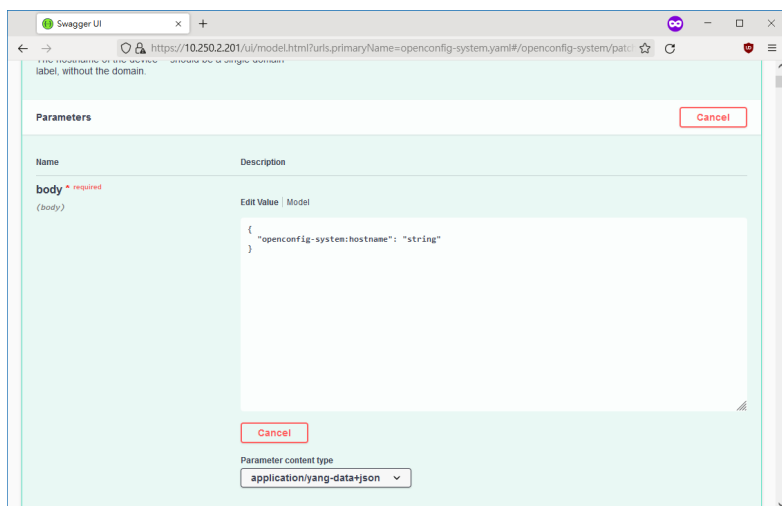
In this block, user can click "Try it out", to fill the necessary parameter and then execute the API operation.

For example, if user want to **GET** the path `/restconf/data/openconfig-system:system/config/hostname`
Click 'Try it out' button, and then click 'Execute' button.
Then the result of this operation will return in this page.

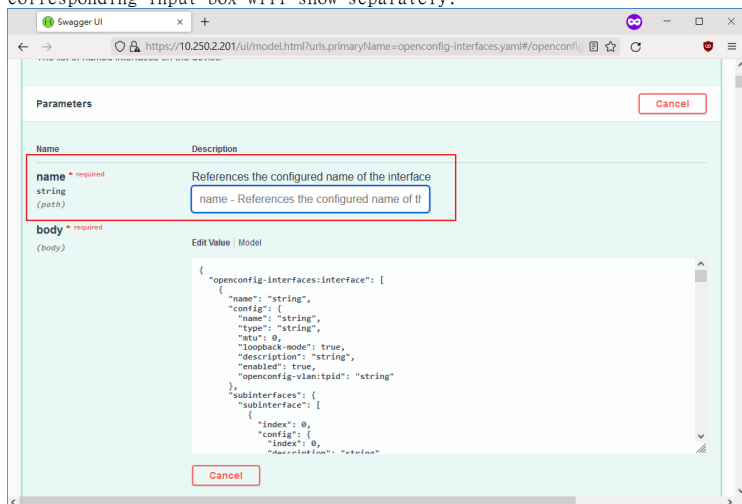
The image consists of three vertically stacked screenshots of the Swagger UI interface, demonstrating the process of executing a REST API call.

- Top Screenshot:** Shows the Swagger UI for the endpoint `GET /restconf/data/openconfig-system:system/config/hostname`. The 'Try it out' button is circled in red.
- Middle Screenshot:** The 'Try it out' button has been clicked, and the 'Execute' button is now visible and circled in red. A 'Cancel' button is also present.
- Bottom Screenshot:** The 'Execute' button has been clicked, and the response is displayed. The response body is `{ "openconfig-system:hostname": "vlab-01" }`. The response headers include `content-length: 40`, `content-type: application/yang-data+json`, `date: Wed, 20 Jul 2022 07:53:22 GMT`, and `x-firefox-spy: h2`.

Some operations need the parameter as payload, which is , such as PATCH, PUT, POST. User needs to fill the parameter in input box. And the input box will be filled in the sample value in advance, user can modify it as user need.



If the parameter is in the request path, such as `/restconf/data/openconfig-interfaces:interfaces/interface={name}`, the corresponding input box will show separately.



REST API by CURL command

Apart from Web UI, user can also use **curl** command to send REST request to do the same operation.

for test, add parameter "-k" to allow insecure server connections when using SSL

Usage of CURL to Access REST API

```
curl -X {METHOD} {URL} -d {payload}
```

- For example, to **GET** the path `/restconf/data/openconfig-system:system/config/hostname`

```
$ curl -k -X GET "https://10.250.2.201/restconf/data/openconfig-system:system/config/hostname" -H
"accept: application/yang-data+json"
{"openconfig-system:hostname":"vlab-01"}
```

- Example for operation with payload

```
$ curl -k -X PATCH "https://10.250.2.201/restconf/data/openconfig-system:system/config/hostname" -H
"accept: application/yang-data+json" -H "Content-Type: application/yang-data+json" -d "{ \"
openconfig-system:hostname\": \"vlab-01-test\"}"
```

- Example of URL path with parameter
path: /restconf/data/openconfig-interfaces:interfaces/interface={name}/openconfig-vlan:routed-vlan/openconfig-if-ip:ipv4
Use parameter "Vlan1000" to replace "{Name}" in the path

```
$ curl -k -X GET "https://10.250.2.201/restconf/data/openconfig-interfaces:interfaces
/interface=Vlan1000/openconfig-vlan:routed-vlan/openconfig-if-ip:ipv4" -H "accept: application/yang-
data+json"
{"openconfig-if-ip:ipv4":{"addresses":{"address":[{"config":{"ip":"192.168.1.1","openconfig-
interfaces-ext:secondary":false,"prefix-length":24},"ip":"192.168.1.1","state":{"ip":"192.168.1.1","
openconfig-interfaces-ext:secondary":false,"prefix-length":24}}]}}}}
```

- Example of both payload and path which contains parameter

```
$ curl -k -X PATCH "https://10.250.2.201/restconf/data/openconfig-interfaces:interfaces
/interface=Vlan1000/openconfig-vlan:routed-vlan/openconfig-if-ip:ipv4" -H"accept: application/yang-
data+json" -H"Content-Type: application/yang-data+json" -d "{\"openconfig-if-ip:ipv4\": {\"
addresses\": {\"address\": [{\"ip\": \"192.168.1.1\", \"config\": {\"ip\": \"192.168.1.1\", \"prefix-
length\": 24}}]}}}"
```

Supported REST APIs

The above is the usage and example, the full supported REST APIs list can access the following link:

- [202111 REST APIs - EdgeCore Switch SW RD - Confluence \(accton.com\)](#)

(202111.2) MIB & Trap Support List

- Standard MIB and Trap
- Private MIB

Standard MIB and Trap

MIB List:

	MIB	RFC	OID	status
1	lldpMIB (ieee8021lldp)	IEEE 802.1AB	1.0.8802.1.1.2	only 1.0.8802.1.1.2.1.3.1~8 1.0.8802.1.1.2.1.4.1.1.4~12 1.0.8802.1.1.2.1.4.2.1.3~5
2	mib-2 (mib)	RFC 1213	1.3.6.1.2.1	only 1.3.6.1.2.1.1.1~9 1.3.6.1.2.1.3.1.1.1~3 1.3.6.1.2.1.4.1.1.2,25,26,31~37 1.3.6.1.2.1.4.21.1.7 1.3.6.1.2.1.4.22.1.2 1.3.6.1.2.1.4.34.1.3.1 1.3.6.1.2.1.5.1~30 1.3.6.1.2.1.6.1~15,19,20 1.3.6.1.2.1.7.1~5,7 1.3.6.1.2.1.10.7.2 1.3.6.1.2.1.11.1~32 1.3.6.1.2.1.15.1~4 1.3.6.1.2.1.25.1~6 1.3.6.1.2.1.28.1~2 1.3.6.1.2.1.55.1 1.3.6.1.2.1.88.1
3	snmpModules	RFC 2578	1.3.6.1.6.3	only 1.3.6.1.6.3.1,10~12,15,16
4	ifMIB	RFC 2863	1.3.6.1.2.1.2	only 1.3.6.1.2.1.2.1 1.3.6.1.2.1.2.2.1.1~21 1.3.6.1.2.1.31.1.1.1~19
5	ipCidrRouteDest (IpCidrRouteTable)	RFC 4292	1.3.6.1.2.1.4.24.4.1.1	only 1.3.6.1.2.1.4.24.4.1.1,16
6	dot1qTpFdbPort (in Q-BRIDGE-MIB)	RFC 4363	1.3.6.1.2.1.17.7.1.2.2.1.2	only 1.3.6.1.2.1.17.7.1.2.2.1.2
7	entityMIB	RFC 2737 (v2)	1.3.6.1.2.1.47	only 1.3.6.1.2.1.47.1.1.1.1.2~16
8	entitySensorMIB	RFC 3433	1.3.6.1.2.1.99	only 1.3.6.1.2.1.99.1.1.1.1~5

Trap List:

Trap Name	snmpTrapOID
coldStart	1.3.6.1.6.3.1.1.5.1
linkDown	1.3.6.1.6.3.1.1.5.3
linkUp	1.3.6.1.6.3.1.1.5.4
authenticationFailure	1.3.6.1.6.3.1.1.5.5
entConfigChange	1.3.6.1.2.1.47.2.0.1

ospfVirtIfStateChange	1.3.6.1.2.1.14.16.2.1
ospfIfStateChange	1.3.6.1.2.1.14.16.2.16
ospfNbrStateChange	1.3.6.1.2.1.14.16.2.2
ospfVirtNbrStateChange	1.3.6.1.2.1.14.16.2.3
bgpEstablishedNotification	1.3.6.1.2.1.15.0.1
bgpBackwardTransNotification	1.3.6.1.2.1.15.0.2

Private MIB

	MIB	File	OID	Note
1	AcctonProdFamily-OP	AcctonProdFamily_OP.MIB	1.3.6.1.4.1.259.11	define sysObjectOID of models

(202111.2) Interface Configuration

(202111.2) Shutting Down and Starting Up Interfaces

Use the **config interface shutdown** command to administratively shut down either a physical interface or port-channel interface and to enable a physical or port-channel interface, use the **config interface startup** command. These two commands will also allow configuring multiple interfaces in one command.

- [Shutting Down The Interface](#)
- [Starting Up the Interfaces](#)

Shutting Down The Interface

Step1: Use the **show interfaces status** command to check the status of the interfaces.

Example

```
admin@sonic:~$ show interfaces status
```

Interface Type	Asym	PFC	Lanes Oper	Speed Speed	MTU	FEC	Alias	Vlan	Oper	Admin
up			Ethernet0 RJ45	2 off	0M	9100	N/A	Eth1(Port1)	routed	up
up			Ethernet1 RJ45	1 off	0M	9100	N/A	Eth2(Port2)	trunk	up
up			Ethernet2 RJ45	4 off	0M	9100	N/A	Eth3(Port3)	trunk	up
up			Ethernet3 RJ45	3 off	0M	9100	N/A	Eth4(Port4)	trunk	up
up			Ethernet4 RJ45	6 off	0M	9100	N/A	Eth5(Port5)	trunk	up
up			Ethernet5 RJ45	5 off	0M	9100	N/A	Eth6(Port6)	trunk	up
up			Ethernet6 RJ45	8 off	0M	9100	N/A	Eth7(Port7)	trunk	up
up			Ethernet7 RJ45	7 off	0M	9100	N/A	Eth8(Port8)	trunk	up

Step 2: Use the **config interface shutdown** command to shut down either one interface, or multiple interfaces as shown below.

Example

```
admin@sonic:~$ sudo config interface shutdown Ethernet1
admin@sonic:~$ sudo config interface shutdown Ethernet2,Ethernet3,Ethernet4,Ethernet5
```

Step 3: Once interfaces are configured, use the **show interfaces status** command to check the status.

Example

```
admin@sonic:~$ show interfaces status Ethernet1,Ethernet2,Ethernet3,Ethernet4,Ethernet5
```

Interface Speed	Lanes	Speed	MTU	FEC	Alias	Vlan	Oper	Admin	Type	Asym PFC	Oper
Ethernet1 off	1 10G	0M	9100	N/A	Eth2(Port2)	trunk	down	down	RJ45		
Ethernet2 off	4 10G	0M	9100	N/A	Eth3(Port3)	trunk	down	down	RJ45		
Ethernet3 off	3 10G	0M	9100	N/A	Eth4(Port4)	trunk	down	down	RJ45		
Ethernet4 off	6 10G	0M	9100	N/A	Eth5(Port5)	trunk	down	down	RJ45		
Ethernet5 off	5 10G	0M	9100	N/A	Eth6(Port6)	trunk	down	down	RJ45		

Starting Up the Interfaces

Step 4: Use the `config interface startup` command to enable either one interface, or multiple interfaces as shown below.

Example

```
admin@sonic:~$ sudo config interface startup Ethernet1
admin@sonic:~$ sudo config interface startup Ethernet2,Ethernet3,Ethernet4,Ethernet5
```

Step 5: Once interfaces are configured, use the `show interfaces status` command again to check the status.

Example

```
admin@sonic:~$ show interfaces status Ethernet1,Ethernet2,Ethernet3,Ethernet4,Ethernet5
```

Interface Speed	Lanes	Speed	MTU	FEC	Alias	Vlan	Oper	Admin	Type	Asym PFC	Oper
Ethernet1 off	1 10G	0M	9100	N/A	Eth2(Port2)	trunk	up	up	RJ45		
Ethernet2 off	4 10G	0M	9100	N/A	Eth3(Port3)	trunk	up	up	RJ45		
Ethernet3 off	3 10G	0M	9100	N/A	Eth4(Port4)	trunk	up	up	RJ45		
Ethernet4 off	6 10G	0M	9100	N/A	Eth5(Port5)	trunk	up	up	RJ45		
Ethernet5 off	5 10G	0M	9100	N/A	Eth6(Port6)	trunk	up	up	RJ45		

For more information, see the [\(202111\) Interfaces](#) section in the CLI reference guide.

(202111.2) Configuring Interface IP Addresses

SONiC can configure an IP address for a physical interface, port-channel, VLAN interface, or loopback interface using the **config interface ip add** command. To specify a VLAN, the VLAN ID replaces the interface name.

When configuring an IP address for the management interface "eth0", you can also provide the default gateway IP address as an optional parameter.

To remove an IP address from an interface, use the **config interface remove** command.

Example

```
admin@sonic:~$ sudo config interface ip add Ethernet63
10.11.12.13/24
admin@sonic:~$ sudo config interface ip add eth0 20.11.12.13/24
20.11.12.254

admin@sonic:~$ sudo config interface ip add vlan100 10.11.12.13/24

admin@sonic:~$ sudo config interface ip remove Ethernet63
10.11.12.13/24
```

For more information, see the [\(202111\) Interfaces](#) section in the CLI reference guide.

(202111.2) Link Aggregation Control Protocol (LACP)

SONiC supports dynamic Link Aggregation Control Protocol (LACP) for an automatic port trunk configuration. LACP-configured ports can automatically negotiate a trunked link with an LACP-configured port on another device. You can configure any number of ports on a switch as LACP, and if ports on another device are also configured as LACP, the switch and the other device will negotiate a trunk link (aggregated link or port channel) between them.

To configure, add, or delete a port trunk identifier, use the **config portchannel** command. The identifier must be in the format "PortChannelxxxx", where "xxxx" is a numeric value that can have one to four digits. For example, "PortChannel0002".

The **config portchannel** command takes the following three optional arguments:

- **min-links** - The minimum number of links required to bring up the aggregated link.
- **fallback** - true/false. When the LACP fallback feature is set to true, one member port per port channel will be selected as active during the fallback mode. The LACP fallback feature allows an active LACP-enabled port to establish an aggregated link (LAG) before it receives an LACP PDUs from its peer.
- **lACP-key** - auto/number of lACP key. Set the generated or given number as the lACP key, the default value is 'auto'.

For more information, see the [\(202111\) Port Channels](#) section in the CLI reference guide.

Once a trunk identifier has been configured, you can add/remove member ports to it using the **config portchannel member** command as shown below.

Example

```
admin@sonic:~$ sudo config portchannel add PortChannel0018

admin@sonic:~$ sudo config portchannel member add PortChannel0018 Ethernet12
admin@sonic:~$ show interfaces portchannel
Flags: A - active, I - inactive, Up - up, Dw - Down, N/A - not available,
       S - selected, D - deselected, * - not synced
```

No.	Team Dev	Protocol	Ports	Oper Key	Admin Key
24	PortChannel24	LACP(A)(Up)	Ethernet28(S) Ethernet24(S)	124	auto
48	PortChannel48	LACP(A)(Up)	Ethernet52(S) Ethernet48(S)	48	48
40	PortChannel40	LACP(A)(Up)	Ethernet44(S) Ethernet40(S)	40	40
0	PortChannel0	LACP(A)(Up)	Ethernet0(S) Ethernet4(S)	1	1
8	PortChannel8	LACP(A)(Up)	Ethernet8(S) Ethernet12(S)	8	8

```
admin@sonic:~$
```

Note

The **Oper Key** and **Admin Key** of portchannel are supported from 202012.4 (Edgecore-SONiC_20220623_052151_ec202012_362, Edgecore-SONiC_20220623_051044_ec202012_bfn_244) versions. These columns may not be available in the older versions of EC SONiC.

(202111.2) Configuring Port Mirroring

Port mirroring allows you to mirror the traffic from any switch source port to a destination port for real-time analysis. SONiC supports two types of mirroring:

- **SPAN:** Switch Port Analyzer (SPAN) is the local mirroring of traffic from one or more source ports on a switch to the destination port on the same switch.
- **ERSPAN:** Encapsulated Remote SPAN (ERSPAN) supports the mirroring of traffic from source ports on multiple switches over a Layer 3 network. ERSPAN uses Generic Routing Encapsulation (GRE) to tunnel the mirrored traffic across the network to the switch that contains the destination port.

To add or remove a SPAN or ERSPAN mirror session in SONiC, use the **config mirror_session** command. Using this command, each mirror session is identified by a session name. In addition, both SPAN and ERSPAN support ACL-based mirroring and can be used in ACL configurations.

To configure a SPAN mirror session, specify the source ports/LAG and destination port, and then select whether you want to mirror ingress/egress traffic or both.

To configure an ERSPAN mirror session, users need to configure the following fields that are used to forward the mirrored packets:

- Source IP address
- Destination IP address
- DSCP (QoS) value with which mirrored packets are forwarded
- TTL value
- Optional - GRE type in the GRE tunnel packet. The range is 0..0xFFFF, although a decimal format is also accepted. The default value is 0x88BE.
- Optional - Queue in which packets shall be sent out of the device. This parameter accepts 0 only in the current implementation.
- Optional - List of source ports that can include both Ethernet and LAG ports.
- Optional - Direction: The mirror session direction when configured along with the source port (Supports rx/tx/both. The default direction is both).
- Optional - Policer that is used to control the rate at which frames are mirrored.

Example

```
admin@sonic:~# config mirror_session span add port0 Ethernet0 Ethernet4,PortChannel10,Ethernet8

admin@sonic:~# config mirror_session erspan add mrr_port 1.2.3.4 20.21.22.23 8 100 0x6558 0 Ethernet0

admin@sonic:~# show mirror_session
ERSPAN Sessions
Name      Status  SRC IP      DST IP      GRE      DSCP      TTL      Queue      Policer      Monitor Port  SRC Port
Direction
-----
mrr_port  inactive 1.2.3.4     20.21.22.23 0x6558    8 100      0
Ethernet0 both

SPAN Sessions
Name      Status  DST Port      SRC Port      Direction      Queue      Policer
-----
port0     active  Ethernet0     Ethernet4,PortChannel10,Ethernet8 both
admin@sonic:~#
```

For more information on port mirroring, see the [\(202111\) Mirroring](#) section in the CLI reference guide.

(202111.2) Configure Interface Speed, Breakout Mode and Auto-Negotiation

- [Configure Interface Speed](#)
- [Configure Interface Breakout Mode](#)
- [Configure Interface Auto-Negotiation](#)

Configure Interface Speed

To configure an interface speed, you need to know the speeds supported by the interface, and then configure the interface with a valid value. The value of 10000, 40000, and 100000 indicates 10G, 40G, and 100G respectively.

Use the CLI command **config interface speed** to specify an interface speed, as shown below.

Example (configure interface speed with command)

```
admin@sonic:~$ sudo config interface speed Ethernet10 40000
```

Or, you can modify the interface speed in the configuration file **config_db.json**.

Example (configure interface speed in configuration file)

```
"PORT": {
  "Ethernet10": {
    "alias": "Eth1/1",
    "description": "Eth1/1",
    "index": "1",
    "lanes": "2",
    "mtu": "9100",
    "parent_port": "Ethernet0",
    "pfc_asym": "off",
    "speed": "40000"
  },
}
```

Use the **show interfaces status** command to check the configuration.

Example (show interface status)

```
admin@sonic:~$ show interfaces status Ethernet10
```

Interface	Lanes	Speed	MTU	FEC	Alias	Oper	Admin
Ethernet10	101,102	40G	9100	rs	fortyGigE1/1/1	up	up



Note

An interface will not be linked up if the value in the configuration file is not a supported value.

Configure Interface Breakout Mode

The port breakout feature allows an interface to extend switch ports by splitting a port into two or four lower-speed ports using breakout cable. For example, to split one 100G port (1x100G) into four 25G interfaces (4x25G), use the **show interfaces breakout** command to see the available breakout modes for the interface.

Example (show interfaces breakout)

```
admin@lnos-x1-a-fab01:~$ show interfaces breakout
{
  "Ethernet0": {
    "index": "1,1,1,1",
    "default_brkout_mode": "1x100G[40G]",
    "child ports": "Ethernet0",
    "child port speed": "100G",
    "breakout_modes": "1x100G[40G],2x50G,4x25G[10G]",
    "Current Breakout Mode": "1x100G[40G]",
    "lanes": "65,66,67,68",
    "alias_at_lanes": "Eth1/1, Eth1/2, Eth1/3, Eth1/4"
    ....
  }
}
```

You can use the **config interface breakout** command to select the mode you need.

Example

```
admin@sonic:~$ sudo config interface breakout Ethernet0
1x100G[40G] 2x50G      4x25G[10G]

admin@sonic:~$ sudo config interface breakout Ethernet0 4x25G[10G]
```



Note

The breakout mode should match the breakout cable used in the connection. For example, when selecting the 4x25G mode, you must use a 4x25G breakout cable.

If the following error message appears while configuring the interface breakout, It may be that a field is missing from the config_db.json.

Error

```
sonic_yang(6):Note: Below table(s) have no YANG models: KDUMP, REST_SERVER, SNMP, SNMP_COMMUNITY, XCVRD_LOG
libyang[0]: Leafref "/sonic-mgmt_port:sonic-mgmt_port/sonic-mgmt_port:MGMT_PORT/sonic-mgmt_port:
MGMT_PORT_LIST/sonic-mgmt_port:name" of value "eth0" points to a non-existing leaf. (path: /sonic-
mgmt_interface:sonic-mgmt_interface/MGMT_INTERFACE/MGMT_INTERFACE_LIST[name='eth0'][ip_prefix='192.168.1.1
/24']/name)
sonic_yang(3):Data Loading Failed:Leafref "/sonic-mgmt_port:sonic-mgmt_port/sonic-mgmt_port:MGMT_PORT/sonic-
mgmt_port:MGMT_PORT_LIST/sonic-mgmt_port:name" of value "eth0" points to a non-existing leaf.
Data Loading Failed
Leafref "/sonic-mgmt_port:sonic-mgmt_port/sonic-mgmt_port:MGMT_PORT/sonic-mgmt_port:MGMT_PORT_LIST/sonic-
mgmt_port:name" of value "eth0" points to a non-existing leaf.
ConfigMgmt Class creation failed
Failed to break out Port. Error: Failed to load the config. Error: ConfigMgmtDPB Class creation failed
```

The above error indicates that the **MGMT_PORT** field, which is typically referred to by MGMT INTERFACE is missing. The MGMT_INTERFACE is configured, when user configure IP address in the management interface (eth0).

It can be fixed by adding the following field to config_db.json.

MGMT_PORT

```
"MGMT_PORT": {  
  "eth0": {  
    "alias": "eth0",  
    "admin_status": "up",  
    "description": "Management Port"  
  }  
}
```

Configure Interface Auto-Negotiation

Refer to [\(202111.2\) Auto-Negotiation](#)

(202111.2) Link-Training

- [User Guide:](#)
 - [Enable Link-Training](#)
 - [Show Link-Training](#)
 - [Disable Link-Training](#)
 - [Show Link-Training](#)
 - [LT Admin](#)
 - [LT Oper](#)
 - [Feature support information](#)
 - [Note](#)

User Guide:

Link training is a process by which the transmitter and receiver on a high-speed serial link communicate with each other to tune the equalization settings. In theory, link training enables automatic tuning of the FIR filter for each channel in an ASIC to achieve the desired bit error rate (BER). The IEEE 802.3 standard defines a set of link training protocols for various mediums, and the feature in this user guide is to focus on IEEE clause 72 and 93 to dynamically improve the link quality over the SFP **coppers** and **backplanes**. This feature could be activated with or without port auto negotiation. In other words, the link-training may or may not be disabled when auto-negotiation is activated. It depends on the switch ASIC limitations in the individual SAI implementation. Therefore, this user guide only defines the Link-Training behavior when auto-negotiation be disabled.

Use the **config interface link-training** command to administratively enable or disable link-training at a physical interface.

Enable Link-Training

Step1: Use "show interfaces link-training status" command to check the link-training configuration and status.

Display link-training configuration and status

```
admin@sonic:~$ show interfaces link-training status
Interface    LT Oper    LT Admin    Oper    Admin
-----
Ethernet0    off        N/A         up       up
Ethernet4    off        off         up       up
Ethernet8    off        on          down     up
...
```

Step 2: Use "config interface link-training <interface_name> on" command to enable link-training.

Enable link-training at Ethernet4

```
admin@sonic:~$ sudo config interface link-training Ethernet4 on
```

Show Link-Training

Step 3: Once configured, use "show interfaces link-training status" command to check the link-training configuration and status.

Display link-training configuration and status

```
admin@sonic:~$ show interfaces link-training status
Interface    LT Oper    LT Admin   Oper    Admin
-----
Ethernet0    off        N/A        up      up
Ethernet4    off        on         down    up
Ethernet8    off        on         down    up
...
```

Disable Link-Training

Step 4: Use "config interface link-training <interface_name> off" command to disable link-training.

Disable link-training at Ethernet4

```
admin@sonic:~$ sudo config interface link-training Ethernet4 off
```

Show Link-Training

Step 5: Once configured, use "show interfaces link-training status" command to check the link-training configuration and status.

Display link-training configuration and status

```
admin@sonic:~$ show interfaces link-training status
Interface    LT Oper    LT Admin   Oper    Admin
-----
Ethernet0    off        N/A        up      up
Ethernet4    off        off        up      up
Ethernet8    off        on         down    up
...
```

LT Admin

LT Admin	Link-Training configuration
N/A	Not configured, default configuration
on	Link-Training is enabled by user configuration
off	Link-Training is disabled by user configuration

LT Oper

LT Oper	Description
off	Disabled
on	Enabled, while the further operational status is not yet available
trained	Enabled, while the pre-emphasis is tuned successfully
not_trained	Enabled, while the pre-emphasis is not yet tuned, no further failure is available

frame_lock	Enabled, while the training frame delineation is detected
snr_low	Enabled, while the SNR low threshold is detected
timeout	Enabled, while the training process is timed out

Feature support information

Chip Family	DUT	Support or not
BCM	AS4630-54PE	O
	AS4630-54TE	
BCM	AS4630-54NPE	O
BCM	AS5812-54X	X
BCM	AS5835-54T	X
BCM	AS5835-54X	X
BCM	AS7326-56X	O
BCM	AS7712-32X	O
BCM	AS7726-32X	O
BCM	AS7816-64X	O
BCM	AS9716-32D	O
BCM	AS9726-32D	O
BCM	AS9736-64D	O
Intel	WEDGE100BF-32X	X
Intel	WEDGE100BF-32QS	X
Intel	WEDGE100BF-65X	X
Intel	AS9516-32D	X

Note

1. The above configuration works when port auto-negotiation is disabled.
2. If the port auto-negotiation is enabled, the link-training configuration shall be auto-enabled.
3. If link-training configuration is enabled on port auto-negotiation enabled, the configuration shall be saved and reapplied when the port auto-negotiation configuration switch to disabled.

(202111.2) Configure Laser Frequency and TX Power for ZR Transceiver

- [Configure transceiver laser frequency](#)
- [Configure transceiver TX power](#)

Configure transceiver laser frequency

You should set both frequency and grid space in this CLI command. Now the supported grid space are only 75GHz and 100GHz.

Example (configure transceiver laser frequency with command)

```
admin@sonic:~$ sudo config interface transceiver frequency Ethernet0 193200 100
Setting laser frequency to 193200 GHz and grid space to 100 GHz on port Ethernet0
```

It needs some time to modify transceiver frequency due to the data-path is needed to re-initialize. You can check the initialization state by following command.

Example (show transceiver states with command)

```
admin@sonic:~$ show logging CMIS
Nov 1 03:46:56.709089 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=INSERTED,
retries=0
Nov 1 03:46:57.304001 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: force Datapath reinit
Nov 1 03:46:57.380135 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=DP_DEINIT,
retries=0
Nov 1 03:46:57.509658 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: unable to enter low-power mode
Nov 1 03:46:57.672149 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=DP_DEINIT,
retries=1
Nov 1 03:46:59.945924 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=AP_CONFIGURED,
retries=1
Nov 1 03:47:49.136637 sonic NOTICE pmon#xcvrd: message repeated 48 times: [ CMIS: Ethernet0: 400G,
lanemask=0xff, state=AP_CONFIGURED, retries=1]
Nov 1 03:47:49.136637 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0 configured laser frequency 193200 GHz grid
space 100 GHz
Nov 1 03:47:50.293814 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=DP_INIT,
retries=1
Nov 1 03:47:51.398751 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=DP_TXON,
retries=1
Nov 1 03:48:09.655975 sonic NOTICE pmon#xcvrd: message repeated 17 times: [ CMIS: Ethernet0: 400G,
lanemask=0xff, state=DP_TXON, retries=1]
Nov 1 03:48:09.655975 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=DP_ACTIVATION,
retries=1
Nov 1 03:48:09.667160 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: READY
```

When the data-path is ready you can use the show command to check the active frequency and grid space.

Example (show transceiver laser frequency with command)

```
admin@sonic:~$ sudo show interfaces transceiver frequency Ethernet0
Port      Frequency  Grid space
-----
Ethernet0 193200GHz  100GHz
```

Configure transceiver TX power

The tx power can be configured by following CLI command. If the parameter is a negative, it needs to add "--" before the negative number.

Example (configure transceiver tx power with command)

```
admin@sonic:~$ sudo config interface transceiver tx_power Ethernet0 -- -10
Setting target Tx output power to -10.0 dBm on port Ethernet0
```

It doesn't need to re-initialize the data-path for TX power modified.

Example (show transceiver state with command)

```
admin@sonic:~$ sudo show logging CMIS
Nov  1 04:00:53.672734 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: 400G, lanemask=0xff, state=INSERTED,
retries=0
Nov  1 04:00:54.816412 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0 Successfully configured Tx power = -9.0
Nov  1 04:00:55.183279 sonic NOTICE pmon#xcvrd: CMIS: Ethernet0: READY
```

You can check the TXPOWER in transceiver eeprom digital optical monitoring (DOM) data.

Example (show transceiver tx power with command)

```
admin@sonic:~$ sudo show interfaces transceiver eeprom Ethernet0 --dom
Ethernet0: SFP EEPROM detected
  Active App Selection Host Lane 1: 1
  Active App Selection Host Lane 2: 1
  Active App Selection Host Lane 3: 1
  Active App Selection Host Lane 4: 1
  Active App Selection Host Lane 5: 1
  Active App Selection Host Lane 6: 1
  Active App Selection Host Lane 7: 1
  Active App Selection Host Lane 8: 1
  Active Firmware Version: 1.9
  Application Advertisement:
    1: 400GAUI-8 C2M (Annex 120E) | 400ZR, DWDM, amplified
  CMIS Revision: 5.0
  Connector: LC
  Encoding: N/A
  Extended Identifier: Power Class 8 (19.0W Max)
  Extended RateSelect Compliance: N/A
  Hardware Revision: 0.0
  Host Electrical Interface: 400GAUI-8 C2M (Annex 120E)
  Host Lane Assignment Options: 1
  Host Lane Count: 8
  Identifier: QSFP-DD Double Density 8X Pluggable Transceiver
  Inactive Firmware Version: 1.9
  Length Cable Assembly(m): 0.0
  Media Interface Code: 400ZR, DWDM, amplified
  Media Interface Technology: C-band tunable laser
  Media Lane Assignment Options: 1
  Media Lane Count: 1
  Nominal Bit Rate(100Mbps): 0
  Specification compliance: sm_media_interface
  Supported Max Laser Frequency: 196100GHz
  Supported Max TX Power: -8.0dBm
  Supported Min Laser Frequency: 191300GHz
  Supported Min TX Power: -15.0dBm
  Vendor Date Code(YYYY-MM-DD Lot): 2022-05-23 00
  Vendor Name: NeoPhotonics
  Vendor OUI: 00-15-06
  Vendor PN: QDDZR400100C1000
```

```
Vendor Rev: 00
Vendor SN: EVT2122019
ChannelMonitorValues:
    RX1Power: -11.57390760389438dBm
    RX2Power: -infdBm
    RX3Power: -infdBm
    RX4Power: -infdBm
    RX5Power: -infdBm
    RX6Power: -infdBm
    RX7Power: -infdBm
    RX8Power: -infdBm
    TX1Bias: 283.6mA
    TX1Power: -8.917733436250714dBm
    TX2Bias: 0.0mA
    TX2Power: -infdBm
    TX3Bias: 0.0mA
    TX3Power: -infdBm
    TX4Bias: 0.0mA
    TX4Power: -infdBm
    TX5Bias: 0.0mA
    TX5Power: -infdBm
    TX6Bias: 0.0mA
    TX6Power: -infdBm
    TX7Bias: 0.0mA
    TX7Power: -infdBm
    TX8Bias: 0.0mA
    TX8Power: -infdBm
ChannelThresholdValues:
    RxPowerHighAlarm : 4.0dBm
    RxPowerHighWarning: 2.0dBm
    RxPowerLowAlarm   : -20.0dBm
    RxPowerLowWarning : -18.013dBm
    TxBiasHighAlarm   : 470.0mA
    TxBiasHighWarning : 400.0mA
    TxBiasLowAlarm    : 20.0mA
    TxBiasLowWarning  : 50.0mA
    TxPowerHighAlarm  : -4.0dBm
    TxPowerHighWarning: -6.002dBm
    TxPowerLowAlarm   : -19.031dBm
    TxPowerLowWarning : -17.011dBm
ModuleMonitorValues:
    Temperature: 70.086C
    Vcc: 3.155Volts
ModuleThresholdValues:
    TempHighAlarm : 80.0C
    TempHighWarning: 78.0C
    TempLowAlarm : -3.0C
    TempLowWarning : 0.0C
    VccHighAlarm : 3.5Volts
    VccHighWarning : 3.45Volts
    VccLowAlarm : 3.085Volts
    VccLowWarning : 3.135Volts
```

(202111.2) Auto-Negotiation

- [Capability](#)
- [General Guide](#)
 - [Configure Interface Auto-Negotiation](#)
 - [CL28/CL37](#)
 - [CL28 Limitation](#)
 - [CL73](#)

Capability

O: Support

DEVICE	CL28	CL37	CL73	UserGuide
AS4630-54NPE	0		0	AutoNego User Guide for AS4630-54NPE/AS4630-54PE/AS4630-54TE
AS4630-54PE	0		0	AutoNego User Guide for AS4630-54NPE/AS4630-54PE/AS4630-54TE
AS4630-54TE	0		0	AutoNego User Guide for AS4630-54NPE/AS4630-54PE/AS4630-54TE
AS5812-54X		0	0	(202111.2) AutoNego User Guide for AS5812-54X
AS583-54T	0		0	(202111.2) AutoNego User Guide for AS5835-54X/AS5835-54T
AS5835-54X			0	(202111.2) AutoNego User Guide for AS5835-54X/AS5835-54T
AS7326-56X			0	(202111.2) AutoNego User Guide for AS7326-56X
AS7712-32X			0	(202111.2) AutoNego User Guide for AS7712-32X
AS7726-36X			0	(202111.2) AutoNego User Guide for AS7726-32X
AS9716-32D			0	(202111.2) AutoNego User Guide for AS9716-32D
AS9726-32D			0	(202111.2) AutoNego User Guide for AS9726-32D
AS9736-64D			0	AutoNego User Guide for AS9736-64D

General Guide

Configure Interface Auto-Negotiation

To enable auto-negotiation, you can specify a set of port speeds using the `config interface advertised-speeds` command that are referred to when auto-negotiating with the peer interface, or "all" speeds supported by the (port) interface if you do not configure the "advertised speeds". The command sequence can be inter-changed, but the latest "advertised speed" command will replace a former "advertised speeds" in the sequence. For example, in the sequence (1) configure advertised speeds (1G, 10G), (2) `enable autonego`, (3) configure advertised speed (100M, 1G), The final active advertised speed is (100M, 1G). The similar behavior also be applied to `config interface advertised-types` command.

CL28/CL37

1. [Enable Auto-Negotiation](#).
2. [Config interface advertised-speeds](#).
3. Refer to [\(202111.2\) AutoNego User Guide for AS5812-54X](#) for CL37 limitation.

Example of commands

```
admin@sonic:~$ sudo config interface autoneg Ethernet0 enabled
admin@sonic:~$ sudo config interface advertised-speeds Ethernet0 100,1000,2500
admin@sonic:~$ sudo config interface advertised-speeds Ethernet0 all
```

CL28 Limitation

There is a statement about [auto-negotiation interoperation request in IEEE 802.3](#), "All 1000BASE-T PHYs shall provide support for Auto-Negotiation (Clause 28) and shall be capable of operating as MASTER or SLAVE. Auto-Negotiation is performed as part of the initial set-up of the link, and allows the PHYs at each end to advertise their capabilities (speed, PHY type, half or full duplex) and to automatically select the operating mode for communication on the link."



In the current implementation, the `config interface advertised-speeds` command advertises the port speed capacity that is applied on copper interfaces, but not half/full duplex or master/slave options in auto-negotiation. That is, (a) enabling auto-negotiation on a fiber interface does not take effect, and (b) you cannot specify a master (or slave) in forced mode. One solution for copper interface configuration is to specify the advertised speeds and enable auto-negotiation to activate the port link at the expected speed.

CL73

1. [Enable Auto-Negotiation](#).
2. [Config interface advertised-speeds](#).
3. [Config interface advertised-types](#).
4. Note that the XCVR type shall be configured by [Config interface advertised-types](#) command, or the function may not work and the port may not link-up.

Example of commands

```
admin@sonic:~$ sudo config interface autoneg Ethernet0 enabled
admin@sonic:~$ sudo config interface advertised-speeds Ethernet0 100000,40000
admin@sonic:~$ sudo config interface advertised-types Ethernet0 CR4,KR4
admin@sonic:~$ sudo config interface advertised-types Ethernet0 all
```

(202111.2) AutoNego User Guide for AS4630-54NPE/AS4630-54PE/AS4630-54TE

- [AS4630-54PE/AS4630-54TE Capability](#)
- [AS4630-54NPE Capability](#)
- [Guide](#)

AS4630-54PE/AS4630-54TE Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x1G	1000,100,10	None	
1x25G[10G]	25000	CR, KR	
1x25G[10G]	10000	KR	
1x100G[40G]	100000, 40000	CR4, KR4	
4x25G[10G]	25000	CR,KR	
4x25G[10G]	10000	KR	

AS4630-54NPE Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x0G	2500,1000,100	None	Ethernet0~Ethernet35
1x0G	10000,5000,2500,1000,100	None	Ethernet36~Ethernet47
1x25G[10G]	25000	CR, KR	
1x25G[10G]	10000	KR	
1x100G[40G]	100000, 40000	CR4, KR4	
4x25G[10G]	25000	CR,KR	
4x25G[10G]	10000	KR	

Guide

- For one specified breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one specified breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid speeds and invalid speeds shall be told.

(202111.2) AutoNego User Guide for AS5812-54X

- [Capability](#)
- [Guide](#)
- [Note](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x10G[1G]	10000, 1000	None	
1x40G	40000	CR4, KR4	
4x10G	10000	KR	

Guide

- For one breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid interface types and invalid interface types shall be told.
- No valid interface types for this breakout mode if "None" is specified in the **support adv-types field**.
- For the Ethernet0~Ethernet47, only the **1x10G[1G]** breakout mode be supported.
- The CL37 1000BASE-X shall not take effect if **10000** is specified in the adv-speeds configuration.

Adv-speeds	Technology applied in Chip	Note
10000	CL73 10GBASE-KR	
10000,1000	CL73 10GBASE-KR	
1000	CL37 1000BASE-X	

Note

1. The KX interface type is not supported from ocp-SAI or community.
2. Not support 1000X at 4x10G interfaces due to no such **fiber** cables.
3. Not support 1000KX at 1x10G[1G] breakout mode.

(202111.2) AutoNego User Guide for AS5835-54X/AS5835-54T

- [AS5835-54X Capability](#)
- [AS5835-54T Capability](#)
- [Guide](#)
- [Note](#)

AS5835-54X Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x10G[1G]	10000	KR	
1x100G[40G]	100000, 40000	CR4, KR4	
4x25G	25000, 10000	CR, KR	
4x10G	10000	KR	

AS5835-54T Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x0G	10000, 5000, 2500,1000	None	
1x100G[40G]	100000, 40000	CR4, KR4	
4x25G	25000	CR,KR	
4x10G	10000	KR	

Guide

- For one breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid interface types and invalid interface types shall be told.
- No valid interface types for this breakout mode if "None" is specified in the **support adv-types field**.

Note

1. The KX interface type is not supported from ocp-SAI or community.
2. Not support 1000KX at 1x10G[1G] breakout mode.

(202111.2) AutoNego User Guide for AS7326-56X

- [Capability](#)
- [Guide](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
4x10G[1G]	10000,1000	KR	
4x10G	10000	KR	
4x25G	25000	CR,KR	
1x100G[40G]	40000,100000	CR4,KR4	

Guide

- For one specified breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one specified breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid speeds and invalid speeds shall be told.

(202111.2) AutoNego User Guide for AS7712-32X

- [Capability](#)
- [Guide](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
4x10G	10000	KR	
4x25G	N/A	N/A	not supported
1x100G[40G]	40000,100000	CR4,KR4	

Guide

- For one specified breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one specified breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid speeds and invalid speeds shall be told.

(202111.2) AutoNego User Guide for AS7726-32X

Capability

Breakout mode	support adv-speeds	support adv-types	Note
4x10G	10000	KR	
4x25G	25000	CR,KR	
1x100G[40G]	40000,100000	CR4,KR4	

Guide

- For one specified breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one specified breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid speeds and invalid speeds shall be told.

(202111.2) AutoNego User Guide for AS9716-32D

- [Capability](#)
- [Guide](#)
- [Before 2111.2](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x400G	400000	CR8, KR8, LR8, SR8	
2x200G	200000	CR4, KR4, LR4, SR4	
	100000	CR4, KR4, LR4, SR4	
4x100G	100000	CR2, KR2, SR2	
1x100G[40G](4)	100000	CR4, KR4, LR4, SR4	
	40000	CR4, KR4, LR4, SR4	
8x50G[25G]	50000	CR, KR	
	25000	CR, KR	
4x25G[10G](4)	25000	CR, KR	

Guide

- For one specified breakout mode, only the speeds defined in **support adv-speeds field** can be configured. If the configuration contains valid and invalid speeds, the configuration shall fail and the valid speeds and invalid speeds shall be told.
- For one specified breakout mode, only the interface types defined in **support adv-types field** can be configured. If the configuration contains valid and invalid interface types, the configuration shall fail and the valid speeds and invalid speeds shall be told.

Before 2111.2

! About AS9716-32D

The AutoNeg is supported at the following break-out modes. The test DUT and link-partner are both AS9716-32D. Note that AutoNeg is only supported on DAC cables. The below table shows the ability if the AutoNeg is enabled in the specified breakout mode with the current speed. The `config interface advertised-speeds` command does not support at AS9716-32D.

Breakout Mode	Current Speed(*1)	Speed Ability(*2)	FEC Ability(*3)	LT (AN = on)(*4)	LT (AN = off)(*4)
1x400G	400G	400G(KR8/CR8)	RS_544_2xN	Enabled	Enabled
2x200G	200G	200G(KR4/CR4) 100G(KR4/CR4)	200G RS_544_2xN 100G RS_FEC	Enabled	200G Enabled 100G Disabled
4x100G	100G	100G(KR2/CR2) 50G(R2)	100G RS_544_1xN 50G RS_FEC	Enabled	100G Enabled 50G Disabled
1x100G[40G](4)	100G	100G(KR4/CR4) 40G(KR4/CR4)	100G RS_FEC 40G BASE_R	Enabled	Disabled
1x100G[40G](4)	40G	40G(KR4/CR4)	40G BASE_R	Enabled	Disabled
2x50G(4)	50G	50G(R2)	50G RS_FEC	Enabled	Disabled
4x25G[10G](4)	25G	25G(KR/KR-S/CR/CR-S/R)	25G RS_FEC 25G BASE_R	Enabled	Disabled
4x25G[10G](4)	10G	Not supported	Not supported	Not supported	Disabled

(*1) The 'current speed' indicates the port speed configured by CLI or the default port speed on the given port breakout mode.

(*2) The 'speed ability' indicates the advertised speed to be set on PHY and will be seen by the link partner when autonego is enabled.

(*3) The 'FEC ability' indicates the advertised FEC mode to be set on PHY and will be seen by the link partner when autonego is enabled.

(*4) On ec202012, there is no CLI to configure 'Link Training'(i.e. 'LT'). The 'LT' status is controlled by the SAI implementation. The table illustrates the LT status for each configuration set.

(*5) A new bcm configuration 'skip_auto_lt' is introduced. The default value is 0(i.e. disabled). If skip_auto_it is set as 1, the LT configuration shall not be changed if AN is disabled.

(*6) LT is only applicable on DAC cables. So the column "LT (AN = off)" that contains "enabled" only takes effect when the inserted transceiver is DAC cable.

(202111.2) AutoNego User Guide for AS9726-32D

- [Capability](#)
- [Limitation:](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x400G	400000	CR8, KR8, LR8, SR8	
2x200G	200000	CR4, KR4, LR4, SR4	
4x100G	100000	CR2, KR2, SR2	
1x100G[40G](4)	100000	CR4, KR4, LR4, SR4	
	40000	CR4, KR4, LR4, SR4	
4x25G[10G](4)	25000	CR, KR, LR, SR	

Limitation:

1. The AUTONEG is not supported at 10G interface(Ethernet256, Ethernet257).

(202111.2) AutoNego User Guide for AS9736-64D

- [Capability](#)
- [Limitation:](#)

Capability

Breakout mode	support adv-speeds	support adv-types	Note
1x400G	400000	CR8, KR8, LR8, SR8	
2x200G	200000	CR4, KR4, LR4, SR4	ONSBUSWII-1194
4x100G	100000	CR2, KR2, SR2	
1x100G[40G](4)	100000	CR4, KR4, LR4, SR4	
	40000	CR4, KR4, LR4, SR4	
4x25G[10G](4)	25000	CR, KR, LR, SR	

Limitation:

1. The AUTONEG is not supported at 10G interface(Ethernet512, Ethernet513).

(202111.2) IP Configuration

(202111.2) IPv4/IPv6 Interfaces and Routes

You can specify IPv4/IPv6 subnets on a switch by manually or dynamically assigning IPv4/IPv6 addresses to port or VLAN interfaces. To specify IP subnets not directly connected to a switch, you can either configure static routes, or use dynamic routing protocols to identify routes that lead to other subnets by exchanging protocol messages with other routers on the network.

To display IPv4 or IPv6 routes that are currently in the routing table, use the **show ip route** or **show ipv6 route** commands.

Example

```
admin@sonic:~$ show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, D - SHARP,
       F - PBR, f - OpenFabric,
       > - selected route, * - FIB route, q - queued route, r - rejected route

S>* 0.0.0.0/0 [200/0] via 10.11.162.254, eth0
C>* 1.1.0.0/16 is directly connected, Vlan100
C>* 10.1.1.0/31 is directly connected, Ethernet112
C>* 10.1.1.2/31 is directly connected, Ethernet116
C>* 10.11.162.0/24 is directly connected, eth0

admin@sonic:~$ show ipv6 route
Codes: K - kernel route, C - connected, S - static, R - RIPng,
       O - OSPFv3, I - IS-IS, B - BGP, N - NHRP, T - Table,
       v - VNC, V - VNC-Direct, A - Babel, D - SHARP, F - PBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued route, r - rejected route

C>* 2018:2001::/126 is directly connected, Ethernet112
C>* 2018:2002::/126 is directly connected, Ethernet116
C>* fc00:2::102/128 is directly connected, eth0
C * fe80::/64 is directly connected, Vlan100
C * fe80::/64 is directly connected, Ethernet112
C * fe80::/64 is directly connected, Ethernet116
C * fe80::/64 is directly connected, Bridge
C * fe80::/64 is directly connected, PortChannel0011
C>* fe80::/64 is directly connected, eth0
```

To display details about all the Layer3 interfaces on a switch for which IPv4/IPv6 addresses have been assigned, use the **show ip interfaces** or **show ipv6 interfaces** command.

The type of interfaces that can be assigned IPv4/IPv6 addresses include the following.

- Front panel physical ports
- PortChannel interfaces
- VLAN interfaces
- Loopback interfaces
- docker interface (IP only)
- management interface

Example

```
admin@sonic:~$ show ip interfaces
```

Interface	Master	IPv4 address/mask	Admin/Oper	BGP Neighbor	Neighbor IP	Flags
-----	-----	-----	-----	-----	-----	

Loopback0		1.0.0.1/32	up/up	N/A	N/A	
Loopback11	Vrf-red	11.1.1.1/32	up/up	N/A	N/A	
Loopback100	Vrf-blue	100.0.0.1/32	up/up	N/A	N/A	
PortChannel01		10.0.0.56/31	up/down	DEVICE1	10.0.0.57	
PortChannel02		10.0.0.58/31	up/down	DEVICE2	10.0.0.59	
PortChannel03		10.0.0.60/31	up/down	DEVICE3	10.0.0.61	
PortChannel04		10.0.0.62/31	up/down	DEVICE4	10.0.0.63	
Vlan100	Vrf-red	1001.1.1/24	up/up	N/A	N/A	
Vlan1000		192.168.0.1/27	up/up	N/A	N/A	
docker0		240.127.1.1/24	up/down	N/A	N/A	
eth0		10.3.147.252/23	up/up	N/A	N/A	
lo		127.0.0.1/8	up/up	N/A	N/A	

```
admin@sonic:~$ show ipv6 interfaces
```

Interface	Master	IPv6 address/mask	Admin/Oper	BGP Neighbor	Neighbor IP
-----	-----	-----	-----	-----	-----

Bridge		fe80::7c45:1dff:fe08:cdd%Bridge/64	up/up	N/A	N/A
Loopback11	Vrf-red	1100::1/128	up/up		
PortChannel01		fc00::71/126	up/down	DEVICE1	fc00::72
PortChannel02		fc00::75/126	up/down	DEVICE2	fc00::76
PortChannel03		fc00::79/126	up/down	DEVICE3	fc00::7a
PortChannel04		fc00::7d/126	up/down	DEVICE4	fc00::7e
Vlan100	Vrf-red	100::1/112	up/up	N/A	N/A
		fe80::eef4:bbff:fe08:880a%Vlan100/64			
eth0		fe80::eef4:bbff:fe08:880a%eth0/64	up/up	N/A	N/A
lo		fc00::1::32/128	up/up	N/A	N/A

For more information on IPv4/IPv6 interfaces and routes, see the [\(202111\) IP/IPv6](#) section in the CLI reference guide.

(202111.2) Neighbor Discovery Protocol (NDP)

IPv6 Neighbor Discovery Protocol supersedes IPv4 Address Resolution Protocol in IPv6 networks. IPv6 nodes on the same network segment use Neighbor Discovery to discover each other's presence, to determine each other's link-local addresses, to find routers and to maintain reachability information about the paths to active neighbors.

To display all the discovered IPv6 neighbors, use the **show ndp** command.

Example

```
admin@sonic:~$ show ndp
Address           MacAddress       Iface  Vlan  Status
-----
fe80::20c:29ff:feb8:b11e 00:0c:29:b8:b1:1e eth0   -    REACHABLE
fe80::20c:29ff:feb8:cff0 00:0c:29:b8:cf:f0 eth0   -    REACHABLE
fe80::20c:29ff:feb9:324  00:0c:29:f9:03:24 eth0   -    REACHABLE
Total number of entries 3
```

For more information on NDP, see the [\(202111\) NDP](#) section in the CLI reference guide.

(202111.2) Static Anycast Gateway (SAG)

The SONiC Static Anycast Gateway (SAG) feature is a method of moving a subnet default gateway close to endpoints in VXLAN networks. In a typical folded-Clos data center network, using a static anycast gateway means that all the leaf switches (VXLAN VTEPs) are configured with the same IPv4/IPv6 address and function as the gateway for all their attached host servers. An active SAG eliminates the need for some protocol traffic across the network fabric and enables host workload flexibility and efficiency.

When implementing SAG, it is important to note that host server VLANs are mapped to each VXLAN by the VNI number. A leaf switch only advertises its gateway IP address and MAC address to the attached host server VLANs, never to the network fabric. By this method, all the anycast gateway IPv4/IPv6 addresses and MAC addresses can remain consistent across all leaf switch VLANs.

To configure SAG on a switch, use the **config sag mac_address add** command to configure the SAG MAC address (only one allowed). Then configure the SAG IPv4/IPv6 address on a specific VLAN interface, use the **config interface sag ip add** command. Use the **config sag ipv4 enable** or **config sag ipv6 enable** command to enable the IPv4/IPv6 address on an interface. To display SAG interface settings, use the **show sag**, **show sag ip**, and **show sag ipv6** commands.

Example

```
admin@sonic:~# sudo config sag mac_address add 00:11:22:33:44:55

admin@sonic:~# sudo config interface sag ip add Vlan100 192.168.1.1/24

admin@sonic:~# sudo config sag ipv4 enable

admin@sonic:~#show sag ip
Vlan Interface Name      IPv4 address/mask
-----
Vlan100                  192.168.1.1/24

admin@sonic:~#show sag
Static Anycast Gateway Information

MacAddress              IPv4      IPv6
-----
00:11:22:33:44:55      enable    enable
```

For more information, see the [\(202111\) SAG](#) section in the CLI reference guide.

(202111.2) Layer 2

(202111.2) Configuring DHCP Relay

In some network environments, the DHCP server is located in another segment that is outside of the broadcast segment. In this case, the network administrator needs to configure DHCP relay to forward DHCP requests to the DHCP server in the remote segment, and to re-direct DHCP offers to DHCP clients.

To configure DHCP relay, the network administrator needs to select the interface (VLAN) from which DHCP requests are received, and the server that services those DHCP requests on the interface (VLAN). The following is an example of configuring DHCP relay on interface (VLAN 1000) and specifying the DHCP server address 7.7.7.7. For detailed information, refer to the [\(202111\) DHCP Relay](#) section in the CLI reference guide.

Example

```
admin@sonic:~$ sudo config vlan dhcp_relay add 1000 7.7.7.7
Added DHCP relay destination address 7.7.7.7 to Vlan1000
Restarting DHCP relay service..

admin@sonic:~$ sudo config vlan dhcp_relay del 1000 7.7.7.7
Removed DHCP relay destination address 7.7.7.7 from Vlan1000
Restarting DHCP relay service...
Running command: systemctl restart dhcp_relay
```

DHCP relay over different VRFs

The DHCP server will determine the destination IP of the DHCP offer based on the Relay agent IP address field(IP address of the client-interface) of DHCP discover packet. But socket cannot receive DHCP offer in server-VRF that the destination IP of the DHCP offer is not included in server-VRF. Therefore, need to change the Relay agent IP address to server-VRF through the source interface command and enable the link-selection sub-option.

The link-selection sub-option of the Agent information option for the DHCP is used by any DHCP relay agent that desires to specify a subnet/link for a DHCP client request that it is relaying but needs the subnet/link specification to be different from the IP address the DHCP server should use when communicating with the relay agent. Therefore, the link-selection sub-option specifies an IP address that determines a subnet on which the DHCP client is located, and the relay agent IP address field can be used to communicate with the relay agent.

The source interface is used to specify the Relay agent IP address field and the source IP address of DHCP discover relayed packet.

Before enable the link selection, need to use the source interface command to specify the Relay agent IP address and the source IP address of DHCP discover that belonging to server-VRF.

Example

```
admin@sonic:~$ sudo config vlan dhcp_relay src_intf add 1000 PortChannel0001
Added DHCP relay source interface PortChannel0001 for Vlan1000
Restarting DHCP relay service...
```

```
admin@sonic:~$ sudo config vlan dhcp_relay link_selection add 1000
Enable DHCP relay link selection for Vlan1000
Restarting DHCP relay service...
```

```
admin@sonic:~$ show ip interfaces
```

Interface	Master	IPv4 address/mask	Admin/Oper	BGP Neighbor	Neighbor IP
Loopback0		10.1.0.32/32	up/up	N/A	N/A
PortChannel0001	Vrf1	10.0.0.56/31	up/up	ARISTA01T1	10.0.0.57
Vlan1000		192.168.0.1/21	up/up	N/A	N/A
docker0		240.127.1.1/24	up/down	N/A	N/A
eth0		10.250.0.196/16	up/up	N/A	N/A
lo		127.0.0.1/16	up/up	N/A	N/A

```
admin@sonic:~$ show vlan brief
```

VLAN ID	IP Address	Ports	Port Tagging	DHCP Helper	DHCP Source	DHCP Link
Proxy ARP				Address	Interface	Selection
1000	192.168.0.1/21	Ethernet4	untagged	10.0.0.57	PortChannel0001	enabled
disabled						

(202111.2) Configuring interface TPID

TPID is used to identify whether the frame received is a tagged or untagged frame. Default TPID is 0x8100.

The primary usage of SONiC TPID configuration is to configure HW ASIC to treat non-matching TPID VLAN tags as customer payload and preserve the VLAN TAG, the control plane traffic will not be handled.

The TPID value only allows 0x8100, 0x9100, 0x9200, and 0x88A8.

To change the TPID value for a physical interface or port-channel interface, use the **config interface tpid** command.

Example

```
admin@sonic:~$ sudo config interface tpid Ethernet0 0x88a8
```

Once TPID is configured, use **show interfaces tpid** command to check the configuration.

Example

```
admin@sonic:~$ show interfaces tpid
```

Interface	Alias	Oper	Admin	TPID
Ethernet0	Eth1(Port1)	up	up	0x88A8

For more information, see the [\(202111\) Interfaces \(Updated\)](#) section in the CLI reference guide.

NOTE: Currently Linux kernel only supports default 802.1Q TPID 0x8100, SONiC TPID configuration does NOT change Linux kernel's TPID value.

(202111.2) Control Plane Policing (CoPP)

Control Plane Policing (CoPP) allows users to manage and rate limit control plane packets. The CoPP configuration file is composed of one or more trap groups. Each trap group includes one or more predefined trap IDs that are mapped to known protocols (STP, LACP, EAPOL, etc.), and a meter (if required) decides how to process the specific packets (forward, trap to CPU, etc.).

The following are the supported trap IDs:

- stp
- lacp
- eapol
- lldp
- pvrst
- igmp_query
- sample_packet
- udld
- switch_cust_range
- arp_req
- arp_resp
- dhcp
- ospf
- pim
- dhcpv6
- neigh_discovery
- src_nat_miss
- dest_nat_miss
- isis
- arp_suppression
- nd_suppression
- ip2me
- ssh
- snmp
- bgp
- bgpv6
- bfd
- bfdv6
- l3_mtu_error
- ttl_error

The following shows an example of a CoPP configuration file:

Example

```
{
  "COPP_TABLE:trap.group.nat": {
    "trap_ids": "src_nat_miss,dest_nat_miss",
    "trap_action": "trap",
    "trap_priority": "1",
    "queue": "1",
    "meter_type": "packets",
    "mode": "sr_tcm",
    "cir": "600",
    "cbs": "600",
    "red_action": "drop"
  },
  "OP": "SET"
}
```

You can update the CoPP configuration by updating the configuration file, which is `/usr/share/sonic/templates/copp.json.j2` located in the swss container, and then use **config reload** to apply the new configuration.

Example

```
admin@sonic:~$ docker exec -it swss bash
root@sonic:/# vi /usr/share/sonic/templates/copp.json.j2
root@sonic:/# exit
exit
admin@sonic:~$ config reload -y
```

For more information, see the [\(202111\) CoPP](#) section in the CLI reference guide.

(202111.2) Creating VLANs

In large networks, you can isolate traffic into separate broadcast domains by grouping network nodes into Layer 2 VLANs. VLANs provide greater network efficiency by reducing broadcast traffic, and allow you to make network changes without having to update IP addresses or IP subnets. VLANs inherently provide a high level of network security since traffic must pass through a configured Layer 3 link to reach a different VLAN.

To create a VLAN, first configure a VLAN ID using the **config vlan add** command, and then add port members using the **config vlan member add** command.

Example

```
admin@sonic:~$ sudo config vlan add 100

admin@sonic:~$ sudo config vlan member add 100 Ethernet4
```

To display information about configured VLANs, use the **show vlan brief** command or the **show vlan config** command.

Example

```
admin@sonic:~$ show vlan brief
+-----+-----+-----+-----+-----+-----+-----+
+-----+
| VLAN ID | IP Address   | Ports      | Port Tagging | Proxy ARP | DHCP Helper | DHCP Source |
| DHCP Link |              |            |              |           | Address     | Interface   |
| Selection |              |            |              |           |             |             |
+=====+=====+=====+=====+=====+=====+=====+
=====+
|      100 | 1.1.2.2/16   | Ethernet0  | tagged       | disabled  | 192.0.0.1   |             |
|          |              | Ethernet4  | tagged       |           | 192.0.0.2   |             |
|          |              |            |              |           | 192.0.0.3   |             |
|          |              |            |              |           |             |             |
+-----+-----+-----+-----+-----+-----+-----+
+-----+

admin@sonic:~$ show vlan config
Name      VID  Member      Mode
-----
Vlan100   100  Ethernet0    tagged
Vlan100   100  Ethernet4    tagged
```

For more information, see the [\(202111\) VLAN](#) section in the CLI reference guide.

(202111.2) Forwarding Database (FDB)

Switches store the addresses for all known devices. This information is used to forward traffic directly between the inbound and outbound ports. All the addresses learned by monitoring traffic are stored in the dynamic address table, or forwarding database (FDB).

To display the MAC (FDB) entries either in full or partial (by VLAN or port), use the **show mac** command. To clear the FDB table of all entries, use the **sonic-clear fdb all** command.

Example

```
admin@sonic:~$ show mac
No.      Vlan      MacAddress      Port
-----
1        1000      E2:8C:56:85:4A:CD Ethernet192
2        1000      A0:1B:5E:47:C9:76 Ethernet192
3        1000      AA:54:EF:2C:EE:30 Ethernet192
4        1000      A4:3F:F2:17:A3:FC Ethernet192
5        1000      0C:FC:01:72:29:91 Ethernet192
6        1000      48:6D:01:7E:C9:FD Ethernet192
7        1000      1C:6B:7E:34:5F:A6 Ethernet192
8        1000      EE:81:D9:7B:93:A9 Ethernet192
9        1000      CC:F8:8D:BB:85:E2 Ethernet192
10       1000      0A:52:B3:9C:FB:6C Ethernet192
11       1000      C6:E2:72:02:D1:23 Ethernet192
12       1000      8A:C9:5C:25:E9:28 Ethernet192
13       1000      5E:CD:34:E4:94:18 Ethernet192
14       1000      7E:49:1F:B5:91:B5 Ethernet192
15       1000      AE:DD:67:F3:09:5A Ethernet192
16       1000      DC:2F:D1:08:4B:DE Ethernet192
17       1000      50:96:23:AD:F1:65 Ethernet192
18       1000      C6:C9:5E:AE:24:42 Ethernet192
Total number of entries 18

admin@sonic:~$ sonic-clear fdb all
FDB entries are cleared.
```

For more information, see the [\(202111\) FDB](#) section in the CLI reference guide.

(202111.2) Link Aggregation (LAG)

LAG (Link Aggregation) combine the bandwidth of multiple Ethernet ports into a single logical link. A link Aggregation Group (LAG) is also referred to as a port channel.

To create a port channel, first configure a port channel name using the **config portchannel add** command, and then add members using the **config portchannel member add** command.

The port channel name should have prefix 'PortChannel' and suffix '<0-9999>'. For example, "PortChannel200".

Example

```
admin@sonic:~$ sudo config portchannel add PortChannel200

admin@sonic:~$ sudo config portchannel member add PortChannel200 Ethernet4
```

To display information about configured port channel, use the **show interfaces portchannel** command.

Example

```
admin@sonic:~$ show interfaces portchannel
Flags: A - active, I - inactive, Up - up, Dw - Down, N/A - not available,
       S - selected, D - deselected, * - not synced
  No.  Team Dev      Protocol    Ports          Oper Key  Admin Key
-----
  200  PortChannel200  LACP(A)(Dw) Ethernet4(D)    1200  auto
```

For more information, see the [\(202111\) Port Channels](#) section in the CLI reference guide.

(202111.2) Link Layer Discovery Protocol (LLDP)

The Link Layer Discovery Protocol (LLDP) is used to discover basic information about neighboring devices in the local broadcast domain. LLDP is a Layer 2 protocol that uses periodic broadcasts to advertise information about the sending device. Advertised information is represented in Type Length Value (TLV) format according to the IEEE 802.1AB standard, and can include details such as device identification, capabilities, and configuration settings. LLDP also defines how to store and maintain information gathered about the neighboring network nodes it discovers. LLDP information can be used by applications to simplify troubleshooting, enhance network management, and maintain an accurate network topology.

You can use the **show lldp table** command to display a summary of discovered LLDP neighbors, or the **show lldp neighbors** command to display detailed information about LLDP neighbors.

Example

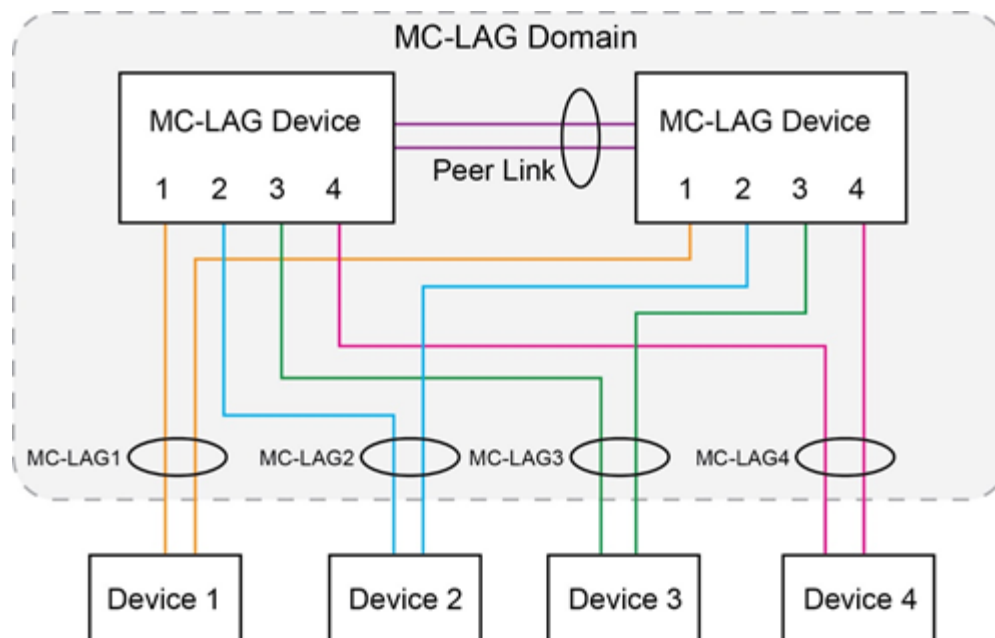
```
admin@sonic:~$ show lldp table
Capability codes: (R) Router, (B) Bridge, (O) Other
LocalPort    RemoteDevice  RemotePortID  Capability  RemotePortDescr
-----
Ethernet112  ARISTA01T1   Ethernet1      BR
Ethernet116  ARISTA02T1   Ethernet1      BR
eth0         00:e0:0c:00:01:05  BR              Ethernet Port on unit 1, port 8
-----
Total entries displayed: 3
```

For more information, see the [\(202111\) LLDP](#) section in the CLI reference guide.

(202111.2) Multi-Chassis Link Aggregation Group (MC-LAG)

What is MC-LAG (Multi-Chassis)

A Multi-Chassis Link Aggregation Group (MC-LAG) is a pair of links that terminate on two cooperating switches and appear as an ordinary link aggregation group (LAG). The cooperating switches are MC-LAG peer switches and communicate through an interface called a peer link. While the peer link's primary purpose is exchanging MC-LAG control information between peer switches, it also carries data traffic from devices that are attached to only one MC-LAG peer and have no alternative path. An MC-LAG domain consists of the peer switches and the control links that connect these switches.



Peer Link: a link between the cooperating switches (or MC-LAG device)

Control Link: a link between peer switches, exchanging control information. It could be in peer link if there is no extra link between MC-LAG devices.

Host device: device which connects MC-LAG device with port-channel. eg Device 1, Device 2.. Device 4.

Configure MC-LAG domain

Currently there can be only one MC-LAG domain configured on a system and only configured port-channels can be MC-LAG interfaces. To set up an MC-LAG domain, you need to configure two switch systems and their Peer Link connection. Follow these basic steps for each MC-LAG switch:

Step 1: Create port-channels for the host device (data forwarding) connections and the MC-LAG Peer Link connection between the switches.

Step 2: Create two VLANs, one for MC-LAG peer keepalive link and the other for the data forwarding port-channels.

- these two VLANs should be different VLANs.

Step 3: Add an IP address for the MC-LAG peer keepalive link.

Step 4: Create the MC-LAG domain by specifying the MC-LAG peer keepalive link IP addresses (local and peer device) and the peer port-channel interface name.

- If keepalive link is set on VLAN interface and peer link port is added to this VLAN, need to enable unique IP for this VLAN.

Step 5: Add the data forwarding port-channels as the MC-LAG members.

Use the **config mlag add** command to configure the MC-LAG domain.

Add the host device port-channel members using the **config mlag member add** command.

To display the MC-LAG configuration and status, use the **show mlag brief** command.

```

admin@sonic:~$ show vlan brief
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| VLAN ID | IP Address | Ports | Port Tagging | DHCP Helper | DHCP Source | DHCP |
| Link | Proxy ARP | | | | | |
| Selection | | | | | | |
+-----+-----+-----+-----+-----+-----+
=====
=====+
| 100 | | PortChannel1 | tagged | | |
| | disabled | PortChannel3 | tagged | | |
| | | | | | |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| 200 | 192.168.1.1/24 | PortChannel3 | tagged | | |
| | disabled | | | | |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+

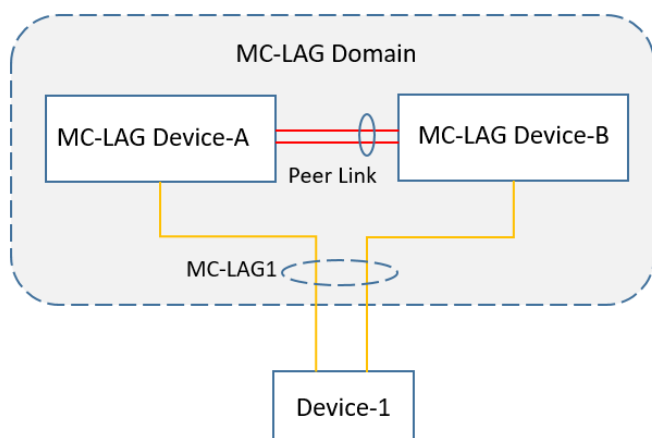
```

- Peer-link PortChannel3 and MC-LAG enabled interface PortChannel1 are both the member of Vlan100. MC-LAG peer keepalive link must belong to another Vlan200.
- Configure unique-ip on VLAN200 due to peer link PortChannel3 is added to this VLAN. To allow separate IP configuration on the VLAN200 interface for keepalive link.
- Both active and standby nodes require same Unique IP configuration on the VLAN interface.

For more information, see the [\(202111\) MC LAG](#) section in the CLI reference guide.

Flush MAC behavior vs host moving over the port-channel

In the current behavior, MAC addresses learned from peer MLAG device will be treated as static entries on local MLAG device , but allow them to move.
When a dynamic MAC address on peer MLAG interface gets age out, same static MAC will be changed to dynamic type if traffic from the MAC is received on local MLAG device.
Otherwise, the static MAC also be deleted.
For details, please refer to following example.



Flush MAC will change the dynamic to static allow port move (MAC learned from MCLAG)

Step 1 : The data flow path from Device-1 to Device-A, the MAC of Device-1 is learned in Device-A (Dynamic) and sync to Device-B (Static)

Step 2 : Device-A flush dynamic MAC, Device-A send delete msg to Device-B

Step 3 : Device-B delete static MAC (learned from MCLAG), but there is traffic flowing through Device-B at this time, it will change to dynamic MAC and notify to Device-A

Step 4 : Device-A add static MAC (learned from MCLAG)

Device-A

```

admin@leaf-1:~$ show mac
  No.   Vlan  MacAddress      Port           Type
  ----  -
  1     201   00:10:94:00:00:02  PortChannel10  Dynamic
Total number of entries 1
admin@leaf-1:~$
admin@leaf-1:~$ sudo sonic-clear fdb all
FDB entries are cleared.

admin@leaf-1:~$
admin@leaf-1:~$ show mac
  No.   Vlan  MacAddress      Port           Type
  ----  -
  1     201   00:10:94:00:00:02  PortChannel10  Static
Total number of entries 1
  
```

Flush can clear the dynamic mac

Step 1 : The data flow path from Device-1 to Device-A, the MAC of Device-1 is learned in Device-A (Dynamic) and sync to Device-B (Static)

Step 2 : Device-A flush dynamic MAC, Device-A send delete msg to Device-B

Step 3 : Device-B delete static MAC (learned from MCLAG), and there is no traffic flowing through Device-B, so Device-B is allowed to delete the static MAC.

Device-A

```
admin@leaf-1:~$ show mac
  No.    Vlan  MacAddress          Port           Type
  ----  -
    1     201  00:10:94:00:00:02  PortChannel10  Dynamic
Total number of entries 1
admin@leaf-1:~$
admin@leaf-1:~$ sudo sonic-clear fdb all
FDB entries are cleared.

admin@leaf-1:~$ show mac
  No.    Vlan  MacAddress          Port           Type
  ----  -
Total number of entries 0
admin@leaf-1:~$
```

(202111.2) PVST and STP

The Spanning Tree Protocol (STP) prevents Layer 2 loops in a network and provides redundant links.

The pvst mode allows for running multiple instances of spanning tree on per VLAN basis.

The stp mode allows for running a single spanning-tree instance on all VLANs.

To enable the spanning tree and specify mode for the device, using the **config spanning-tree** command.

Example

```
admin@sonic:~$ config spanning-tree enable pvst
```

After creating a VLAN and port is added to the VLAN, the spanning tree on the VLAN starts running.

Example

```
admin@sonic:~$ config vlan add 100
admin@sonic:~$ config vlan member add 100 Ethernet4
```

You can use the **show spanning-tree** command to display running status.

Example

```
admin@sonic:~$ show spanning-tree
Spanning-tree Mode: PVST

VLAN 100 - STP instance 0
-----
STP Bridge Parameters:
Bridge          Bridge Bridge Bridge Hold  LastTopology Topology
Identifier      MaxAge Hello  FwdDly Time  Change        Change
hex            sec    sec    sec    sec    sec          cnt
806480a235321cbb 20    2      15    1      11           1

RootBridge      RootPath  DesignatedBridge RootPort          Max Hel Fwd
Identifier      Cost      Identifier          Age lo  Dly
hex             hex             sec sec sec
806480a235321cbb 0          806480a235321cbb Root      20  2  15

STP Port Parameters:
Port           Prio Path      Port Uplink State      Designated  Designated  Designated
Name          rity Cost      Fast Fast      Cost        Root        Bridge
Ethernet4     128 200        N    N      FORWARDING  0          806480a235321cbb 806480a235321cbb
```

For more information, see the [\(202111\) PVST \(STP\)](#) section in the CLI reference guide.

(202111.2) Virtual Extensible LAN (VxLAN)

VxLAN is a networking scheme that encapsulates MAC-based Layer 2 Ethernet frames in Layer 3 UDP packets to aggregate and tunnel multiple Layer 2 networks across a Layer 3 infrastructure. VxLAN scales up to 16 million logical networks and supports Layer 2 adjacency for isolated tenants across IP networks. Multicast transmission is used for BUM (broadcast, unknown-unicast, and multicast) traffic. In this implementation, broadcast, multicast and unknown unicast traffic can also use unicast tunneling.

To forward (or flood) a received frame on a port across VxLAN, the switch performs following examination to determine the egress interface:

- Checks the VLAN-to-VxLAN mapping table, where the ingress port should be a member of the VLAN that the received frame belongs to.
 - If an entry is found, searches for the destination port in the bridge table.
 - If the destination port is found, encapsulates the frame with a VxLAN header and sends it to the corresponding VTEP (Virtual End Point).
 - If the destination port is not found, encapsulates the frame with a VxLAN header and floods it to all VTEPs on the VxLAN ID.

The flooded packet is encapsulated as a unicast packet or multicast packet according to the configured setting that is described in RFC 7348.

For example, a host sends a frame to a VM (Virtual Machine) on a remote host over VxLAN; it sends a frame as usual. The VTEP of the ingress port searches for the VNI (Virtual Network Identifier) to which the VM is associated and finds the destination address is mapped to a remote VTEP. The switch adds an outer header (MAC address, IP address, and VxLAN header), encapsulates the frame, and then forwards it to the remote VTEP. The remote VTEP validates the decoded frame with the VNI to which the destination address of the original frame is matched, and then forwards the frame to the host on which the VM runs. The remote VTEP, in addition to forwarding the frame to the VM, also learns the source (MAC) address with outer source IP address and stores the mapping in a bridge lookup table. This entry can be used when the VM sends a response frame to the host.

Determining the MAC address of a destination VM prior to transmission by the source VM is performed as in non-VxLAN environments. Broadcast frames are used, but are encapsulated within a multicast packet.

For data center network virtualization, VxLAN is often used to extend Layer 2 segments across a Layer 3 infrastructure. However on its own, VxLAN does not have a control plane. To meet the requirements of multi-tenant data centers and to be able to scale VxLANs effectively, Ethernet Virtual Private Network (EVPN) is often used as a control plane Network Virtualization Overlay (NVO) solution.

To configure VxLAN, first use the **config vxlan add** command and specify a source VTEP name and IPv4 address. Then use the **config vxlan evpn_nvo add** command to define an NVO name for the VTEP. You can then use the **config vxlan map add** command to map a VLAN ID to a VNI for the VTEP.

Example

```
sudo config vxlan add vtep 1.1.1.1

sudo config vxlan evpn_nvo add nvo vtep

sudo config vxlan map add vtep 30 3000
```

To display the VxLAN configuration, use the **show vxlan tunnel** command or **show vxlan vlanvni** command.

Example

```
admin@7726:~$ show vxlan tunnel
+-----+-----+-----+-----+
| SIP    | DIP    | Creation Source | OperStatus |
+-----+-----+-----+-----+
| 1.1.1.1 | 2.2.2.2 | EVPN            | oper_down  |
+-----+-----+-----+-----+
Total count : 1

admin@7726:~$ show vxlan vlanvni
+-----+-----+
| VLAN  | VNI   |
+-----+-----+
| Vlan30 | 3000  |
+-----+-----+
Total count : 1
```

For more information, see the [VxLAN](#) section in the Command Line Interface guide.

(202111.2) Layer 3

(202111.2) SONiC Routing Stack

SONiC uses the Free Range Routing (FRRouting or FRR) IP routing suite for the configuration of routing protocols. FRR supports a number of common routing protocols, such as BGP, OSPF, RIP etc, all of which can be configured through the FRR VTY shell "vtysh."

Enable Routing Protocols

BGP protocol is always enabled. Other routing protocols, including OSPF, ISIS, BFD etc, can be enabled by setting "frr_mgmt_framework_config" field to "true" in config_db's DEVICE_META table.

Details of FRR unified configuration management framework can refer to https://gitlab.edge-core.com/root/sonic/-/blob/ec202012/doc/mgmt/SONiC_Design_Doc_Unified_FRR_Mgmt_Interface.md

CONFIG_DB

```
"DEVICE_METADATA": {
  "localhost": {
    "bgp_asn": "65100",
    "buffer_model": "traditional",
    "cloudtype": "None",
    "default_bgp_status": "up",
    "default_pfcwd_status": "disable",
    "deployment_id": "1",
    "docker_routing_config_mode": "split",
    "frr_mgmt_framework_config": "true",
    "hostname": "as7816-64x",
    "hwsku": "Accton-AS7816-64X",
    "mac": "68:21:5f:d2:f1:33",
    "platform": "x86_64-accton_as7816_64x-r0",
    "region": "None",
    "synchronous_mode": "enable",
    "type": "ToRRouter"
  }
},
```

Configuration of Routing Protocols

The startup configuration files in FRR is controlled by the setting of "docker_routing_config_mode" in config_db's DEVICE_META table.

There are three modes which can be set: "separated", "unified" and "split". Among them, "separated" mode is the default routing config mode in ec202012 (Note: "split" mode is the default in ec202006).

The different between these three routing config modes are as follow:

docker_routing_config_mode	Config files for FRR's daemons	Config files generation during docker startup
separated	individual configuration file for each daemons (bgpd.conf, ospfd.conf, staticd.conf, ... etc)	generates and overwrites configuration files using config_db
unified	one unified configuration file (frr.conf)	generates and overwrites configuration files using config_db
split	one unified configuration file (frr.conf)	won't modify any configuration file

Note: If add route by CLI in split mode, it needs to execute "copy running-config startup-config" in vtysh to preserve the route setting. Because it won't generates configuration files from config_db.

Full details of FRR routing support and configuration can be found at: <http://docs.frrouting.org/en/latest/index.html>

Enter the **vtysh** command to access the FRR VTY shell.

Example

```
admin@sonic:~$ vtysh

Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

sonic#
sonic# show route-map
ZEBRA:
route-map: RM_SET_SRC Invoked: 82595
  permit, sequence 10 Invoked 82595
    Match clauses:
    Set clauses:
      src 10.1.0.32
    Call clause:
    Action:
      Exit routemap
```

(202111.2) Border Gateway Protocol (BGP)

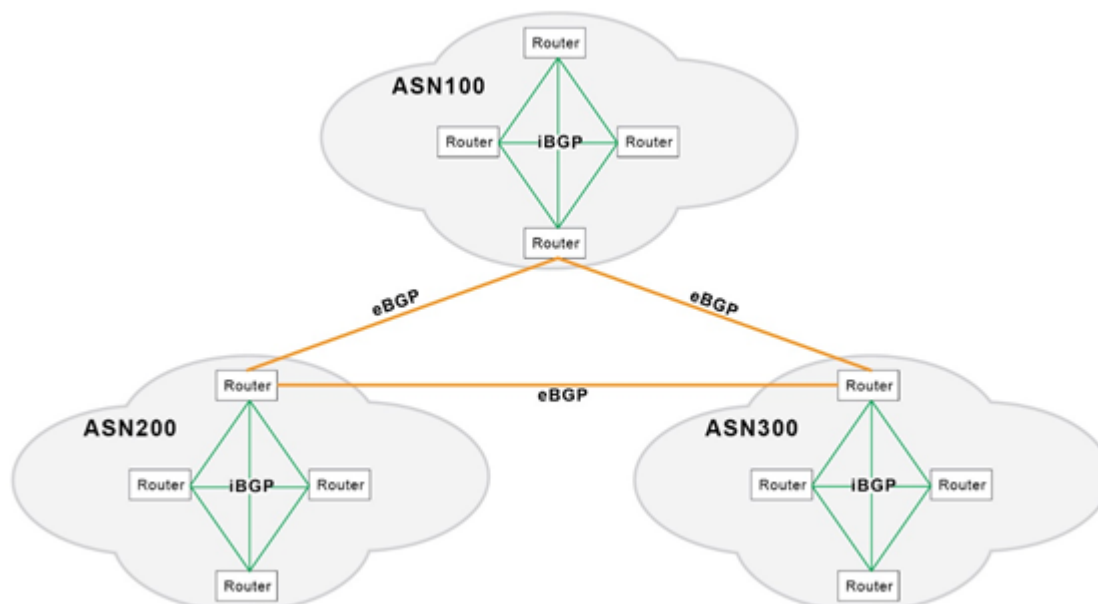
An autonomous system (AS) functions as a separate routing domain under one administrative authority, which implements its own routing policies. An AS exchanges routing information within its boundaries using Interior Gateway Protocols (IGPs) such as RIP or OSPF, and connects to external organizations or to the Internet using an Exterior Gateway Protocol (EGP). BGP version 4 is the primary EGP deployed on the Internet today.

A communication session must be maintained between bordering ASs to support the periodic exchange of routing information. One of the major design choices for BGP is the use of a TCP connection to exchange routing information between peers. Exchanging connectivity information over a reliable transport mechanism effectively delegates all error control functions to TCP.

The other major innovation for BGP is the use of path vectors that carry the full list of transit networks, or ASs, traversed between the source and destination. Loops are prevented simply by checking the path vector to see if same AS is listed twice. This approach solves many of the scalability problems encountered when applying distance-vector or link-state methods to make routing decisions in complex topologies.

External and Internal BGP

When connecting to the Internet, external BGP (eBGP) is used. Although BGP is widely used as an exterior gateway protocol (EGP), it is also used in many organizations with complex internal networks. Internal networks can be simplified by exchanging routing information among BGP peers within the same organization through internal BGP (iBGP) peering sessions.



External BGP - eBGP interconnects different ASs through border routers, or eBGP peers. These peering routers are commonly connected over a WAN link using a single physical path. Alternatively, multiple eBGP peer connections may be used to provide redundancy or load balancing. Distinct BGP sessions are used between redundancy peers to ensure that if one session fails, another will take over.

BGP uses the AS path attribute to record the ASs that must be followed to reach the prefix for a network aggregate. When a prefix is announced to an eBGP peer, the local AS number (ASN) is prepended to the AS path. This prevents routing loops by rejecting any prefix announcements that include the local ASN in the AS path. These announcements are also used by eBGP in the best path selection process.

eBGP speakers, can communicate with other external peers or with iBGP peers. A BGP speaker can determine if it is communicating with an external or internal peer by comparing the ASN sent in OPEN messages by a peer with that of its own internal value. If it matches, then this neighbor is an iBGP speaker, and if it does not, then it is an eBGP speaker. An eBGP speaker can advertise prefixes it has learned from another eBGP speaker to a neighboring iBGP speaker; and it can also advertise prefixes it has learned from an iBGP speaker to an eBGP speaker.

Internal BGP - In contrast to eBGP peers which have different ASNs, iBGP peers are configured with the same ASN. All iBGP peers within the same AS should be connected to one another in a full-mesh connection (except when using route reflection). When a prefix is announced from one iBGP peer to another, the AS path is not changed. Since all iBGP peers are fully meshed, they will have the same information in their BGP table, unless routing policies have been modified for some of the peers.

When an iBGP peer receives a prefix announcement, it uses the best path selection algorithm to see if the received announcement is the best path for that prefix. If it is, the peer inserts this route into its routing table, and announces it to all of its peers, both iBGP and eBGP. If it is not the best available path, the peer keeps a copy of it in its routing table so that if path information for that prefix changes (such as if the current best available path is withdrawn), it can be used to calculate a new best available path.

BGP cannot detect routes and provide reachability information. To ensure that each iBGP peer knows how to reach other, each peer must run some sort of Interior Gateway Protocol (such as static routes, direct routes, RIP or OSPF) which provides neighbor IP addresses. In order to avoid routing loops, an iBGP speaker cannot advertise prefixes it has learned from one iBGP peer to another neighboring iBGP peer.

Configuring SONiC BGP

SONiC uses the Free Range Routing (FRRouting or FRR) IP routing suite for the configuration of BGP and other routing protocols. The FRR protocol daemons can all be configured through the FRR VTY shell “vtysh.” Full details of FRR BGP configuration can be found at: <http://docs.frrouting.org/en/latest/bgp.html>

Enter the **vtysh** command to access the FRR VTY shell.

Example

```
admin@sonic:~$ vtysh

Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

sonic#
```

The commands available for BGP configuration in SONiC are limited. You can shutdown all BGP IPv4 and IPv6 sessions using the **config bgp shutdown all** command, or shutdown a specific BGP session with a neighbor using the **config bgp shutdown neighbor** command. Similarly, you can start up sessions with all neighbors or a specific neighbor using the **config bgp startup all** or **config bgp startup neighbor** commands. You can also remove a specific IPv4 or IPv6 BGP neighbor using the **config bgp remove neighbor** command.

Example

```
admin@sonic:~$ sudo config bgp shutdown all
Shutting down BGP session with neighbor 10.0.0.57...
Shutting down BGP session with neighbor 10.0.0.59...
Shutting down BGP session with neighbor 10.0.0.61...
Shutting down BGP session with neighbor 10.0.0.63...

admin@sonic:~$ sudo config bgp startup all
Starting up BGP session with neighbor 10.0.0.57...
Starting up BGP session with neighbor 10.0.0.59...
Starting up BGP session with neighbor 10.0.0.61...
Starting up BGP session with neighbor 10.0.0.63...

admin@sonic:~$ sudo config bgp remove neighbor 10.0.0.57
Removed configuration of BGP neighbor 10.0.0.57
```

To display the BGP configuration in SONiC, use the **show bgp summary** command.

Example

```
admin@sonic:~$ show bgp summary
```

IPv4 Unicast Summary:

BGP router identifier 10.1.0.32, local AS number 65100 vrf-id 0

BGP table version 6465

RIB entries 12807, using 2001 KiB of memory

Peers 4, using 83 KiB of memory

Peer groups 2, using 128 bytes of memory

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
10.0.0.57	4	64600	3995	4001	0	0	0	00:39:32	6400
10.0.0.59	4	64600	3995	3998	0	0	0	00:39:32	6400
10.0.0.61	4	64600	3995	4001	0	0	0	00:39:32	6400
10.0.0.63	4	64600	3995	3998	0	0	0	00:39:32	6400

Total number of neighbors 4

IPv6 Unicast Summary:

BGP router identifier 10.1.0.32, local AS number 65100 vrf-id 0

BGP table version 12803

RIB entries 12805, using 2001 KiB of memory

Peers 4, using 83 KiB of memory

Peer groups 2, using 128 bytes of memory

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
fc00::72	4	64600	3995	5208	0	0	0	00:39:30	6400
fc00::76	4	64600	3994	5208	0	0	0	00:39:30	6400
fc00::7a	4	64600	3993	5208	0	0	0	00:39:30	6400
fc00::7e	4	64600	3993	5208	0	0	0	00:39:30	6400

Total number of neighbors 4

For more information on SONiC BGP commands, see the [\(202111\) BGP](#) section in the CLI reference guide.

(202111.2) Bidirectional Forwarding Detection (BFD)

Bidirectional Forwarding Detection (BFD) is an IETF protocol (RFC5880) that is designed to rapidly detect failures in communications between systems. BFD functions independently of other routing protocols by checking if the next-hop router on a forwarding path is alive. Where routing protocols normally use “hello” methods to detect link failures that take seconds, BFD is able to detect forwarding path failures in milliseconds. The BFD protocol operates between two systems, sending unicast packets that can be carried in any encapsulation that might be implemented in a network.

In the SONiC system, BFD is included as part of the FRR BGP container. To configure a BFD peer, use the **peer** command in FRR BFD daemon configuration mode.

Example

```
admin@sonic:~$ vtysh

Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

sonic# config
sonic(config)# bfd
sonic(config-bfd)# peer 192.169.1.11
sonic(config-bfd-peer)# end
sonic# show bfd peer 192.169.1.11
BFD Peer:
    peer 192.169.1.11 vrf default
        ID: 1991080464
        Remote ID: 3120344281
        Status: up
        Uptime: 4 hour(s), 29 minute(s), 12 second(s)
        Diagnostics: ok
        Remote diagnostics: ok
        Local timers:
            Receive interval: 1000ms
            Transmission interval: 1000ms
            Echo transmission interval: 100ms
        Remote timers:
            Receive interval: 300ms
            Transmission interval: 300ms
            Echo transmission interval: 50ms
```

For more information on BFD configuration and BGP BFD commands, see the [\(202111\) BFD \(Basic\)](#) section in the CLI reference guide.

(202111.2) Virtual Routing and Forwarding (VRF)

Virtual Routing and Forwarding (VRF) is the support for multiple independent routing tables working simultaneously on the same switch. VRF-aware routing protocols, such as FRR BGP, are used to maintain separate VRF tables for each configured VRF instance (identified by a specified name). Once you have configured a VRF instance name, you can bind Layer 3 interfaces (VLANs, ports, and port channels) to the VRF. By default, all Layer 3 interfaces are bound to the default VRF.

Use the **config vrf add** command to create a VRF instance with a specified name. Note that VRF names should always start with "Vrf." Note that the name "mgmt" is reserved for the management VRF and should not be used.

To bind a Layer 3 interface to a non-default VRF, use the **config interface vrf bind** command. Use the **config interface vrf unbind** command to remove an interface from a non-default VRF.

Use the **show vrf** command to display all VRFs configured on the system.

Example

```
admin@sonic:~$ sudo config vrf add Vrf01

admin@sonic:~$ config interface vrf bind PortChannel0001 Vrf01

admin@sonic:~$ show vrf
VRF      Interfaces
-----
Vrf01    PortChannel0001
```

For more information on VRF commands, see the [\(202111\) VRF Configuration](#) and [\(202111\) Interfaces](#) sections in the CLI reference guide.

(202111.2) Management VRF Configuration

Management Virtual Routing and Forwarding (VRF) supports a separate routing table for management traffic that is forwarded by out-of-band management ports on a switch. This enables management traffic to be routed across a management network and still remain secure and separate from network data traffic.

Use the **config vrf add mgmt** command to enable the management VRF. This binds the management interface "eth0" to the "mgmt" VRF.

Use the **show mgmt-vrf** command to display details about the management VRF, or the **show mgmt-vrf routes** to display the management VRF routing table.

Example

```
admin@sonic:~$ sudo config vrf add mgmt
```

```
admin@sonic:~$ show mgmt-vrf
```

```
ManagementVRF : Enabled
```

```
Management VRF interfaces in Linux:
```

```
348: mgmt: <NOARP,MASTER,UP,LOWER_UP> mtu 65536 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether f2:2a:d9:bc:e8:f0 brd ff:ff:ff:ff:ff:ff
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master mgmt state UP mode DEFAULT group
default qlen 1000
    link/ether 4c:76:25:f4:f9:f3 brd ff:ff:ff:ff:ff:ff
350: lo-m: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue master mgmt state UNKNOWN mode DEFAULT
group default qlen 1000
    link/ether b2:4c:c6:f3:e9:92 brd ff:ff:ff:ff:ff:ff
```

```
admin@sonic:~$ show mgmt-vrf routes
```

```
Routes in Management VRF Routing Table:
```

```
default via 10.16.210.254 dev eth0 metric 201
broadcast 10.16.210.0 dev eth0 proto kernel scope link src 10.16.210.75
10.16.210.0/24 dev eth0 proto kernel scope link src 10.16.210.75
local 10.16.210.75 dev eth0 proto kernel scope host src 10.16.210.75
broadcast 10.16.210.255 dev eth0 proto kernel scope link src 10.16.210.75
broadcast 127.0.0.0 dev lo-m proto kernel scope link src 127.0.0.1
127.0.0.0/8 dev lo-m proto kernel scope link src 127.0.0.1
local 127.0.0.1 dev lo-m proto kernel scope host src 127.0.0.1
broadcast 127.255.255.255 dev lo-m proto kernel scope link src 127.0.0.1
```

For more information, see the [\(202111\) Management VRF](#) section in the CLI reference guide.

(202111.2) Leaking VRF Routes

Virtual Routing and Forwarding (VRF) supports multiple independent routing tables on the same switch. Although each VRF is essentially isolated from other VRFs, there are cases where it is a benefit for a VRF to share or “leak” information to another VRF. This VRF route leaking is where routes and next-hops of connected destinations in one VRF are made available to another VRF.

SONiC supports both static and dynamic VRF route leaking through the Free Range Routing (FRR) IP routing suite. For static VRF route leaking, the FRR CLI can be used to specify a next-hop IP that is reachable through a next-hop VRF. With dynamic VRF route leaking, route maps are used to automatically import routes from other VRFs. Prefix lists within route maps are used to match route prefixes in the source VRF and various actions can be applied. If a route map action is permitted, the matched routes will be imported into the destination VRF. The leaked routes are automatically deleted when the corresponding routes are deleted in the source VRF.

As an example of configuring a static VRF route leak, consider the following that leaks a next-hop IP route from a “Vrf1” into a “Vrf2.”

Example

```
admin@sonic:~# vtysh
sonic# configure
sonic(config)# vrf Vrf1
sonic(config-vrf)# ip route 2.2.2.2/32 10.0.0.59 nexthop-vrf Vrf2
sonic(config-vrf)# end
sonic# exit
admin@sonic:~#
admin@sonic:~# show ip route vrf Vrf1
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, D - SHARP,
       F - PBR, f - OpenFabric,
       > - selected route, * - FIB route, q - queued route, r - rejected route
VRF Vrf1:
S>* 2.2.2.2/32 [1/0] via 10.0.0.59, PortChannel0002(vrf Vrf2), 01:34:15
C>* 10.0.0.56/31 is directly connected, PortChannel0001, 01:40:04
```

To configure a dynamic VRF route leak, consider the following example that imports BGP routes from the default VRF into “Vrf1.”

Example

```
admin@sonic:~# vtysh
sonic# configure
sonic(config)# router bgp 65200 vrf Vrf1
sonic(config-router)# address-family ipv4 unicast
sonic(config-router-af)# import vrf default
sonic(config-router-af)# end
sonic# exit
admin@sonic:~#
admin@sonic:~# show ip route vrf Vrf1 193.11.248.128
Routing entry for 193.11.248.128/25
  Known via "bgp", distance 200, metric 0, vrf Vrf1, best
  Last update 00:03:47 ago
    * 10.0.0.59, via PortChannel0002(vrf default)
    * 10.0.0.61, via PortChannel0003(vrf default)
    * 10.0.0.63, via PortChannel0004(vrf default)
```

For more information, see the [\(202111\) VRF Route Leaking](#) section in the CLI reference guide, or refer to details in the FRR manual at: <http://docs.frrouting.org/en/latest/index.html>

(202111.2) Network Address Translation (NAT)

Network Address Translation (NAT) is a standard method of mapping “internal” (private) network IP addresses to “external” (public) IP address on devices at the edge of a network. Typically, NAT not only translates the inbound and outbound IP addresses of network traffic, but also Layer 4 ports. This allows multiple internal IP addresses to “share” one public external IP address and also provides a level of security for private network hosts that remain “hidden” from the public network.



- NAT is supported on the Broadcom Trident2+, Trident3, Tomahawk, Tomahawk2 and Intel Tofino, Tofino2 switches, including the following models:
 - DCS810(AS9716-32X) (Intel Tofino2 with **Y2** profile)
 - DCS800(Wedge100BF) (Intel Tofino with **X2** profile)
 - DCS510(AS7712-32X) (Broadcom Tomahawk)
 - DCS500(AS7816-64X) (Broadcom Tomahawk2)
 - DCS208(AS5812-54X) (Broadcom Trident2+)
 - DCS203(AS7326-56X, AS7726-32X) (Broadcom Trident3)
 - DCS201(AS5835-54X) (Broadcom Trident3)
- Static NAT/NAPT is supported to translate all packets with IP header (including TCP, UDP) on **Broadcom** switches. On **Intel** switches is supported only for the traffic of IP protocol types TCP/UDP/ICMP.

To enable NAT, use the **config nat feature** command. To configure the NAT IP address pool, use the **config nat add pool** command. To configure a static NAT entry, use the **config nat add static** command. To configure a NAT zone on a Layer 3 interface, use the **config nat add interface** command.

Example

```
admin@sonic:~$ sudo config nat feature enable

admin@sonic:~$ sudo config nat add pool pool1 65.55.45.2-65.55.45.10

admin@sonic:~$ sudo config nat add static basic 65.55.45.1 12.12.12.14 -nat_type dnat

admin@sonic:~$ sudo config nat add interface Ethernet28 -nat_zone 1
```

To show the NAT configuration, use the **show nat config** command. Or, to show the NAT entries, use the **show nat translations** command.

Example

```
admin@sonic:~$ show nat config static
```

Nat Type	IP Protocol	Global IP	Global L4 Port	Local IP	Local L4 Port	Twice-Nat Id
dnat	all	65.55.45.5	---	10.0.0.1	---	---
dnat	all	65.55.45.6	---	10.0.0.2	---	---
dnat	tcp	65.55.45.7	2000	20.0.0.1	4500	1
snat	tcp	20.0.0.2	4000	65.55.45.8	1030	

```
admin@sonic:~$ show nat translations
```

```
Static NAT Entries           ..... 4
Static NAPT Entries          ..... 2
Dynamic NAT Entries           ..... 0
Dynamic NAPT Entries          ..... 4
Static Twice NAT Entries      ..... 0
Static Twice NAPT Entries     ..... 4
Dynamic Twice NAT Entries     ..... 0
Dynamic Twice NAPT Entries     ..... 0
Total SNAT/SNAPT Entries      ..... 9
Total DNAT/DNAPT Entries      ..... 9
Total Entries                 ..... 14
```

Protocol	Source	Destination	Translated Source	Translated Destination
all	10.0.0.1	---	65.55.42.2	---
all	---	65.55.42.2	---	10.0.0.1
all	10.0.0.2	---	65.55.42.3	---
all	---	65.55.42.3	---	10.0.0.2
tcp	20.0.0.1:4500	---	65.55.42.1:2000	---
tcp	---	65.55.42.1:2000	---	20.0.0.1:4500
udp	20.0.0.1:4000	---	65.55.42.1:1030	---
udp	---	65.55.42.1:1030	---	20.0.0.1:4000
tcp	20.0.0.1:6000	---	65.55.42.1:1024	---
tcp	---	65.55.42.1:1024	---	20.0.0.1:6000
tcp	20.0.0.1:5000	65.55.42.1:2000	65.55.42.1:1025	20.0.0.1:4500
tcp	20.0.0.1:4500	65.55.42.1:1025	65.55.42.1:2000	20.0.0.1:5000
tcp	20.0.0.1:5500	65.55.42.1:2000	65.55.42.1:1026	20.0.0.1:4500
tcp	20.0.0.1:4500	65.55.42.1:1026	65.55.42.1:2000	20.0.0.1:5500

For more information, see the [\(202111\) NAT](#) section in the CLI reference guide.

(202111.2) Displaying ARP Entries

The Address Resolution Protocol (ARP) is used to map IP addresses to a MAC addresses. When an IP frame is received by a router, it first looks up the MAC address corresponding to the destination IP address in the ARP cache. If the address is found, the router writes the MAC address into the appropriate field in the frame header, and forwards the frame on to the next hop. IP traffic passes along the path to its final destination in this way, with each routing device mapping the destination IP address to the MAC address of the next hop toward the recipient, until the packet is delivered to the final destination.

To display entries in the ARP cache, use the **show arp** command to either list all entries, or list for a specific IP address or interface.

Example

```
admin@sonic:~$ show arp
  Address           MacAddress          Iface      Vlan
  -----
192.168.1.183      88:5a:92:fb:bf:41   Ethernet44 -
192.168.1.175      88:5a:92:fc:95:81   Ethernet28 -
192.168.1.181      e4:c7:22:c1:07:7c   Ethernet40 -
192.168.1.179      88:5a:92:de:a8:bc   Ethernet36 -
192.168.1.118      00:1c:73:3c:de:43   Ethernet64 -
192.168.1.11       00:1c:73:3c:e1:38   Ethernet88 -
192.168.1.161      24:e9:b3:71:3a:01   Ethernet0  -
192.168.1.189      24:e9:b3:9d:57:41   Ethernet56 -
192.168.1.187      74:26:ac:8b:8f:c1   Ethernet52 -
192.168.1.165      88:5a:92:de:a0:7c   Ethernet8  -

Total number of entries 10
```

For more information, see the [\(202111\) ARP](#) section in the CLI reference guide.

(202111.2) EVPN/VxLAN

- What is the EVPN/VxLAN
- How it works
- Example of EVPN/VxLAN Configuration
 - Topology
 - Environment Setup
- How about the forwarding among multi-VTEPs and multi-VNI/VLAN
 - When tunnel be created
 - When the (VNI, VLAN) mapping provisioned
 - Handle the customer VLAN in encapsulation/decapsulation
 - How the forwarding works
- Reference
- Terminology

What is the EVPN/VxLAN

EVPN/VxLAN provides an easy way to construct the VxLAN tunnel and reduce flooding with managing Layer 2 addresses and Layer 3 routing information across the tunnels. For example, advertising the MAC addresses learned from CEs to peer endpoints

EVPN, Ethernet VPNs, uses MP-BGP mechanism to exchange information among PEs.

VxLAN constructs a tunnel between two endpoints(VTEPs) over Layer 3 network.

How it works

EVPN uses MP-BGP to exchange routing information, e.g. Type 2(MAC/IP advertisement), Type 3(IMET, Inclusive Multicast Ethernet Tag), and Type 5(IP prefix). The VxLAN tunnel is automatically created when two VTEPs are reachable over Layer 3 IP network. This reduces the overhead of managing large scale (or complex) networks.

VxLAN uses VNI to indicate the bridging domain (aka VNI domain or bridging domain), which flooding the frames over the domain if it is not learned in the VTEP. Once the MAC address is learned in one VTEP, EVPN uses MP-BGP actively synchronizing the learned MAC address to VTEPs of the same VNI domain and provisioning to data plane. The same mechanism applies to MAC address with IP address (type 2) and IP prefix route (type 5). **VNI** is a field of VxLAN header. The original packet is encapsulated as payload of VxLAN packet and forwarded over IP network (aka underlay network) from one VTEP to peer VTEP of the tunnel, which links both segments of VTEPs as one connected segment.

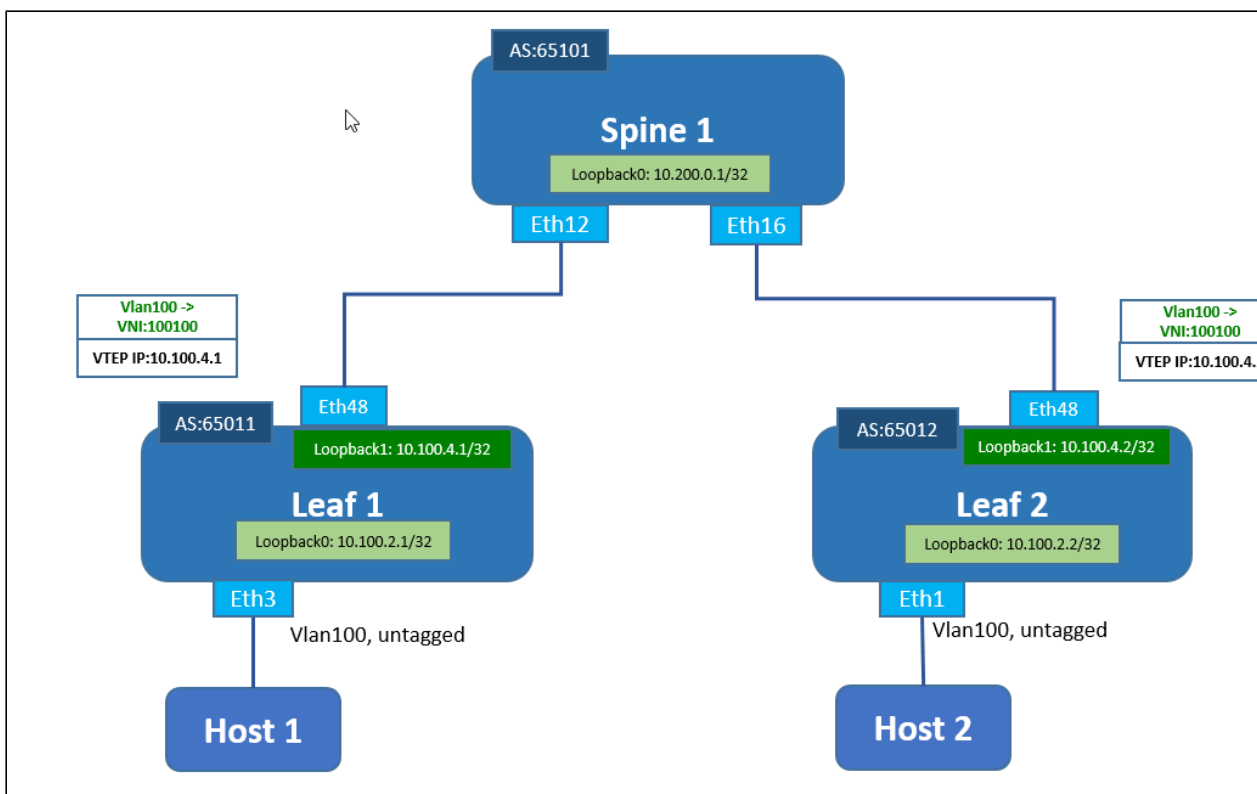
With EVPN/VxLAN capability, deploying devices which require Layer 2 connectivity across different IP subnets, for example Campus, become flexible and reuse existing IP network and architecture. In data center, IP fabric (or Spine-Leaf architecture) is simplified and extensible when introducing EVPN/VxLAN. User's hosts in different racks can be easily configured along leaf and formed as a tenant. Multi-tenancy is maturely built-in, user's VM hosts can be assigned into the same VNI on different rack across different zones and worked as connected devices in L2/L3 network. It also handles MAC address mobility in EVPN control plane.

Example of EVPN/VxLAN Configuration

In this example, it shows how to configure the SPINE and LEAFs, and create a tunnel between two VTEPs (endpoints) across IP network of LEAF - SPINE - LEAF(a CLOS network). As the L2VPN is announced and VTEPs are created, "host1" can ping "host2" even there are not physically connected.

The "host1" in the drawing can be a switch which connects a lots of hosts. All traffic, from "host1" to remote "host2", would be encapsulated into VxLAN packet (VNI 100100) which mapped from V100 to VNI.100100 in "Leaf1" and forwarded to "Leaf2".

Topology



Environment Setup

The configuration here, BGP Session establishment is using BGP unnumbered, it's doesn't require configuration ip on peering interface, establishment is automated by IPv6 Link local neighbor discovery.

1. Config BGP and L2VPN EVPN in spine-1
 - a. spine-1

```
admin@spine-1:~$ vtysh -c "configure" -c "interface Ethernet12" -c "ipv6 nd ra-interval 10" -c
"no ipv6 nd suppress-ra"
admin@spine-1:~$ vtysh -c "configure" -c "interface Ethernet16" -c "ipv6 nd ra-interval 10" -c
"no ipv6 nd suppress-ra"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "bgp router-id 10.200.0.1" -c
"no bgp ebgp-requires-policy"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "neighbor FABRIC peer-group"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "neighbor FABRIC remote-as
external"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "neighbor FABRIC capability
extended-nexthop"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "neighbor Ethernet12 interface
peer-group FABRIC"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "neighbor Ethernet16 interface
peer-group FABRIC"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "address-family ipv4 unicast" -c
"network 10.200.0.1/32" -c "neighbor FABRIC soft-reconfiguration inbound"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "address-family ipv6 unicast" -c
"neighbor FABRIC activate"
admin@spine-1:~$ vtysh -c "configure" -c "router bgp 65101" -c "address-family l2vpn evpn" -c
"neighbor FABRIC activate"
```

2. Create VLAN 100 and add Ethernet3 of leaf-1 and Ethernet1 of leaf-2 to VLAN 100
 - a. ext-leaf-1

```
admin@leaf-1:~$ sudo config vlan add 100
admin@leaf-1:~$ sudo config vlan member add -U 100 Ethernet3
```

- b. ext-leaf-2

```
admin@leaf-2:~$ sudo config vlan add 100
admin@leaf-2:~$ sudo config vlan member add -U 100 Ethernet1
```

3. Configure IP address to Loopback1 of both switches..

- a. leaf-1

```
admin@leaf-1:~$ sudo config interface ip add Loopback1 10.100.4.1/32
```

- b. leaf-2

```
admin@leaf-2:~$ sudo config interface ip add Loopback1 10.100.4.2/32
```

4. Establish BGP Session between Ethernet48 and announce the network.
 - a. leaf-1

```
admin@leaf-1:~$ vtysh -c "configure" -c "interface Ethernet48" -c "ipv6 nd ra-interval 10" -c
"no ipv6 nd suppress-ra"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "bgp router-id 10.100.2.1" -c "no
bgp ebgp-requires-policy"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "neighbor FABRIC peer-group"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "neighbor FABRIC remote-as
external"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "neighbor FABRIC capability
extended-nexthop"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "neighbor Ethernet48 interface
peer-group FABRIC"
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "address-family ipv4 unicast" -c
"network 10.100.2.1/32" -c "neighbor FABRIC soft-reconfiguration inbound"
```

b. leaf-2

```
admin@leaf-2:~$ vtysh -c "configure" -c "interface Ethernet48" -c "ipv6 nd ra-interval 10" -c
"no ipv6 nd suppress-ra"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "bgp router-id 10.100.2.2" -c "no
bgp ebgp-requires-policy"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "neighbor FABRIC peer-group"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "neighbor FABRIC remote-as
external"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "neighbor FABRIC capability
extended-nexthop"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "neighbor Ethernet48 interface
peer-group FABRIC"
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "address-family ipv4 unicast" -c
"network 10.100.2.2/32" -c "neighbor FABRIC soft-reconfiguration inbound"
```

5. Announce L2VPN EVPN routes.

a. leaf-1

```
admin@leaf-1:~$ vtysh -c "configure" -c "router bgp 65011" -c "address-family l2vpn evpn" -c
"neighbor FABRIC activate" -c "advertise-all-vni"
```

b. leaf-2

```
admin@leaf-2:~$ vtysh -c "configure" -c "router bgp 65012" -c "address-family l2vpn evpn" -c
"neighbor FABRIC activate" -c "advertise-all-vni"
```

6. Create VxLAN

a. leaf-1

```
admin@leaf-1:~$ sudo config vxlan add vtep 10.100.4.1
admin@leaf-1:~$ sudo config vxlan evpn_nvo add nvo vtep
admin@leaf-1:~$ sudo config vxlan map add vtep 100 100100
```

b. leaf-2

```
admin@leaf-2:~$ sudo config vxlan add vtep 10.100.4.2
admin@leaf-2:~$ sudo config vxlan evpn_nvo add nvo vtep
admin@leaf-2:~$ sudo config vxlan map add vtep 100 100100
```

7. Check VxLAN status. Vxlan tunnel will be connected automatically.

a. leaf-1

```
admin@leaf-1:~$ show vxlan
interface

VTEP
Information:
VTEP Name : vtep, SIP :
10.100.4.1

NVO Name : evpnnv01, VTEP :
vtep

Source interface : Loopback1

admin@leaf-1:~$ show vxlan
tunnel

vxlan tunnel name    source ip    destination ip    tunnel map name    tunnel map mapping(vni -
>
> vlan)
-----
-----

vtep                10.100.4.1                map_100100_Vlan100  100100 -> Vlan100

admin@leaf-1:~$ show vxlan
remotevtep

+-----+-----+-----+-----+
+
| SIP      | DIP      | Creation Source | OperStatus
|
+=====+=====+=====+=====+
| 10.100.4.1 | 10.100.4.2 | EVPN           | oper_up
|
+-----+-----+-----+-----+
+

Total count : 1
```

b. leaf-2

```
admin@leaf-2:~$ show vxlan
interface

VTEP
Information:
VTEP Name : vtep, SIP :
10.100.4.2

NVO Name : evpnvno1, VTEP :
vtep

Source interface : Loopback1

admin@leaf-2:~$ show vxlan
tunnel

vxlan tunnel name      source ip      destination ip      tunnel map name      tunnel map mapping(vni -
>
vlan)
-----
vtep                  10.100.4.2                  map_10100_Vlan100  100100 -> Vlan100

admin@leaf-2:~$ show vxlan
remotevtep

+-----+-----+-----+-----+
+
| SIP      | DIP      | Creation Source | OperStatus
|
+=====+=====+=====+=====+
| 10.100.4.2 | 10.100.4.1 | EVPN           | oper_up
|
+-----+-----+-----+-----+
+

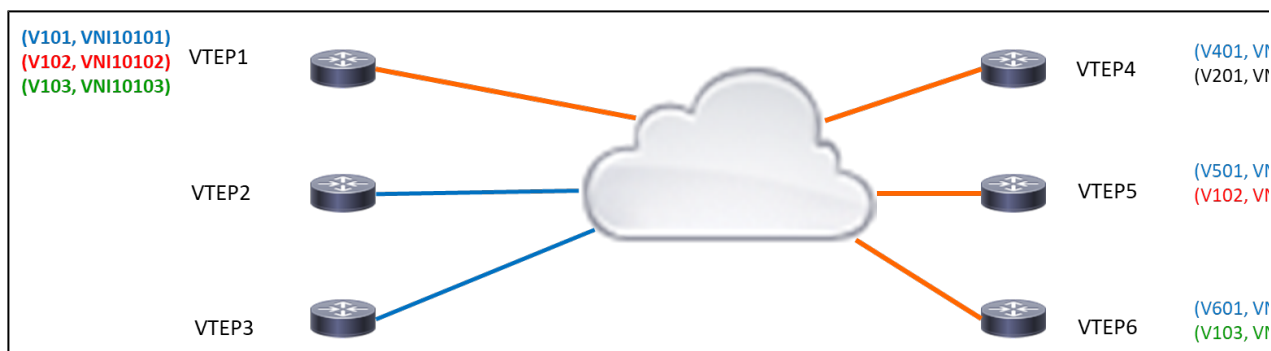
Total count : 1
```

EVPN/VxLAN capacity

The maximum number of VxLAN tunnels can be created is limited by chipset (or hardware model) and use case. Please contact representatives before making your network plan.

How about the forwarding among multi-VTEPs and multi-VNI/VLAN

Let's use following drawing to explain the forwarding among multi-VTEPs and VNIs.



In this architecture, there are 6 nodes configured as VTEP1..VTEP6, and each node has configured with VLAN and VNI mapping to form the VNI domain. For example, VNI10101, there are four nodes (VTEP1, VTEP4, VTEP5, VTEP6) joined which form a bridging domain for learning and forwarding.

When tunnel be created

Tunnel is created when the endpoints (VTEPs) can be reached via Layer 3 IP network. For example, via BGP routes exchanging, VTEP1 knows it can reach VTEP4..VTEP6, then VTEP1 creates 3 different tunnels to VTEP4..VTEP6. This uses 3 entries of tunnel table on endpoint (VTEP1). This also occurred in VTEP2.. VTEP6 to create the tunnel to the peer endpoint (VTEP1) if it is reachable in Layer 3 IP network.

As the tunnel created, EVPN on VTEP1 delivers VNI list (VNI10101, VNI10102, VNI10103) to peer endpoints (VTEP4, VTEP5, VTEP6). With this VNI list, the endpoint (eg VTEP4) knows - if there is (destination) unknown frame, which mapped to VNI 10101, it can flood the frames to VTEP1. Of course, EVPN delivers the learned information to peer endpoints to reduce the flooding on the IP network, aka "underlayer network".

When the (VNI, VLAN) mapping provisioned

The mapping of (VNI, VLAN) is provisioned when the configuration is loaded. It uses entries of VNI table and handles the encapsulation /decapsulation to/from VxLAN tunnel.

Handle the customer VLAN in encapsulation/decapsulation

Current implementation of EVPN/VxLAN is VLAN base, which

- encapsulation: the customer VLAN-ID is not encapsulated in VxLAN frame. It is used to derive the VNI number and filled in VxLAN header.
- decapsulation: with the VNI number and referring the mapping, the VLAN-ID is filled in tagged frame and forward to customer device.

How the forwarding works

Forwarding an unknown destination frame, which is never known (or learned) in VTEPs, the VTEP would flood the frame to all VTEPs which join the VNI domain. For example, VTEP1 floods the (V101) unknown frame to VTEP4, VTEP5, and VTEP6 since they join VNI-10101.

If the frame is learned in VTEP4 and propagated to VTEP1 via EVPN protocol, VTEP1 uses unicast and forward the frame through the tunnel between VTEP1 and VTEP4.

Reference

- RFC4760, Multiprotocol Extensions for BGP-4
- RFC7348, Virtual eXtensible Local Area Network (VXLAN)
- RFC7432, BGP MPLS-Based Ethernet VPN

Terminology

CE: Customer Edge device. eg host, router, switch

PE: Provider Edge device

VTEP: VXLAN Tunnel End Point, termination of VXLAN tunnel.

VxLAN: Virtual eXtension Local Area Network

(202111.2) EVPN-MH

EVPN multihoming (EVPN-MH) provides all-active mode of redundancy support for host connection through LAG (port-channel). It is a standard protocol which distribute routes through BGP extension protocol and intended to become a replacement/alternative of MCLAG active-active mode.

An Ethernet Segment is a set of links on different routers which connected to the same host operated as a normal LAG which provides L2-ECMP and redundancy functionally. An Ethernet Segment is uniquely identified by a 10-byte Ethernet Segment Identifier (ESI).

EVPN-MH uses EVPN type-1 and type-4 routes to distributed Ethernet Segment (ES) information in an EVPN instance. It also extends type-2 routes to support MACs/neighbors learning on ES besides on Vxlan tunnels.

EVPN-MH normally configured on top of the EVPN/VxLAN feature. For the usage of EVPN/VxLAN configuration, please refer to the EVPN/VxLAN user guide: [\(202111.2\) EVPN/VxLAN](#).

Basic EVPN-MH Configuration

To configure EVPN-MH:

1. Enable EVPN and create EVPN instance (VNI)
2. Create PortChannel interface
3. Configure Ethernet Segment ID (ESI), choose one of the following
 - Configure Local Discriminator value and system MAC address on PortChannel interface
 - Configure ESI with arbitrary 9-octet value which starts with 0x00

Ethernet Segment ID (ESI) Format

ESI is encoded as a 10-octet integer which consisted of (from high to low order):

```
+-----+-----+-----+-----+-----+
| T |               ESI Value               |
+-----+-----+-----+-----+-----+
```

- Type 0 (T=0x00):
 - 1 octet of ES type
 - 9 octets of arbitrary value, which is managed and configured by the operator
- Type 3 (T=0x03)
 - 1 octet of ES type: currently fixed of value 0x03 indicated it is a MAC-based ESI Value that can be auto-generated or configured by the operator.
 - 6 octets of system MAC address which is used as the representative of the provider edge (PE) device's MAC address connected to the host
 - 3 octets of Local Discriminator value (1-16777215)

Configure Ethernet Segment Local Discriminator and System MAC address

The Ethernet Segment Local Discriminator value and system MAC address must be configured in both SONiC CLI and FRR's vtysh shell.

The DF preference is configured per-ES in FRR's vtysh shell.

configure ethernet segment example (type 0)

```
admin@as7326-56x:~$ sudo config vxlan evpn_es add PortChannel0001 00:11:22:33:44:55:66:77:88:99
admin@as7326-56x:~$ vtysh
```

```
Hello, this is FRRouting (version 8.0).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
```

```
as7326-56x# configure
as7326-56x(config)# in PortChannel0001
as7326-56x(config-if)# evpn mh es-id 00:11:22:33:44:55:66:77:88:99
as7326-56x(config-if)# evpn mh es-df-pref 123
as7326-56x(config-if)# end
as7326-56x# exit
admin@as7326-56x:~$
```

configure ethernet segment example (type 3)

```
admin@as7326-56x:~$ sudo config vxlan evpn_es add PortChannel0001 168 00:11:22:33:ab:cd
admin@as7326-56x:~$ vtysh

Hello, this is FRRouting (version 8.0).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

as7326-56x# configure
as7326-56x(config)# in PortChannel0001
as7326-56x(config-if)# evpn mh es-id 168
as7326-56x(config-if)# evpn mh es-sys-mac 00:11:22:33:ab:cd
as7326-56x(config-if)# evpn mh es-df-pref 123
as7326-56x(config-if)# end
as7326-56x# exit
admin@as7326-56x:~$
```

Show Ethernet Segment

To show the Ethernet Segment, use "show vxlan evpn es" command. The 'status' field indicates the operation status of the corresponding PortChannel interface.

show vxlan evpn es example

```
admin@as7326-56x:~$ show vxlan evpn es
Type: L local, R remote, N non-DF
```

ESI	Type	ES Interface	status	VTEPS
03:44:38:39:ab:cd:01:00:00:01	LRN	PortChannel0001	down	1.1.1.1 2.2.2.2 3.3.3.3 4.4.4.4 5.5.5.5
03:44:38:39:ab:cd:01:00:00:02	L	PortChannel0002	up	
03:44:38:39:ab:cd:01:00:00:03	RN			172.0.0.24 172.0.0.25 172.0.0.26

```
Total count : 3

admin@as7326-56x:~$
```

Show Remote MAC Address

To show all learned remote MAC addresses by EVPN (includes EVPN-MH), use 'show vxlan remotemac' command. Option 'all' means display all remote MAC addresses and option ESI means display MAC address learned from specific Ethernet Segment.

show vxlan remotemac example

```
admin@as7326-56x:~$ show vxlan remotemac all
+-----+-----+-----+-----+-----+-----+
| VLAN   | MAC               | RemoteVTEP | ESI               | VNI | Type   |
+=====+=====+=====+=====+=====+=====+
| Vlan1000 | 00:12:34:56:78:90 | 10.0.0.65   |                   | 100 | dynamic |
+-----+-----+-----+-----+-----+-----+
| Vlan1000 | 00:55:66:77:88:AA |             | 03:00:11:22:33:44:55:00:00:a8 | 100 | dynamic |
+-----+-----+-----+-----+-----+-----+
Total count : 2

admin@as7326-56x:~$ show vxlan remotemac 03:00:11:22:33:44:55:00:00:a8
+-----+-----+-----+-----+-----+-----+
| VLAN   | MAC               | RemoteVTEP | ESI               | VNI | Type   |
+=====+=====+=====+=====+=====+=====+
| Vlan1000 | 00:55:66:77:88:AA |             | 03:00:11:22:33:44:55:00:00:a8 | 100 | dynamic |
+-----+-----+-----+-----+-----+-----+
Total count : 1

admin@as7326-56x:~$
```

Uplink Tracking

Uplink tracking is configured in FRR's vtysh shell.

configure uplink tracking example

```
admin@as7326-56x:~$ vtysh

Hello, this is FRRouting (version 8.0).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

as7326-56x# configure
as7326-56x(config)# in PortChannel0001
as7326-56x(config-if)# evpn mh uplink
as7326-56x(config-if)# end
as7326-56x# exit
admin@as7326-56x:~$
```

EVPN-MH Capacity

The maximal number of ES supported is different among devices, please refer to: [ecSONiC 202012 Scaling Factors Matrix on 2021 June](#)

The maximal number of member supported per ES is 10.

(202111.2) DHCP Relay on EVPN/VxLAN

Overview

Dynamic Host Configuration Protocol (DHCP) protocol is essential in data center deployments. In traditional usage, DHCP clients and DHCP servers must be located in the same VLAN. DHCP relay agent breaks this limitation for which it helps to forward DHCP message between DHCP clients and DHCP servers over the networks. With DHCP relay help, both DHCP clients and DHCP servers do not need any extra configuration compared to their traditional usage.

In many data center environments, EVPN/VxLAN uses underlay IP fabrics to provide both Layer 2 and Layer 3 connectivity between hosts/servers.

In this document, we describe the solution and provide a configuration example that uses DHCP relay on EVPN/VxLAN networks. DHCP clients and DHCP servers may be connected to different VNIs and may also be located at different VTEPs. User must correctly configure the VLAN-VNI mapping and/or IRB (integrated routing and bridging) on the spine/leaf devices such that DHCP traffic can be forwarded between DHCP client and server. DHCP relay function is also supported on both normal VLANs or SAG (static anycast gateway) interfaces.

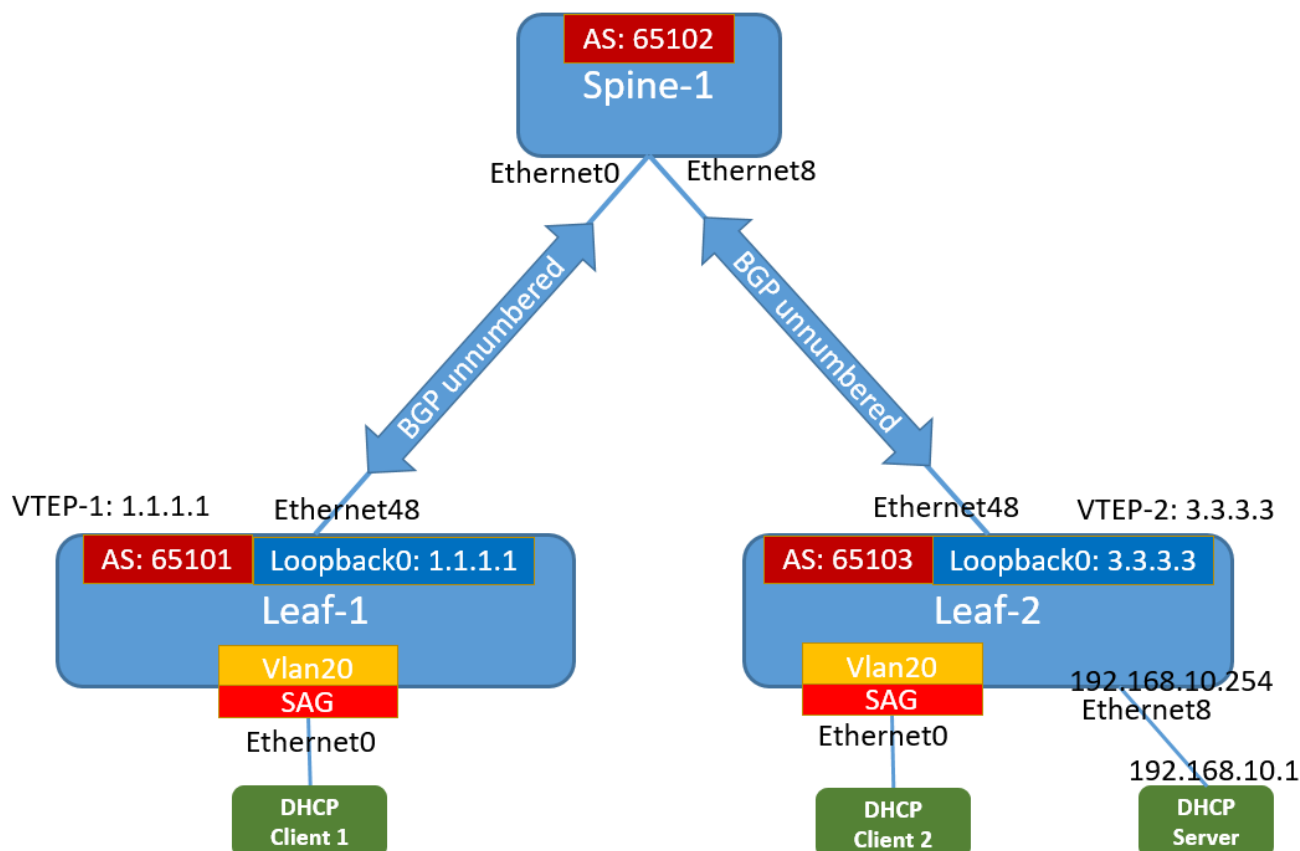
Use Case Scenario

In the following scenario, we do the following setting:

1. CLOS (spine-leaf) topology with eBGP routers running bgp unnumbered between them to exchange routes
2. Deploy a symmetric IRB EVPN/VxLAN (L3-VRF)
3. Configure SAG on VLAN interfaces which map to VNIs
4. Attach DHCP server into the L3-VRF

Topology

DHCP client1 connected to Vlan20 interface in Leaf1, DHCP client2 connected to Vlan20 interface in Leaf2, and DHCP server connected to Ethernet8 in Leaf2.



Configurations

DHCP Relay

To configure DHCP relay, the network administrator needs to select the VLAN interface from which DHCP requests are received, and the server that services those DHCP requests on the VLAN interface. Note that the server IP may access through EVPN/VxLAN overlay networks.

The following example adding Vlan20 into relay service and setting their server as 192.168.10.1.

DHCP Relay Configuration Example

```
admin@sonic:~$ sudo config vlan dhcp_relay add 20 192.168.10.1
```

Static Anycast Gateway (SAG) (Optional)

Create EVPN/VxLAN

```
admin@sonic:~$ sudo config sag mac_address add 00:11:22:33:44:55
admin@sonic:~$ sudo config sag ipv4 enable
admin@sonic:~$ sudo config interface sag ip add Vlan20 192.168.20.254/24
```

EVPN/VxLAN

There are two parts in ecSONiC which need to be configured to enable the EVPN/VxLAN feature.

In SONiC CLI part, we need to create EVPN/VxLAN and add corresponding VLAN-VNI mapping and L3-VRF. In the following example, we create a EVPN/VxLAN with VTEP 1.1.1.1, adding VLAN-VNI mapping of Vlan20/30 to VNI 2000/3000 correspondingly. Further more, since we want to use symmetric IRB in VxLAN overlay routing, a L3-VRF (Vrf1) is created and mapping to L3-VNI 3000.

Create EVPN/VxLAN

```
admin@sonic:~$ sudo config vxlan add vtep 1.1.1.1
admin@sonic:~$ sudo config vxlan evpn_nvo add nvo vtep
admin@sonic:~$ sudo config vxlan map add vtep 20 2000
admin@sonic:~$ sudo config vxlan map add vtep 30 3000
admin@sonic:~$ sudo config vrf add_vrf_vni_map Vrf1 3000
```

In FRR's vtysh shell, we do the same configuration corresponding to SONiC CLI commands, including enable the EVPN routes exchange with neighbors, create L3-VRF and announce local network as EVPN type-5 route.

Configure in frr

```
sonic# configure terminal
sonic(config)# router bgp 65101
sonic(config-router)# neighbor Ethernet48 interface remote-as external
sonic(config-router)# network 1.1.1.1/32
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# neighbor Ethernet48 activate
sonic(config-router-af)# advertise-all-vni
sonic(config-router-af)# end
sonic# configure terminal
sonic(config)# vrf Vrf1
sonic(config-vrf)# vni 3000
sonic(config-vrf)# exit
sonic(config)# router bgp 65101 vrf Vrf1
sonic(config-router)# address-family ipv4
sonic(config-router-af)# network 192.168.20.0/24
sonic(config-router-af)# exit
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# advertise ipv4 unicast
```

Leaf1 Configuration Example

Leaf1 configuration

Step 1: Create VLAN and VLAN member

```
admin@sonic:~$ sudo config vlan add 20
admin@sonic:~$ sudo config vlan add 30
admin@sonic:~$ sudo config vlan member add 20 -u Ethernet0
```

Step 2: Create VRF

```
admin@sonic:~$ sudo config vrf add Vrf1
admin@sonic:~$ sudo config interface vrf bind Vlan20 Vrf1
admin@sonic:~$ sudo config interface vrf bind Vlan30 Vrf1
```

Step 3: Enable static anycast gateway on Vlan20

```
admin@sonic:~$ sudo config sag mac_address add 00:11:22:33:44:55
admin@sonic:~$ sudo config sag ipv4 enable
admin@sonic:~$ sudo config interface sag ip add Vlan20 192.168.20.254/24
```

Step 4: Add IP address on Loopback0

```
admin@sonic:~$ sudo config interface ip add Loopback0 1.1.1.1/32
```

Step 5: enable VxLan

```
admin@sonic:~$ sudo config vxlan add vtep 1.1.1.1
admin@sonic:~$ sudo config vxlan evpn_nvo add nvo vtep
admin@sonic:~$ sudo config vxlan map add vtep 20 2000
admin@sonic:~$ sudo config vxlan map add vtep 30 3000
admin@sonic:~$ sudo config vrf add_vrf_vni_map Vrf1 3000
```

Step 6: Enable ARP suppress

```
admin@sonic:~$ sudo config neigh_suppress enable 20
```

Step 7: establish the BGP session.

```
admin@sonic:~$ vtysh
```

```
Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
```

```
sonic# configure terminal
sonic(config)# router bgp 65101
sonic(config-router)# neighbor Ethernet48 interface remote-as external
sonic(config-router)# network 1.1.1.1/32
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# neighbor Ethernet48 activate
sonic(config-router-af)# advertise-all-vni
sonic(config-router-af)# end
sonic# configure terminal
sonic(config)# vrf Vrf1
sonic(config-vrf)# vni 3000
sonic(config-vrf)# exit
sonic(config)# router bgp 65101 vrf Vrf1
sonic(config-router)# address-family ipv4
sonic(config-router-af)# network 192.168.20.0/24
sonic(config-router-af)# exit
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# advertise ipv4 unicast
```

Step 8: enable DHCP relay on Vlan20.

```
admin@sonic:~$ sudo config vlan dhcp_relay add 20 192.168.10.1
```

Leaf2 Configuration Example

Leaf2 Configuration

Step 1: Create VLAN and VLAN member

```
admin@sonic:~$ sudo config vlan add 20
admin@sonic:~$ sudo config vlan add 30
admin@sonic:~$ sudo config vlan member add 20 -u Ethernet0
```

Step 2: Create VRF

```
admin@sonic:~$ sudo config vrf add Vrf1
admin@sonic:~$ sudo config interface vrf bind Vlan20 Vrf1
admin@sonic:~$ sudo config interface vrf bind Vlan30 Vrf1
admin@sonic:~$ sudo config interface vrf bind Ethernet8 Vrf1
```

Step 3: Enable static anycast gateway on Vlan20

```
admin@sonic:~$ sudo config sag mac_address add 00:11:22:33:44:55
admin@sonic:~$ sudo config sag ipv4 enable
admin@sonic:~$ sudo config interface sag ip add Vlan20 192.168.20.254/24
```

Step 4: Add IP address on Ethernet0 (DHCP Server), and Loopback0

```
admin@sonic:~$ sudo config interface ip add Ethernet8 192.168.10.254/24
admin@sonic:~$ sudo config interface ip add Loopback0 3.3.3.3/32
```

Step 5: enable VxLan

```
admin@sonic:~$ sudo config vxlan add vtep 3.3.3.3
admin@sonic:~$ sudo config vxlan evpn_nvo add nvo vtep
admin@sonic:~$ sudo config vxlan map add vtep 20 2000
admin@sonic:~$ sudo config vxlan map add vtep 30 3000
admin@sonic:~$ sudo config vrf add_vrf_vni_map Vrf1 3000
```

Step 6: Enable ARP suppress

```
admin@sonic:~$ sudo config neigh_suppress enable 20
```

Step 7: establish the BGP session.

```
admin@sonic:~$ vtysh
```

```
Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
```

```
sonic# configure terminal
sonic(config)# router bgp 65103
sonic(config-router)# neighbor Ethernet48 interface remote-as external
sonic(config-router)# network 3.3.3.3/32
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# neighbor Ethernet48 activate
sonic(config-router-af)# advertise-all-vni
sonic(config-router-af)# end
sonic# configure terminal
sonic(config)# vrf Vrf1
sonic(config-vrf)# vni 3000
sonic(config-vrf)# exit
sonic(config)# router bgp 65103 vrf Vrf1
sonic(config-router)# address-family ipv4
sonic(config-router-af)# network 192.168.10.0/24
sonic(config-router-af)# network 192.168.20.0/24
sonic(config-router-af)# exit
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# advertise ipv4 unicast
```

Step 8: enable DHCP relay on Vlan20.

```
admin@sonic:~$ sudo config vlan dhcp_relay add 20 192.168.10.1
```

Spine Configuration Example

Spine connects Leaf1 and Leaf2 using BGP unnumbered interfaces.

Spine Configuration

Step 1: establish the BGP session.

```
admin@sonic:~$ vtysh
```

```
Hello, this is FRRouting (version 7.2.1-sonic).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
```

```
sonic# configure t
sonic(config)# router bgp 65102
sonic(config-router)# neighbor Ethernet0 interface remote-as external
sonic(config-router)# neighbor Ethernet8 interface remote-as external
sonic(config-router)# address-family l2vpn evpn
sonic(config-router-af)# neighbor Ethernet0 activate
sonic(config-router-af)# neighbor Ethernet8 activate
```

Limitation

DHCP relay agent IP address must be unique inside the network which means DHCP client cannot be located under MLAG-AA + SAG interfaces. Both MLAG-AA peers use the same VTEP address to participate in EVPN/VxLAN network. It may send to the wrong relay agent/VTEP which doesn't have the DHCP client information and can't forward the DHCP traffic reply from the server.

References

EVPN/VxLAN user guide: [\(202111.2\) EVPN/VXLAN](#)

DHCP relay user guide: [\(202111.2\) Configuring DHCP Relay](#)

(202111.2) In Band Management VRF

Management Virtual Routing and Forwarding (VRF) supports a separate routing table for management traffic that is forwarded by in-band L3 interfaces on a switch.

Use the `config vrf add mgmt` command to create the management VRF and use `config in-band-mgmt enable` to enable in-band management VRF.

Example

```
admin@sonic:~$ sudo config vrf add mgmt
admin@sonic:~$
admin@sonic:~$ sudo config in-band-mgmt enable
```

To bind a Layer 3 interface to management VRF, use the `config interface vrf bind` command. Use the `config interface vrf unbind` command to remove an interface from management VRF.

Example

```
admin@sonic:~$ sudo config interface vrf bind Ethernet4 mgmt
```

Use the `show mgmt-vrf` command to display details about the management VRF.

Example

```
admin@sonic:~$ show mgmt-vrf

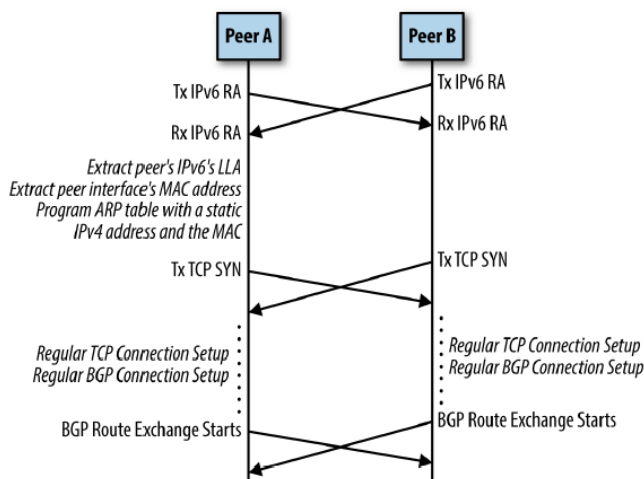
ManagementVRF : Enabled

In-band ManagementVRF : Enabled

Management VRF interfaces in Linux:
44: mgmt: <NOARP,MASTER,UP,LOWER_UP> mtu 65575 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether fe:d2:e8:7e:16:03 brd ff:ff:ff:ff:ff:ff promiscuity 0 minmtu 1280 maxmtu 65575
    vrf table 5000 addrngenmode eui64 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master mgmt state UP mode DEFAULT group default
qlen 1000
    link/ether 80:a2:35:46:08:99 brd ff:ff:ff:ff:ff:ff
10: Ethernet4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9100 qdisc pfifo_fast master mgmt state UP mode DEFAULT
group default qlen 1000
    link/ether 80:a2:35:46:08:99 brd ff:ff:ff:ff:ff:ff
45: lo-m: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue master mgmt state UNKNOWN mode DEFAULT group
default qlen 1000
    link/ether 3e:06:93:45:7d:f8 brd ff:ff:ff:ff:ff:ff
```

(202111.2) BGP Unnumbered

In the original BGP configuration, it needs to specify a peer IPv4 address to establish peer connections. For CLOS deployment in the data center, the IP interfaces grow dramatically due to the full connection between spines and LEAFs. (each peer connection needs IPv4 addresses on it). To eliminate this situation, we need to use BGP unnumbered to automatically establish peer connections by protocol itself.



This Layer-3 link is capable of running IPv6, so the system will use IPv6 link-local addresses that are automatically generated by each IPv6 interface of the local and remote peer. These addresses will be used to establish the BGP TCP session. The ASN number is ignored during the BGP session establishment.

Once IPv6 BGP session is established, the system is able to exchange IPv4 NLRIs (prefixes) over IPv6 BGP session using IPv6 link-local neighbor address as a next hop. The system associates the IPv6 link local address with that neighbor so that the neighbor will be used as a next hop for the routes.

i BGP unnumbered uses 169.254.0.1 as the unnumbered next hop. As such, while using BGP unnumbered, do not use this address in your topology in the following usages:

1. The interface's IPv4 addresses
2. The prefix or next hop of static routes
3. The ARP neighbor address

i IBGP is not supported for BGP unnumbered.

Limitation:

One neighbor can only have one unnumbered interface to be connected. In current FRR implementation, it can't identify correct next hop through multiple unnumbered interface. The route table might have a problem when more than one unnumbered interface is connected to the same peer.

(202111.2) Network Monitoring

(202111.2) Critical Resource Monitoring (CRM)

SONiC supports the monitoring of critical switch ASIC resources by polling Switch Abstraction Interface (SAI) attributes and setting threshold values.

You can query the current usage or availability of various resources on a switch. You can also set thresholds for these resources, which result in messages being sent. The default high threshold for a resource is 85%, and the default low threshold is 70%. When a high threshold is exceeded, a Syslog Warning message is sent, and when the resource falls back below the low threshold, a Clear message is sent. The CRM feature suppresses the Syslog messages after sending ten times.

The CRM feature monitors the currently used and available number of entries for the following resources:

- IPv4 and IPv6 routes
- IPv4 and IPv6 Nexthops
- IPv4 and IPv6 Neighbors
- Next-hop group members and objects
- ACLs: Table, Group, Entries, and Counters/Statistics
- FDB Entries
- IPMC entries
- SNAT entries
- DNAT entries
- SRV6 MY_SID entries and nexthop

You can use the following commands to display information on CRM resources:

- **crm show resources all** - CRM information for all resources
- **crm show resources acl** - CRM information for ACL resources
 - support : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
 - unsupported : None
- **crm show resources dnat** - CRM information for DNAT resource
 - support : (intel)x2_profilely2_profile
 - unsupported : (broadcom) , (intel)x1_profilely1_profilely3_profile
- **crm show resources fdb** - CRM information for FDB resources
 - support : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
 - unsupported : None
- **crm show resources ipmc** - CRM resources IPMC address family
 - support : (broadcom)
 - unsupported : (intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
- **crm show resources ipv4** - CRM resources IPv4 address family
 - support : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
 - unsupported : None
- **crm show resources ipv6** - CRM resources IPv6 address family
 - support : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
 - unsupported : None
- **crm show resources mpls** - CRM resources MPLS address family
 - support : (intel)y3_profile
 - unsupported : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profile
- **crm show resources nexthop** - CRM information for nexthop resources
 - support : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profilely3_profile
 - unsupported : None
- **crm show resources snat** - CRM information for SNAT resources
 - support : (broadcom),(intel)x2_profilely2_profile
 - unsupported : (intel)x1_profilely1_profilely3_profile
- **crm show resources srv6-my-sid-entry** - CRM information for SRV6 MY_SID entry
 - support : (intel)y3_profile
 - unsupported : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profile
- **crm show resources srv6-nexthop** - CRM information for SRV6 Nexthop
 - support : (intel)y3_profile
 - unsupported : (broadcom),(intel)x1_profilelx2_profilely1_profilely2_profile

Example

```
admin@sonic:~$ crm show resources all
```

Resource Name	Used Count	Available Count
-----	-----	-----
ipv4_route	33	32719
ipv6_route	35	10206
ipv4_nexthop	0	65534
ipv6_nexthop	0	65534
ipv4_neighbor	33	16283
ipv6_neighbor	33	8141
nexthop_group_member	0	16384
nexthop_group	0	256
fdb_entry	0	32767
ipmc_entry	0	16383
snat_entry	0	1023

Stage	Bind Point	Resource Name	Used Count	Available Count
-----	-----	-----	-----	-----
INGRESS	PORT	acl_group	32	224
INGRESS	PORT	acl_table	3	2
INGRESS	LAG	acl_group	0	224
INGRESS	LAG	acl_table	0	2
INGRESS	VLAN	acl_group	0	224
INGRESS	VLAN	acl_table	0	5
INGRESS	RIF	acl_group	0	224
INGRESS	RIF	acl_table	0	5
INGRESS	SWITCH	acl_group	0	224
INGRESS	SWITCH	acl_table	0	5
EGRESS	PORT	acl_group	0	224
EGRESS	PORT	acl_table	0	2
EGRESS	LAG	acl_group	0	224
EGRESS	LAG	acl_table	0	2
EGRESS	VLAN	acl_group	0	224
EGRESS	VLAN	acl_table	0	2
EGRESS	RIF	acl_group	0	224
EGRESS	RIF	acl_table	0	2
EGRESS	SWITCH	acl_group	0	224
EGRESS	SWITCH	acl_table	0	2

Table ID	Resource Name	Used Count	Available Count
-----	-----	-----	-----
0x7000000000a20	acl_entry	2	6142
0x7000000000a20	acl_counter	2	6142

To set the switch ASIC SAI polling interval, use the **crm config polling interval** command.

To set resource thresholds, use one of the following commands for the specific resource:

- **crm config thresholds acl** - CRM configuration for ACL resources
- **crm config thresholds dn timer** - CRM configuration for Destination NAT resources
- **crm config thresholds fdb** - CRM configuration for FDB resources
- **crm config thresholds ipmc** - CRM configuration for IPMC resource
- **crm config thresholds ipv4** - CRM resource IPv4 address-family
- **crm config thresholds ipv6** - CRM resource IPv6 address-family
- **crm config thresholds mpls** - CRM resource MPLS address-family
- **crm config thresholds nexthop** - CRM configuration for nexthop resources
- **crm config thresholds snat** - CRM configuration for Source NAT resource

- `crm config thresholds srv6-my-sid-entry` - CRM configuration for SRV6 MY_SID resource
- `crm config thresholds srv6-nexthop` - CRM configuration for SRV6 Nexthop resource

Example

```
admin@sonic:~$ crm config polling interval 400

admin@sonic:~$ crm config thresholds acl group high 90

admin@sonic:~$ crm config thresholds acl group low 10
```

For more information, see the [\(202111\) Critical Resource Monitoring](#) section in the CLI reference guide.

(202111.2) Flow Sampling (sFlow)

Flow sampling (sFlow) can be used with a remote sFlow Collector to provide an accurate, detailed and real-time overview of the types and levels of traffic present on the network. The sFlow Agent samples 1 out of n packets from all data traversing a switch, re-encapsulates the samples as sFlow datagrams and transmits them to the sFlow Collector. This sampling occurs at the internal hardware level where all traffic is seen, whereas traditional probes only have a partial view of traffic as it is sampled at the monitored interface. Moreover, the processor and memory load imposed by the sFlow agent is minimal since local analysis does not take place.

As the Collector receives streams from the various sFlow agents (other switches or routers) throughout the network, a timely, network-wide picture of utilization and traffic flows is created. Analysis of the sFlow stream(s) can reveal trends and information that can be leveraged to detect, diagnose, and fix network problems.



sFlow is supported on all Broadcom switches and Intel Tofino with **XI** profile and Tofino2 with **YI** profile switches.

To configure sFlow in SONiC, first define a collector using the **config sflow collector add** command (up to two collectors can be defined) to specify the IPv4 or IPv6 address. By default, sFlow is disabled globally on a switch, but enabled on all interfaces. To enable sFlow globally, use the **config sflow** command to start sampling will start on all interfaces.

To display the global sFlow configuration, use the **show sflow** command.

Example

```
admin@sonic:~# sudo config sflow collector add collector_A 10.11.46.2

admin@sonic:~# sudo config sflow enable

admin@sonic:~# show sflow
sFlow Global Information:
sFlow Admin State: up
sFlow Polling Interval: default
sFlow AgentID: lo
2 Collectors configured:
Name: collector_A IP addr: 10.11.46.2 UDP port: 6343
Name: collector_lo IP addr: 127.0.0.1 UDP port: 6343
```

For more information on sFlow and how to configure the polling interval and sampling rates, see the [\(202111\) sFlow](#) section in the CLI reference guide.

(202111.2) Configuring Drop Counters

SONiC includes a feature that monitors packet drops to help troubleshoot and debug issues in a network. The feature provides counters that are configurable for specific drop reasons and therefore can be easily applied to various network priorities.

Before configuring the packet drop counters, use the **show dropcounters capabilities** command to display the counter capabilities (drop counters and drop reasons) that are available on a switch. Then use the **config dropcounters install** command to initialize a new drop counter. You must specify a name for the counter, indicate the available counter type, and include an initial list of drop reasons.

Example

```
admin@sonic:~$ show dropcounters capabilities
```

Counter Type	Total
PORT_INGRESS_DROPS	3
SWITCH_EGRESS_DROPS	2

```
PORT_INGRESS_DROPS:
  L2_ANY
  SMAC_MULTICAST
  SMAC_EQUALS_DMACE
  INGRESS_VLAN_FILTER
  EXCEEDS_L2_MTU
  SIP_CLASS_E
  SIP_LINK_LOCAL
  DIP_LINK_LOCAL
  UNRESOLVED_NEXT_HOP
  DECAP_ERROR
```

```
SWITCH_EGRESS_DROPS:
  L2_ANY
  L3_ANY
  A_CUSTOM_REASON
```

```
admin@sonic:~$ sudo config dropcounters install DEBUG_2 PORT_INGRESS_DROPS \[EXCEEDS_L2_MTU,DECAP_ERROR\] -d
"More port ingress drops" -g BAD -a BAD_DROPS
```

To display the current configured drop counters, use the **show dropcounters configuration** command. To display the collected statistics for the configured drop counters, use the **show dropcounters count** command.

Example

```
admin@sonic:~$ show dropcounters configuration
```

Counter	Alias	Group	Type	Reasons	Description
DEBUG_0	RX_LEGIT	LEGIT	PORT_INGRESS_DROPS	SMAC_EQUALS_DMACE INGRESS_VLAN_FILTER	Legitimate port-level RX pipeline drops
DEBUG_1	TX_LEGIT	None	SWITCH_EGRESS_DROPS	EGRESS_VLAN_FILTER	Legitimate switch-level TX pipeline drops

```
admin@sonic:~$ show dropcounters configuration -g LEGIT
```

Counter	Alias	Group	Type	Reasons	Description
DEBUG_0	RX_LEGIT	LEGIT	PORT_INGRESS_DROPS	SMAC_EQUALS_DMACE INGRESS_VLAN_FILTER	Legitimate port-level RX pipeline drops

```
admin@sonic:~$ show dropcounters counts -t SWITCH_EGRESS_DROPS
```

DEVICE	TX_LEGIT
sonic	1000

For more information, see the [\(202111\) Drop Counters](#) section in the CLI reference guide.

(202111.2) Setting the Watermark Telemetry Interval

Network telemetry monitors network health and helps identify problems and latency issues so they can be resolved quickly. SONiC supports a telemetry system whereby devices in the network periodically stream information to data collectors in the management system.

The interval for the telemetry can be set using the **config watermark telemetry interval** command. This parameter leverages the Linux timer and is set to a default 120 seconds. In the following example, the telemetry interval is set to 999 seconds.

Example

```
admin@sonic:~$ sudo config watermark telemetry interval 999
```

For more information, see the [\(202111\) Watermark](#) section in the CLI reference guide.

(202111.2) QoS Configuration

(202111.2) Class of Service (CoS)

QoS Classification is used to classify the packets to assign a Traffic Class (TC) value based on service requirements. TC is the internal priority in switch (8 possible value) using for packet buffering and scheduling. TC-to-priority-group (PG) and TC-to-Queue maps depend on the TC maps to the ingress buffer (priority-group) and egress queue, respectively. TC is used as a priority field for packet re-marking on egress port. Determines the TC based on the priority of the incoming packet and trust mode of a port. There have two trust mode value: **dot1p**, and **dscp**.

- **dot1p**:
 - If the packet is VLAN tagged, the Dot1p is used to map TC.
 - Else, if the packet is untagged, the port default priority (0, and can not be configured) is used as Dot1p.
- **dscp**:
 - If the packet is IP packet, the DSCP is used to map TC.
 - Else, if the packet is VLAN tagged, the Dot1p is used.
 - Else, if the packet is untagged, the port default priority is used.

To determine the trust mode of a port based on the following rules:

- The default trust mode of a port is **dot1p**.
- If a port be configured a DSCP-to-TC mapping, the trust mode of the port is **dscp**.
- Default mapping tables are used to determine the TC when Dot1p-to-TC not be configured on a port. Default mapping tables vary by switch platform.

Table 1: Default Dot1p-to-TC mapping on Broadcom switches.

Dot1p	0	1	2	3	4	5	6	7
TC	0	1	2	3	4	5	6	7

Table 2: Default Dot1p-to-TC mapping on Intel switches.

Dot1p	0	1	2	3	4	5	6	7
TC	0	0	0	0	0	0	0	0

Table 3: Default DSCP-to-TC mapping.

DSCP	0 ~ 63
TC	0

Dot1p-to-TC Mapping

- Assign the TC based on PCP in the VLAN tag.
- The mapping can be configured per ingress physical port.

DSCP-to-TC Mapping

- Assign the TC based on DSCP field in the IP header.
- The mapping can be configured per ingress physical port.

TC-to-queue Mapping

- Assign the queue ID based on TC.
- The mapping can be configured per ingress physical port.
- The default mapping tables vary by switch platform.
- Support only one profile on Intel switches.

Table 6: Default TC-to-Queue mapping on Broadcom switches.

TC	0	1	2	3	4	5	6	7
Queue	0	1	2	3	4	5	6	7

Table 7: Default TC-to-Queue mapping on Intel switches.

TC	0	1	2	3	4	5	6	7
Queue	0	0	0	0	0	0	0	0

To configure a Dot1p-to-TC profile, using **config qos dot1p-tc** command. To configure Dot1p-to-TC on an interface, using **config interface qos dot1p-tc** command.

To configure a DSCP-to-TC profile, using **config qos dscp-tc** command. To configure DSCP-to-TC on an interface, using **config interface qos dscp-tc** command.

To configure a TC-to-queue profile, using **config qos tc-queue** command. To configure TC-to-queue on an interface, using **config interface qos tc-queue** command.

- [Apply a Dot1p-to-TC Table to a Port](#)
- [Apply a DSCP-to-TC Table to a Port](#)
- [Apply a TC-to-Queue Table to a Port](#)

Apply a Dot1p-to-TC Table to a Port

1. Create a Dot1p-to-TC profile, using **config qos dot1p-tc** command. The following example is to create a Dot1p-to-TC profile **dot1p-tc-prof** and adds a mapping dot1p(0) to TC(3) when creating.

```
admin@sonic:~$ sudo config qos dot1p-tc add dot1p-tc-prof --dot1p 0 --tc 3
```

2. Update the Dot1p-to-TC profile for adding/removing mapping, using **config qos dot1p-tc update** command. The following example is to add a new mapping dot1p(1) to TC(4).

```
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 1 --tc 4
```

3. Apply the Dot1p-to-TC profile to a port, using **config interface qos dot1p-tc bind** command. The following example is to apply the Dot1p-to-TC profile **dot1p-tc-prof** to **Ethernet0**.

```
admin@sonic:~$ sudo config interface qos dot1p-tc bind Ethernet0 dot1p-tc-prof
```

Apply a DSCP-to-TC Table to a Port

1. Create a DSCP-to-TC profile, using **config qos dscp-tc** command. The following example is to create a DSCP-to-TC profile **dscp-tc-prof** and adds a mapping DSCP(8) to TC(3) when creating.

```
admin@sonic:~$ sudo config qos dscp-tc add dscp-tc-prof --dscp 8 --tc 3
```

2. Update the DSCP-to-TC profile for adding/removing mapping, using **config qos dscp-tc update** command. The following example is to add a new mapping DSCP(32) to TC(4).

```
admin@sonic:~$ sudo config qos dscp-tc update dscp-tc-prof --dscp 32 --tc 4
```

3. Apply the DSCP-to-TC profile to a port, using **config interface qos dscp-tc bind** command. The following example is to apply the DSCP-to-TC profile **dscp-tc-prof** to **Ethernet0**.

```
admin@sonic:~$ sudo config interface qos dscp-tc bind Ethernet0 dscp-tc-prof
```

Apply a TC-to-Queue Table to a Port

1. Create a TC-to-queue profile, using **config qos tc-queue** command. The following example is to create a TC-queue profile **tc-queue-prof** and adds a mapping TC(3) to queue(5) when creating.

```
admin@sonic:~$ sudo config qos tc-queue add tc-queue-prof --tc 3 --queue 5
```

2. Update the TC-queue profile for adding/removing mapping, using **config qos tc-queue update** command. The following example is to add a new mapping TC(4) to queue(6).

```
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 4 --queue 6
```

3. Apply the TC-queue profile to a port, using **config interface qos tc-queue bind** command. The following example is to apply the TC-queue profile **tc-queue-prof** to **Ethernet0**.

```
admin@sonic:~$ sudo config interface qos tc-queue bind Ethernet0 tc-queue-prof
```

For more information, see the (202111.2) QoS section in the CLI reference guide.

(202111.2) QoS Marking

Once the packets have been classified by QoS Classification, QoS Marking allows to set or change the priority value in the packets before sending them. The new priority value is brought to the next switches for fine-tuning or uniform the class of the traffic in the network.

QoS Marking enables to rewrite the Dot1p or DSCP of incoming packets based on Traffic Class (TC) value.

- Enable Dot1p rewrite by configuring a TC-to-Dot1p mapping on an egress port. On Broadcom switches, the Dot1p rewrite is always enabled.
- Enable DSCP rewrite by configuring a TC-to-DSCP mapping on an egress port. The DSCP rewrite is disabled by default on all ports.

Table 1: Default TC-to-Dot1p mapping.

TC	0	1	2	3	4	5	6	7
Dot1p	0	1	2	3	4	5	6	7

Table 2: Default TC-to-DSCP mapping.

TC	0	1	2	3	4	5	6	7
DSCP	0	0	0	0	0	0	0	0

TC-to-Dot1p Mapping

- Rewrite the PCP in the VLAN tag based on the TC-to-Dot1p mapping table.
- Any TC value that is not specified in the configuration is assigned to a default Dot1p value for rewriting. Refer Table 1 for the default mapping.
- The mapping can be configured per egress physical port.

TC-to-DSCP Mapping

- Rewrite the DSCP field in the IP header based on the TC-to-DSCP mapping table.
- Any TC value that is not specified in the configuration is assigned to a default DSCP value for rewriting. Refer Table 2 for the default mapping.
- The mapping can be configured per egress physical port.



Only the DCS201(AS5835-54X), EPS202(AS4630-54PE), DCS203(AS7326-56X), DCS204(AS7726-32X), and DCS510(AS9716-32D) are supported to change the Dot1p and DSCP remarking mapping.

To configure a Dot1p remarking profile, using **config qos remark dot1p** command. To configure Dot1p remarking on an interface, using **config interface qos remark dot1p** command.

To configure a DSCP remarking profile, using **config qos remark dscp** command. To configure DSCP remarking on an interface, using **config interface qos remark dscp** command.

Configuring marking PCP

1. To create a Dot1p-TC profile, using **config qos dot1p-tc** command. For example, create a Dot1p-TC profile "dot1p_tc" to map the packet Dot1p priority of 0,2 to switch priority 0,2, respectively.

Example

```
sudo config qos dot1p-tc add dot1p_tc --dot1p 0 --tc 0
sudo config qos dot1p-tc update dot1p_tc --dot1p 2 --tc 2
```

2. To configure a Dot1p-TC profile on the ingress port, using **config interface qos dot1p-tc** command. For example, configure the Dot1p-TC profile "dot1p_tc" on Ethernet0 as the following command.

Example

```
sudo config interface qos dot1p-tc bind Ethernet0 dot1p_tc
```

3. To create a Dot1p remarking profile, using **config qos remark dot1p** command. For example, create a Dot1p remarking profile "remark_pcp" to map the switch priority 0,2 to PCP value 1,3 for rewriting.

Example

```
sudo config qos remark dot1p add remark_pcp --tc 0 --dot1p 1
sudo config qos remark dot1p update remark_pcp --tc 2 --dot1p 3
```

4. To enable Dot1p remarking on the egress port, using **config interface qos remark dot1p bind** command. For example, enable Dot1p remarking with the mapping "remark_pcp" on Ethernet2 as the following command.

Example

```
sudo config interface qos remark dot1p bind Ethernet2 remark_pcp
```

5. Verify the configuration, using **show interface qos** command.

Example

```
admin@sonic:~$ show interface qos
Ethernet0:
  Dot1p to TC: dot1p_tc

Ethernet2:
  Dot1p remark: remark_pcp
```

Configuring marking DSCP

1. To create a DSCP-TC profile, using **config qos dscp-tc** command. For example, create a DSCP-TC profile "dscp_tc" to map the DSCP 0,16 to switch priority 0,2, respectively.

Example

```
sudo config qos dscp-tc add dscp_tc --dscp 0 --tc 0
sudo config qos dscp-tc update dscp_tc --dscp 16 --tc 2
```

2. To configure a DSCP-TC profile on the ingress port, using **config interface qos dscp-tc** command. For example, configure the DSCP-TC profile "dscp_tc" on Ethernet0 as the following command.

Example

```
sudo config interface qos dscp-tc bind Ethernet0 dscp_tc
```

3. To create a DSCP remarking profile, using **config qos remark dscp** command. For example, create a DSCP remarking profile "remark_dscp" to map the switch priority 0,2 to DSCP value 1,17 for rewriting.

Example

```
sudo config qos remark dscp add remark_dscp --tc 0 --dscp 1
sudo config qos remark dscp update remark_dscp --tc 2 --dscp 17
```

4. To enable DSCP remarking on the egress port, using **config interface qos remark dscp bind** command. For example, enable DSCP remarking with the mapping "remark_dscp" on Ethernet2 as the following command.

Example

```
sudo config interface qos remark dscp bind Ethernet2 remark_dscp
```

5. Verify the configuration, using **show interface qos** command.

Example

```
admin@sonic:~$ show interface qos
Ethernet0:
  DSCP to TC: dscp_tc

Ethernet2:
  IP DSCP remark: remark_dscp
```

For more information, see the (202111.2) QoS section in the CLI reference guide.

(202111.2) Explicit Congestion Notification (ECN)

Explicit congestion notification (ECN) enables traffic congestion control between two ECN-enabled devices. When the congestion is detected by the switch, the ECN field is set in the IP header of a packet before forwarding it. If the receiver device received the packets with ECN marked, it sends the congestion notification packets (CNP) to the sender device to reduce the transmit packet rate. The CNP packets continue to send to the sender until the congestion is released.

ECN uses the last two bits, ECT and CE, of the DS field of the IP header as the following:

ECT	EC	Combinations Indicate
0	0	Non ECN-Capable Transport, Non-ECT
1	0	ECN Capable Transport, ECT(0)
0	1	ECN Capable Transport, ECT(1)
1	1	Congestion Encountered, CE

ECN uses the WRED thresholds being applied to an egress queue. When ECN enabled, process the packets based on following rules:

- When the average queue length is below the **minimum threshold**, no packet is marked.
- When the average queue length is between **minimum threshold** and **maximum threshold**, one of the following scenarios can occur:
 - If received the IP packet with ECT(0) or ECT(1) codepoints, changes the ECT and EC bits to 1 based drop probability and forwards the packet.
 - If received the IP packet with CE codepoint '11', the packet is transmitted. No further marking is required.
 - If received the IP packet with not-ECT codepoint '00', the packet is transmitted.
- When the average queue length exceeds the **maximum threshold**, all the incoming IP packets are marked.

RDMA over Converged Ethernet is a network protocol that allows Remote Direct Memory Access (RDMA) over an Ethernet network. RoCEv2 (RoCE version 2) enhances RoCEv1 with a UDP/IP and using the well-known UDP port 4791. It defines congestion control mechanism based on ECN marking. The RoCEv2 Congestion Management defines the three points:

- Reaction Point (RP): reduces the transmission rate, according to the received CNPs. RP is the sender device.
- Congestion Point (CP): detects congestion and marks EC bit. CP is a switch.
- Notification Point (NP): reacts to the ECN marked packets by sending congestion notification packets (CNPs). NP is the receiver device.

The following processes show how ECN works using RoCEv2 as an example.

RP(sender) ----- CP(switch) ----- NP(receiver)

1. **RoCEv2 with ECT(0) or ECT(1)**
----->

The RP(sender) send the RoCEv2 packet with ECT(0) or ECT(1) to NP(receiver).

2. **set ECN field to '11'**
----->

If the congestion is detected by CP(switch), set the ECN field to '11' of the packet.

3. **notify with RoCEv2 CNP packet**
<-----

When the NP(receiver) receives the packet with ECN field as '11', send a RoCEv2 CNP packet to the RP(sender).

4. **Change the transmission rate**

When the RP(sender) receives the CNP packet, slow down the transmission rate.

 ECN does not support on DCS240 (using Broadcom Trident 4).

To configure the WRED profile, referring this document [\(202111.2\) Weighted Random Early Detection \(WRED\)](#) for more information.

The ECN can be configured through CLI command. You need to first configure a WRED profile, using **config wred add** command, and then bind the profile to the port interface's queues, using **config interface wred bind queue** command.

The following example is to configure a WRED profile **wred-prof**. This example uses green max threshold of 800k bytes, green min threshold of 100k bytes, and green drop probability of 100%.

Example

```
admin@sonic:~$ sudo config wred add wred-prof --mode ecn --gmin 100000 --gmax 800000 --gdrop 100
```

Once a profile has been created, it can be applied on the port interface's queues, as follows:

Example

```
admin@sonic:~$ sudo config interface wred bind queue Ethernet0 0 wred-prof
```

Verify the configuration using **show wred** and **show interface wred** command.

Example

```
admin@sonic:~$ show wred
Profile: wred-prof
Color    Mode      Min Threshold  Max Threshold  Drop Probability
-----
Green    ECN        100000        800000        100
Yellow   ECN
Red      ECN

admin@sonic:~$ show interfaces wred
Ethernet0
Queue: 0
ECN/WRED: wred-prof
Color    Mode      Min Threshold  Max Threshold  Drop Probability
-----
Green    ECN        100000        800000        100
Yellow   ECN
Red      ECN

admin@sonic:~$
```

The following example is to unbind the profile from the port interface's queues, and delete the profile:

Example

```
admin@sonic:~$ sudo config interface wred unbind queue Ethernet0 0 wred-prof
admin@sonic:~$ sudo config wred del wred-prof
```

For more information, see the (202111.2) WRED & ECN section in the CLI reference guide.

(202111.2) Priority Flow Control (PFC)

Priority Flow Control (PFC) is defined in the IEEE 802.1Qbb provides the hop-by-hop layer 2 congestion control. PFC extends the IEEE 802.3x Ethernet PAUSE, allows to pause the traffic based on its priority (802.1p) rather than pause all the traffic on a link. When the congestion happened on the ingress buffer of downstream switch, it sends the PFC PAUSE frame to upstream switch. The PFC PAUSE frame contains a 2-octet priority specifying which traffic of the priority class should be paused and a 2-octet time value specifying the pause time. When the upstream switch receives a PFC PAUSE frame, it stop transmitting based on the priorities and the period in the PFC frame. The upstream switch resumes transmitting after time expires. Or receives a new PFC PAUSE frame with pause time is zero. All the incoming packets are stored in central buffer called Shared Buffer.

Shared Buffer can be divided into three parts, including reserved buffer, shared buffer and headroom buffer. Each buffer has a maximum limit value and uses counters to record the usage. Each packet is counted in both ingress and egress based on its priority. Packet drop is happening if the usage hit to the maximum limit. The PFC PAUSE frame is generated if the usage of ingress buffer reaches a predetermined threshold call XOFF limit (sum of size of reserved buffer and shared buffer). The headroom buffer is used to store the packets are in flight during the time takes for the PAUSE to take effect. It includes the PFC PAUSE frame is sent to the upstream switch and stop the transmission on the egress queue. The PFC PAUSE frame is generated with pause time as zero if the usage of ingress buffer falls below to a predetermined XON limit.

This section describes the parameters for PFC feature.

- PFC is supported to enable per-port per-priority. PFC is disabled by default on all ports.
- PFC is supported to configure the XOFF limit of a priority-group. An 802.3x PAUSE frame is generated when the buffer limit at the ingress port reaches to this threshold.
- PFC is supported to configure the XON limit of a priority-group. The XOFF state will be released when the buffer limit at the ingress port falls below this threshold.
- PFC is supported to configure the traffic-class-to-priority-group (TC2PG) mapping on each port that allow to map the traffic class to a PG.
- PFC is supported to configure the Priority-to-queue (PRIO2Q) mapping on each port that allow to map the priority(s) in PFC frame to a queue. Determine which queue will be paused if received a 802.3 PAUSE frame include the priority.

Traffic-class-to-priority-group (TC2PG) Mapping

The TC2PG map has two purposes. One is used to determine the priority-group (PG) for buffering. Another is used to determine the priority in the PFC PAUSE frame. The behavior of the mapping varies by switch platform.

Traffic-class-to-priority-group (TC2PG) Mapping - Broadcom switches

- Maps to the priority-group (PG) based on the rules below.
 - If the packet is VLAN tagged, the Dot1p in the packet is used as TC to map the PG.
 - Else, if the packet is untagged. The TC, which is determined by QoS classification, is used to map the PG.
- The PG range is 0 ~ 7.
- The TCs, which map to the same PG, are used as priorities in the PFC PAUSE frame if the PG is under pressure.
- Table 1 shows below is the default TC-to-PG mapping on Broadcom switches.

Traffic-class-to-priority-group (TC2PG) Mapping - Intel switches

- The TC, which is determined by QoS classification, is used to map the PG.
- The PG range is 0, 1 ~ 5. The PG 0 is a default PG, not able to change the buffer thresholds and be mapped by TC2PG.
- The TCs, which map to the same PG, are used as priorities in the PFC PAUSE frame if the PG is under pressure.
- Table 2 shows below is the default TC-to-PG mapping on Intel switches.

Table 1: Default TC-to-PG mapping on Broadcom switches.

TC	0 ~ 7
PG	7

Table 2: Default TC-to-PG mapping on Intel switches.

TC	0 ~ 7
PG	0

Priority-to-queue (PRIO2Q) Mapping

Maps the 802.1p priority(s) in PFC frame to the queue specifying which queue stops the transmission if received the PFC frame with the given priority(s). Only Broadcom switches is supported to change the mapping. Table 3 shows the default mapping.

Table 3: Default Priority-to-Queue mapping.

PFC priority	0	1	2	3	4	5	6	7
queue	0	1	2	3	4	5	6	7



PFC does not support on Broadcom Tomahawk3, Trident4 switches, including the following models:

- DCS510(AS9716) series
- DCS240(AS9726) series

To configure the Shared Buffer refer to [\(202111.2\) Shared Buffer Configuration](#) for more information.

To configure the TC-to-PG mapping, using **config qos tc-pg** and **config interface qos tc-pg** commands.

To configure the PFC-priority-to-Queue mapping, using **config qos pfc-queue** and **config interface qos pfc-queue** commands.

To enable PFC on an interface, use the **config interface pfc priority** command.

Configuration Example

1. Set buffer pools and profiles for lossless traffic.

To configure the buffer setting as lossless, using **config qos reload** command.



NOTE

DCS801(Wedge100BF-32QS) switch only, issue the following commands once before run the **config qos reload** command.

```
sudo cp /usr/share/sonic/device/x86_64-accton_wedge100bf_32x-r0/montara/qos.json.j2 /usr/share/sonic/device/x86_64-accton_wedge100bf_32qs-r0/montara/qos.json.j2
sudo cp /usr/share/sonic/device/x86_64-accton_wedge100bf_32x-r0/montara/buffers_defaults_t0.j2 /usr/share/sonic/device/x86_64-accton_wedge100bf_32qs-r0/montara/buffers_defaults_t0.j2
sudo cp /usr/share/sonic/device/x86_64-accton_wedge100bf_32x-r0/montara/buffers_defaults_t1.j2 /usr/share/sonic/device/x86_64-accton_wedge100bf_32qs-r0/montara/buffers_defaults_t1.j2
```

Example

```
admin@sonic:~$ sudo config qos reload
```

The below buffer pool and profiles will be created after the above command.

- ingress_lossy_pool - ingress buffer pool for lossy traffic
- ingress_lossless_pool - ingress buffer pool for lossless traffic
- egress_lossy_pool - egress buffer pool for lossy traffic
- egress_lossless_pool - egress buffer pool for lossless traffic
- ingress_lossy_profile - ingress buffer profile that is used to apply lossy priority-group. By default, all priority-group on all ports are as lossy.
- ingress_lossless_profile - ingress buffer profile that is used to apply lossless priority-group.
- egress_lossy_profile - egress buffer profile that is used to apply lossy queue. By default, all queues on all ports are as lossy.
- egress_lossless_profile - egress buffer profile that is used to apply lossless queue.

Then. Save the configuration and restart the switch for the change to take effect.

Example

```
admin@sonic:~$ sudo config save -y
admin@sonic:~$ sudo reboot
```

2. Configure the priority-group to use lossless buffer settings.

To configure the priority-group for applying the PFC thresholds (**ingress_lossless_profile** is predefined profile), use the **config interface buffer** command. Use priority group 3 as example:

Example

```
admin@sonic:~$ sudo config interface buffer bind priority-group Ethernet60 3 ingress_lossless_profile
```

3. Configure the queue to use lossless buffer settings.

To configure the queue for lossless traffic, use the **config interface buffer** command. **egress_lossless_profile** is predefined buffer profile. Use queue 3 as example:

Example

```
admin@sonic:~$ sudo config interface buffer bind queue Ethernet60 3 egress_lossless_profile
```

4. Configure the TC-to-PG and TC-to-Queue QoS mapping to specify the PG and queue which configured on step 2 and 3.

To configure the TC-to-PG QoS mapping, it need to create a TC-to-PG profile, using **config qos tc-pg add** command. Then apply the profile to an interface, using **config interface qos tc-pg bind** command.

Example

```
admin@sonic:~$ sudo config qos tc-pg add tc-pg-prof --tc 3 --pg 3
admin@sonic:~$ sudo config interface qos tc-pg bind Ethernet60 tc-pg-prof
```

To configure the TC-to-Queue QoS mapping, it need to create a TC-to-Queue profile, using **config qos tc-queue add** command. Then apply the profile to an interface, using **config interface qos tc-queue bind** command.

Example

```
admin@sonic:~$ sudo config qos tc-queue add tc-queue-prof --tc 3 --queue 3
admin@sonic:~$ sudo config interface qos tc-queue bind Ethernet60 tc-queue-prof
```

i Recommend using the 1:1 mapping for TC-to-PG and TC-to-Queue mapping for PFC. Below example shows if you want to enable PFC priority 0 ~ 7 the mapping will be:

- TC 0 maps to PG 0 and queue 0
- TC 1 maps to PG 1 and queue 1
- TC 2 maps to PG 2 and queue 2
- TC 3 maps to PG 3 and queue 3
- TC 4 maps to PG 4 and queue 4
- TC 5 maps to PG 5 and queue 5
- TC 6 maps to PG 6 and queue 6
- TC 7 maps to PG 7 and queue 7

i On Intel switches, it only has 5 PGs. The range is 1 ~ 5. PG 0 is the default PG, is not able to be mapped in TC-to-PG. The below mapping shows for TC-to-Pg and TC-to-Queue mapping if you want to enable PFC priority 1 ~ 5:

- TC 1 maps to PG 1 and queue 1
- TC 2 maps to PG 2 and queue 2
- TC 3 maps to PG 3 and queue 3
- TC 4 maps to PG 4 and queue 4
- TC 5 maps to PG 5 and queue 5

5. Enable the PFC on an interface.

To enable PFC on an interface, use the **config interface pfc priority** command and set the priority to "on". When successfully configured, the current lossless priorities on the interface are displayed.

Example

```
admin@sonic:~$ sudo config interface pfc priority Ethernet60 3 on
Interface      Lossless priorities
-----
Ethernet60    3
```

To configure asymmetric PFC on an interface, use the **config interface pfc asymmetric** command and set it to "on". When set to asymmetric PFC, an interface accepts pause frames for all priorities, but only sends pause frames for lossless priorities.

To check the status of asymmetric PFC, use the **show interfaces status** command.

Example

```
admin@sonic:~$ sudo config interface pfc asymmetric Ethernet60 on
```

6. Configure the mapping between the packet priority to Traffic Class (TC).

To configure the packet priority to switch priority mapping. Use Dot1p-to-TC as example, use **config qos dot1p-tc add** command to create the profile. Then use **config interface qos dot1p-tc bind** command to apply the profile to an interface.

Example

```
admin@sonic:~$ sudo config qos dot1p-tc add 1p-tc-prof --dot1p 3 --tc 3
admin@sonic:~$ sudo config interface qos dot1p-tc bind Ethernet60 1p-tc-prof
```

 Recommend using the 1:1 mapping for Dot1p-to-TC mapping for PFC. For example:

- 802.1p 0 maps to TC 0
- 802.1p 1 maps to TC 1
- 802.1p 2 maps to TC 2
- 802.1p 3 maps to TC 3
- 802.1p 4 maps to TC 4
- 802.1p 5 maps to TC 5
- 802.1p 6 maps to TC 6
- 802.1p 7 maps to TC 7

Summary the above example, the QoS mapping shows as below table:

Dot1P	TC	PG	Queue
3	3	3	3

- PFC frame with 3 priority will be send, if the buffer usage of PG 3 reach to the xoff-threshold.
- Queue 3 will pause if received PFC frame with 3 priority.

For more information, see the (202111.2) Priority Flow Control (PFC), (202111.2) QoS section in the CLI reference guide.

(202111.2) Starting the PFC Watchdog

Priority Flow Control (PFC) watchdog provides to detect and mitigate the PFC deadlock (storm) problem. PFC deadlock is the one of well-known problems if used the PFC mechanism. PFC deadlock is caused the PFC PAUSE frames are propagated running into a loop. When the deadlock is happening, the buffer resource in each switch is not able to release to lift the XOFF (transmission-off) state. In this situation, no any packet can be transmitted in the whole network. To remove the deadlock, it can temporarily disables PFC to ignore the PFC PAUSE frames. Or remove all packets have been stored in the queue.

PFC watchdog has three function blocks, i.e. detection, mitigation and restoration.

- PFC deadlock detection - to detect whether the PFC deadlock happened by monitoring the queue PAUSE state. If the queue is in a paused state exceed the detection time, the deadlock is happening.
- PFC deadlock mitigation - once the deadlock is detected, take the one of the actions per queue (priority) level:
 - forward - temporarily disables PFC to ignore the PFC PAUSE frames.
 - drop - drop the all packets (in / out direction) of the queue.
 - log - record the deadlock event to syslog.
- PFC deadlock restoration - monitor the PFC PAUSE frame on the link while taking the mitigation action. If the period of no receiving any PFC PAUSE frame over a configurable time. Re-start the PFC mechanism on the queue.

The PFCWD can enable/disable per port. It is only available for lossless queue. Default is disable for all port.



PFCWD is not supported on Broadcom Trident3 X3, Trident4, and Tomahawk3 switches, including the following models:

- DCS510(AS9716) series
- DCS240(AS9726) series
- EPS202(AS4630) series

To configure the PFC refer to [\(202111.2\) Priority Flow Control \(PFC\)](#) for more information.

To enable the PFC watchdog feature on a specified port or all ports, use the **config pfcwd start** command. To disable the PFC watchdog feature on all ports, use the **config pfcwd stop** command.

Configuration Example

Step1, Prerequisite, configure the lossless buffer setting

Configure the lossless buffer setting.

Example

```
admin@sonic:~$ sudo config qos reload
```

Save the configuration and restart the switch for the change to take effect.

Example

```
admin@sonic:~$ sudo config save -y
admin@sonic:~$ sudo reboot
```

Step2, Configure the QoS mapping

Broadcom switches only

The below table is the QoS mapping is used for enabling PFC on Broadcom switches.

Dot1p	TC	PG	Queue
0	0	0	0

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7

The following commands are used to configure the above QoS mappings. Configure the Dot1p-to-TC mapping as:

Example, Broadcom switches only

```
admin@sonic:~$ sudo config qos dot1p-tc add dot1p-tc-prof --dot1p 0 --tc 0
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 1 --tc 1
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 2 --tc 2
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 3 --tc 3
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 4 --tc 4
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 5 --tc 5
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 6 --tc 6
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 7 --tc 7
admin@sonic:~$ sudo config interface qos dot1p-tc bind Ethernet64 dot1p-tc-prof
```

Configure the Tc-to-PG mapping as:

Example, Broadcom switches only

```
admin@sonic:~$ sudo config qos tc-pg add tc-pg-prof --tc 0 --pg 0
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 1 --pg 1
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 2 --pg 2
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 3 --pg 3
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 4 --pg 4
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 5 --pg 5
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 6 --pg 6
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 7 --pg 7
admin@sonic:~$ sudo config interface qos tc-pg bind Ethernet64 tc-pg-prof
```

Configure the TC-to-Queue mapping as:

Example, Broadcom switches only

```
admin@sonic:~$ sudo config qos tc-queue add tc-queue-prof --tc 0 --queue 0
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 1 --queue 1
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 2 --queue 2
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 3 --queue 3
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 4 --queue 4
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 5 --queue 5
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 6 --queue 6
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 7 --queue 7
admin@sonic:~$ sudo config interface qos tc-queue bind Ethernet64 tc-queue-prof
```

Intel switches only

The below table is the QoS mapping is used for enabling PFC on Intel switches. The 802.1p priority of 0, 6, 7 aren't able to use for enabling PFC WD.

Dot1p	TC	PG	Queue
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5

The following commands are used to configure the above QoS mappings. Configure the Dot1p-to-TC mapping as:

Example, Intel switches only

```
admin@sonic:~$ sudo config qos dot1p-tc add dot1p-tc-prof --dot1p 1 --tc 1
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 2 --tc 2
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 3 --tc 3
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 4 --tc 4
admin@sonic:~$ sudo config qos dot1p-tc update dot1p-tc-prof --dot1p 5 --tc 5
admin@sonic:~$ sudo config interface qos dot1p-tc bind Ethernet64 dot1p-tc-prof
```

Configure the Tc-to-PG mapping as:

Example, Intel switches only

```
admin@sonic:~$ sudo config qos tc-pg add tc-pg-prof --tc 1 --pg 1
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 2 --pg 2
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 3 --pg 3
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 4 --pg 4
admin@sonic:~$ sudo config qos tc-pg update tc-pg-prof --tc 5 --pg 5
admin@sonic:~$ sudo config interface qos tc-pg bind Ethernet64 tc-pg-prof
```

Configure the TC-to-Queue mapping as:

Example, Intel switches only

```
admin@sonic:~$ sudo config qos tc-queue add tc-queue-prof --tc 1 --queue 1
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 2 --queue 2
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 3 --queue 3
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 4 --queue 4
admin@sonic:~$ sudo config qos tc-queue update tc-queue-prof --tc 5 --queue 5
admin@sonic:~$ sudo config interface qos tc-queue bind Ethernet64 tc-queue-prof
```

Step3, Apply the lossless buffer profile settings to PGs and queues

Apply the lossless buffer profile settings to PGs and queue which are enabled PFC function. For Intel switches, only the 1 .. 5 PGs and queue can be used to enable PFC.

Apply the lossless buffer profile of **ingress_lossless_profile** to PG as the example:

Example

```
admin@sonic:~$ sudo config interface buffer bind priority-group Ethernet64 1 ingress_lossless_profile
```

Apply the lossless buffer profile to **egress_lossless_profile** to queue as the below example:

Example

```
admin@sonic:~$ sudo config interface buffer bind queue Ethernet64 1 egress_lossless_profile
```

Step4, Enable the priority of PFC

Enable the 802.1p priority of PFC on the interface. For Intel switches, only 1 .. 5 priority can be used to enable for PFC WD. For example, enable priority 1 on Ethernet64 as below:

Example

```
admin@sonic:~$ sudo config interface pfc priority Ethernet64 1 on
```

Step5, Start the PFC WD

The below command shows how to enable PFC watchdog. This example uses the action of the forward, detection time of 400 ms, restoration time of 400 ms, and enabled port of Ethernet64. Always use the forward action as "**forward**" for Broadcom switches.

Example

```
admin@sonic:~$ config pfcwd start --action forward --restoration-time 400 ports Ethernet64 detection-time 400

admin@sonic:~$ show pfcwd config
      PORT      ACTION  DETECTION TIME  RESTORATION TIME
-----
Ethernet64      drop           400             400

admin@sonic:~$
```

For more information, see the [\(202111.2\) PFC Watchdog](#) section in the CLI reference guide.

(202111.2) Displaying Queue and Priority-Group Counters

SONiC supports commands that display details of port queue counters and buffer watermarks.

Use the **show queue counters** command to display packet and byte counters for all queues of all ports or one specific-port.

Example

```
admin@sonic:~$ show queue counters Ethernet72
```

Port	TxQ	Counter/pkts	Counter/bytes	Drop/pkts	Drop/bytes
Ethernet72	UC0	0	0	0	0
Ethernet72	UC1	0	0	0	0
Ethernet72	UC2	0	0	0	0
Ethernet72	UC3	0	0	0	0
Ethernet72	UC4	0	0	0	0
Ethernet72	UC5	0	0	0	0
Ethernet72	UC6	0	0	0	0
Ethernet72	UC7	0	0	0	0
Ethernet72	UC8	0	0	0	0
Ethernet72	UC9	0	0	0	0
Ethernet72	MC0	0	0	0	0
Ethernet72	MC1	0	0	0	0
Ethernet72	MC2	0	0	0	0
Ethernet72	MC3	0	0	0	0
Ethernet72	MC4	0	0	0	0
Ethernet72	MC5	0	0	0	0
Ethernet72	MC6	0	0	0	0
Ethernet72	MC7	0	0	0	0
Ethernet72	MC8	0	0	0	0
Ethernet72	MC9	0	0	0	0

NOTE: Queue counters can be cleared by using the **sonic-clear queuecounters** command.

To display the user watermark for port queues (egress shared pool occupancy per queue), either for unicast queues or multicast queues, use the **show queue watermark** command.

Example

```
admin@sonic:~$ show queue watermark unicast
```

Egress shared pool occupancy per unicast queue:

Port	UC0	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9
Ethernet0	0	0	0	0	0	0	0	0	0	0
Ethernet4	416	0	0	0	0	0	0	0	0	0
Ethernet8	416	0	0	0	0	0	0	0	0	0
Ethernet12	416	0	0	0	0	0	0	0	0	0

To display the user watermark or persistent-watermark for port ingress "headroom" or "shared pool occupancy" per priority-group for all ports, use the **show priority-group** command. Or, use the **show queue persistent-watermark** command to display the user persistent-watermark for either unicast or multicast queues (egress shared pool occupancy per queue) for all ports.

Example

```
admin@sonic:~$ show priority-group watermark shared
```

```
Ingress shared pool occupancy per PG:
```

Port	PG0	PG1	PG2	PG3	PG4	PG5	PG6	PG7
Ethernet0	0	0	0	0	0	0	0	0
Ethernet4	0	0	0	0	0	0	0	0
Ethernet8	0	0	0	0	0	0	0	0
Ethernet12	0	0	0	0	0	0	0	0

```
admin@sonic:~$ show queue persistent-watermark unicast
```

```
Egress shared pool occupancy per unicast queue:
```

Port	UC0	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9
Ethernet0	0	0	0	0	0	0	0	0	0	0
Ethernet4	416	0	0	0	0	0	0	0	0	0
Ethernet8	416	0	0	0	0	0	0	0	0	0

NOTE: Both the "user watermark" and "persistent watermark" can be cleared using the following commands:

- **sonic-clear queue persistent-watermark unicast**
- **sonic-clear queue persistent-watermark multicast**
- **sonic-clear priority-group persistent-watermark shared**
- **sonic-clear priority-group persistent-watermark headroom**

For more information, see the (202111.2) QoS section in the CLI reference guide.

(202111.2) Queue Scheduling

Queue scheduling provides preferential treatment of traffic classes mapped to specific egress queues. SONiC supports SP, WRR, and DWRR scheduling disciplines.

- SP - Higher priority egress queues get scheduled for transmission over lower priority queues.
- WRR - Egress queues receive bandwidth proportional to the configured weight. The scheduling granularity is per packet which causes large and small packets to be treated the same. Flows with large packets have an advantage over flows with smaller packets.
- DWRR - Similar to WRR but uses deficit counter scheduling granularity to account for packet size variations and provide a more accurate proportion of bandwidth.



- Queue scheduling is only supported on physical port(s) and unicast queue(s).
- WRR scheduling is not supported on Intel switches.

The queue scheduling can be configured through CLI command. You need to first configure a “scheduler” profile, and then bind the profile to port queues. Following steps show how to configure queue scheduling:

Configuring queue scheduling

1. Configure the scheduler profile, using **config scheduler add** command. For example, create a scheduler profile "sched-prof-wrr40", the scheduler type is set to "WRR", the bandwidth is set to 40 percentage.

Example

```
admin@sonic:~$ sudo config scheduler add sched-prof-wrr40 --sched_type WRR --weight 40
```

To create a scheduler profile "sched-prof-wrr20", the scheduler type is set to "WRR", the bandwidth is set to 20 percentage.

Example

```
admin@sonic:~$ sudo config scheduler add sched-prof-wrr20 --sched_type WRR --weight 20
```

2. Configure the scheduler profile to queue, using **config interface scheduler bind queue** command. For example, apply the scheduler profile "sched-prof-wrr40" to queue 3 of Ethernet48.

Example

```
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet48 3 sched-prof-wrr40
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet48 4 sched-prof-wrr20
```

Apply the scheduler profile "sched-prof-wrr20" to queue4 of Ethernet48.

Example

```
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet48 4 sched-prof-wrr20
```

The queue(s) is not specified the scheduler using the default value.

- On Broadcom switches, the default value is WRR with weight 1.
- On Intel switches, the default value is DWRR with weight 1.

3. Verify the configuration, using **show interface scheduler** command.

Example

```
admin@sonic:~$ show interfaces scheduler
Ethernet48
```

Type	Queue	Profile	Scheduling Mode	Weight	Shaper Type	Bandwidth
Queue	3	sched-prof-wrr40	WRR	40	N/A	0
Queue	4	sched-prof-wrr20	WRR	20	N/A	0

For more information, see the (202111.2) Scheduling & Shaping section in the CLI reference guide.

(202111.2) Weighted Random Early Detection (WRED)

Weighted Random Early Detection (WRED) is a variation of Random Early Detection (RED) algorithm provides early detection of congestion per traffic class. RED is an Active Queue Management (AQM) mechanism that is implemented for congestion avoidance. WRED /RED addresses the issues caused by Tail-Drop which responses the congestion too late. It detects the congestion just the queue is already full and has no choice but to drop all incoming packets in this situation. That causes the throughput degraded because there have too much packets are dropped. The delay also increased due to the mean of queue is increased. All the incoming packets be discarded cause the congestion notification sent simultaneously that heads the global synchronization. The global synchronization heads to the issue of unfairness of flows. WRED probabilistic drop packets when the average queue length reaches a defined threshold. With TCP, the congestion control will be used to slow down the transmission, and as the result, it keeps the queue of the switch as low as possible. The WRED drop probability is used to determine a packet shall be dropped or enqueued according to current average queue length. The parameters that are used to calculate the drop probability are as below:

- minimum threshold: minimum threshold to start WRED dropping.
- maximum threshold: maximum threshold for WRED dropping.
- maximum drop probability: maximum dropping probability.

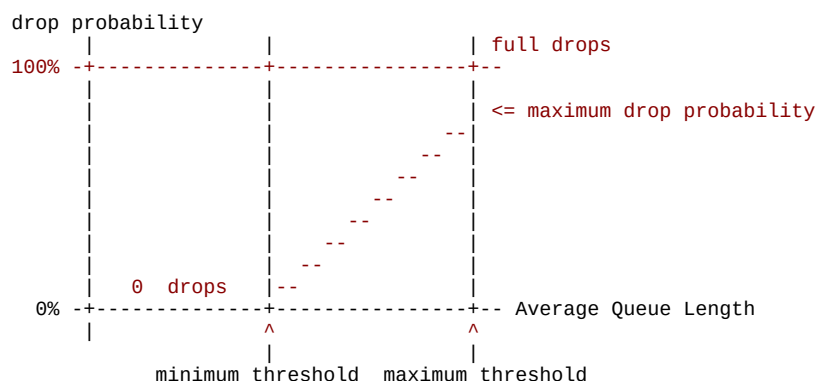
When WRED enabled, process the packets based on following rules:

- When the average queue length is below the **minimum threshold**, no packet is dropped.
- When the average queue length is between **minimum threshold** and **maximum threshold**, incoming packets are dropped according to the drop probability below.

$$\text{Drop probability} = \text{maximum drop probability} \times (\text{average queue length} - \text{minimum threshold}) \div (\text{maximum threshold} - \text{minimum threshold})$$

- When the average queue length exceeds the **maximum threshold**, all the incoming packets are dropped.

The below figure visualizes the above behaviors.



i The following rules should be applied for setting the WRED parameters:

- The minimum threshold should be set high enough to avoid the WRED mechanism not be triggered too early. With network traffic mainly being bursty in nature, the minimum threshold should be set as a corresponding high level to make the link utilization at an acceptable level.
- The size between minimum and maximum threshold should be set large enough to make the connections have enough time to slow down their transmission. If the difference is too small the behavior is similar to Tail-Drop to cause global synchronization.

The WRED can be configured through CLI command. You need to first configure a WRED profile, using **config wred add** command, and then bind the profile to the port interface's queues, using **config interface wred bind queue** command.

Configuring WRED

1. Configure the WRED packet drop profile **wred-prof**. This example uses a minimum threshold of 100k bytes, a maximum threshold of 800k bytes, a maximum drop probability of 100%.

```
admin@sonic:~$ sudo config wred add wred-prof --mode wred --gmin 100000 --gmax 800000 --gdrop 100
```

2. Apply the drop profile **wred-prof** to queue 0 on Ethernet176.

```
admin@sonic:~$ sudo config interface wred bind queue Ethernet176 0 wred-prof
```

3. Verify the configuration using **show interface wred** command.

```
admin@sonic:~$ show interface wred
Ethernet176
Queue: 0
ECN/WRED: wred-prof
Color    Mode      Min Threshold  Max Threshold  Drop Probability
-----
Green    WRED        100000         800000         100
Yellow
Red
```

For more information, see the (202111.2) WRED & ECN section in the CLI reference guide.

(202111.2) RDMA over Converged Ethernet (RoCE)

Remote Direct Memory Access (RDMA) is a direct memory access technology provided by InfiniBand networks. It enables to access the memory of remote computers like locally DMA. This happens without the involvement of the processor and operating system for all RDMA-enabled computers. Without copying the data between the application, operation system, and NIC the data transfer rate and the latency are sizable improved. The usage of the CPU is nearly zero consumption.

RDMA over Converged Ethernet (RoCE) is a network protocol that enables RDMA over an Ethernet network. RoCEv2 (RoCE version 2) enhances RoCEv1 with a UDP/IP and using the well-known UDP port 4791. To construct a RoCE network, the congestion control will be the critical challenge needs to deal with. It needs the key features is provided in the switch.

- Priority Flow Control (PFC) provides the hop-by-hop layer 2 congestion control. PFC extends the IEEE 802.3x Ethernet PAUSE, allows to pause the traffic based on its priority (802.lp) rather than pause all the traffic on a link.
- Explicit congestion notification (ECN) provides the end-to-end layer 3 congestion control. When the congestion is detected by the switch, the ECN field is set in the IP header of a packet before forwarding it. If the receiver device received the packets with ECN marked, it sends the congestion notification packets (CNP) to the sender device to reduce the transmit packet rate. The CNP packets continue to send to the sender until the congestion is released.
- Enhanced Transmission Selection (ETS) provides minimum guaranteed bandwidth per class basis.

If only enable PFC on an RoCE network, it can guarantee no packet loss under congestion. PFC is implemented by NIC resulting in a micro-second level response time. However, PFC mechanism has numerous well-known problems which reduces the throughput of the uncongested (victim) flows, e.g., Head-of-Line Blocking (HoLB), congestion spreading and deadlock. Uses ECN providing the congestion notification to the source of congestion can decrease the number of PFC triggering. RoCEv2 Congestion Management (RCM) defines the congestion flows based on the ECN field mark.

To configure PFC you can refer to this document [\(202111.2\) Priority Flow Control \(PFC\)](#) for more information.

To configure PFC Watchdog you can refer to this document [\(202111.2\) Starting the PFC Watchdog](#) for more information.

To configure ECN you can refer to this document [\(202111.2\) Explicit Congestion Notification \(ECN\)](#) for more information.

To configure ETS you can refer to this document [\(202111.2\) Queue Scheduling](#) for more information.

Below provides an example of how to configure **DCS810 (Intel Tofino 2)** for RoCE over a DSCP-based lossless network.

There have multiple traffic types on network:

- Lossy traffic for LAN traffic.
- Loss-less traffic for RDMA traffic.

The below table is used for the following example:

Traffic Flow	Loss-less	DSCP	PG	Queue	Bandwidth (%)
LAN flow	No	0, 63	2	2	50%
RoCE, data	Yes	26	3	3	50%
RoCE, CNP	Yes	48	4	4	strict

Configuring Buffer

1. Change the buffer configuration to lossless.

Example

```
admin@sonic:~$ sudo config qos reload
```

2. Save the configuration and restart the switch for the change to take effect.

Example

```
admin@sonic:~$ sudo config save -y
admin@sonic:~$ sudo reboot
```

Configuring ETS: Map DSCP to TCs and allocate bandwidth

1. Create a DSCP-to-TC profile and add the following mapping.

DSCP	TC
0, 63	0
26	3
48	4

Example

```
admin@sonic:~$ sudo config qos dscp-tc add dscp-tc-prof --dot1p 0,63 --tc 0
admin@sonic:~$ sudo config qos dscp-tc update dscp-tc-prof --dot1p 26 --tc 3
admin@sonic:~$ sudo config qos dscp-tc update dscp-tc-prof --dot1p 48 --tc 4
```

2. Configure the DSCP-to-TC profile on the specified port(s).

Example

```
admin@sonic:~$ sudo config interface qos dot1p-tc bind Ethernet0 dot1p-tc-prof
```

3. Create a TC-Queue profile and add the following mapping.

TC	Queue
0	0
3	3
4	4

Example

```
admin@sonic:~$ sudo config qos tc-queue add tc-q-prof --tc 0 --queue 0
admin@sonic:~$ sudo config qos tc-queue update tc-q-prof --tc 3 --queue 3
admin@sonic:~$ sudo config qos tc-queue update tc-q-prof --tc 4 --queue 4
```

4. Configure the TC-to-Queue profile on the specified port(s).

Example

```
admin@sonic:~$ sudo config interface qos tc-queue bind Ethernet0 tc-q-prof
```

5. Create the following scheduler profiles.

Scheduler Profile	Bandwidth
sched-wrr-50	WRR, 50%
sched-strict	strict

Note: On Intel switches, use DWRR instead of WRR.

Example

```
admin@sonic:~$ sudo config scheduler add sched-wrr-50 --sched_type WRR --weight 50
admin@sonic:~$ sudo config scheduler add sched-strict --sched_type STRICT
```

6. Configure the scheduler profiles on the specified queues.

Configure the scheduler profile sched-wrr-50 to queue 0 and 3, sched-strict to queue 4.

Example

```
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet0 0 sched-wrr-50
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet0 3 sched-wrr-50
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet0 4 sched-strict
```

Configuring PFC: Enable PFC for lossless traffic

1. Configure the ingress PG to apply lossless buffer settings.

Example

```
admin@sonic:~$ sudo config interface buffer bind priority-group Ethernet0 3 ingress_lossless_profile
admin@sonic:~$ sudo config interface buffer bind priority-group Ethernet0 4 ingress_lossless_profile
```

2. Configure the egress queue to apply lossless buffer settings.

Example

```
admin@sonic:~$ sudo config interface buffer bind queue Ethernet0 3 egress_lossless_profile
admin@sonic:~$ sudo config interface buffer bind queue Ethernet0 4 egress_lossless_profile
```

3. Enable PFC on the specified port(s).

Example

```
admin@sonic:~$ sudo config interface pfc priority Ethernet0 3 on
admin@sonic:~$ sudo config interface pfc priority Ethernet0 4 on
```

4. Intel Wedge100BF and AS9516 only. Configure the PFC-to-PG mapping.

Create a PFC-to-PG profile.

Example

```
admin@sonic:~$ sudo config qos pfc-priority-pg add pfc-pg-prof --pfc-priority 3 --pg 3
admin@sonic:~$ sudo config qos pfc-priority-pg update pfc-pg-prof --pfc-priority 4 --pg 4
```

Configure the PFC-to-PG profile to the specified port(s).

Example

```
sudo config interface qos pfc-priority-pg bind Ethernet0 pfc-pg-prof
```

Configuring ECN: Enable ECN for congestion notification

1. Create a WRED profile.

Example

```
admin@sonic:~$ sudo config wred add wred-prof --mode ecn --gmin 100000 --gmax 800000 --gdrop 100
```

2. Configure the WRED profile on the specified queue(s).

Example

```
admin@sonic:~$ sudo config interface wred bind queue Ethernet0 0 wred-prof
admin@sonic:~$ sudo config interface wred bind queue Ethernet0 3 wred-prof
```

(202111.2) Port and Queue Shaping

Port shaping allows you to control the traffic bandwidth of a port so that it passes less than the full line rate. This feature is important for preventing congestion on certain network links that might drop packets if the maximum bandwidth is exceeded.

Similarly, queue shaping limits the rate at which specific port queues transmit packets. Queue shaping can be useful for preventing high-priority traffic from blocking lower-priority queues.

SONiC supports egress port and queue shaping by two types of metering; either by kilo-bits per second (kbps) or packets per second (pps). Also, you can specify a maximum bandwidth.



- Egress port and queue shaping is only supported on physical port(s) and unicast queue(s).
- Egress port shaping is only support on Intel switches.

The port and queue shaping can be configured through CLI command. You need to first configure a “scheduler” profile, and then bind the profile to port interfaces or port queues. Following steps show how to configure port/queue shaping:

Configuring port shaping

1. Configure the scheduler profile "profile-1", using **config scheduler add** command. The meter type "bytes" is defined, the maximum bandwidth rate is set to 10 Gbps.

Example

```
admin@sonic:~$ sudo config scheduler add profile-1 --shaper_type=bytes --bandwidth=10g
```

2. Apply the scheduler profile "profile-1" to Ethernet0, using **config interface scheduler bind port** command.

Example

```
admin@sonic:~$ sudo config interface scheduler bind port Ethernet0 profile-1
```

3. Verify the configuration using **show interface scheduler** command.

Example

```
admin@sonic:~$ show interface scheduler
Ethernet0
Name      Scheduling Type  Weight  Shaper Type  Bandwidth
-----
prof_1    N/A              N/A     bytes        10 Gbps
```

Configuring queue shaping

1. Configure the scheduler profile "profile-1", using **config scheduler add** command. The meter type "bytes" is defined, the maximum bandwidth rate is set to 10 Gbps.

Example

```
admin@sonic:~$ sudo config scheduler add profile-1 --shaper_type=bytes --bandwidth=10g
```

2. Apply the scheduler profile "profile-1" to queue 0 on Ethernet0, using **config interface scheduler bind queue** command.

Example

```
admin@sonic:~$ sudo config interface scheduler bind queue Ethernet0 0 profile-1
```

3. Verify the configuration using **show interface scheduler** command.

Example

```
admin@sonic:~$ show interface scheduler
Ethernet0
Type      Queue  Profile  Scheduling Mode  Weight  Shaper Type  Bandwidth
-----
Queue     0      profile-1 N/A              N/A     bytes       10 Gbps
```

For more information, see the (202111) Scheduling & Shaping section in the CLI reference guide.

(202111.2) Port Rate Limit (Traffic Policing)

Port rate-limit, allows you to control the traffic bandwidth of a port so that it passes less than the full line rate. This feature is important for preventing congestion on certain network links that might drop packets if the maximum bandwidth is exceeded.

SONiC supports ingress rate limit by two types of metering; either by kilo-bits per second (kbps) or packets per second (pps). Also, you can specify a maximum bandwidth and a maximum burst size.



- Port Rate Limit is only supported on Broadcom switches.
- Port Rate Limit uses the token bucket algorithm to manage the maximum rate of traffic can pass-through on a port.
- Port Rate Limit is the only support ingress direction of a physical port.
- Port Rate Limit can not work with ACL at the same time.

The port rate-limit can be configured through CLI command. To enable the port rate-limit on a specified port, use the **config interface rate-limit add** command. To disable the port rate-limit on a specified port, use the **config interface rate-limit del** command.

The following example is to enable port rate limit on port Ethernet0. The meter type is "bytes", the rate is set to 10 Gbps, and the burst size is set to 1024 bytes.

Example

```
sudo config interface rate-limit add Ethernet0 --meter-type bytes --rate 10g --burst-size 1k
```

The following example is to enable port rate limit on port Ethernet0. The meter type is "packets", the rate is set to 100 pps.

Example

```
sudo config interface rate-limit add Ethernet0 --meter-type packets --rate 100
```

The following example is shows how to verifying the configuration by using **show interfaces rate-limit** command.

Example

```
admin@sonic:~$ show interface rate-limit
```

Interface	Meter Type	Rate	Burst Size
Ethernet0	bytes	10 Gbps	1 Kib

The following example is to disable port rate limit on port Ethernet0.

Example

```
sudo config interface rate-limit del Ethernet0
```

For more information, see the (202111.2) Port Rate Limit section in the CLI reference guide.

(202111.2) Shared Buffer Configuration

Shared Buffer Configuration allows users to configure the shared buffer for packet buffering within the device. All the received packets will be stored in the internal buffer before being transmitted. The internal buffer is also called "shared buffer" because the buffer is shared by all ingress and egress ports. To avoid the problem of starvation and unfairness, the buffer configuration allows users to define the buffer pool including the maximum amount of shared buffer that can be used. A buffer profile template can be further defined to refer to the buffer pool and be applied to ingress priority-groups (PG) and egress queues on ports.

To configure the buffer pool, use the **config buffer pool** command. To configure the buffer profile, use the **config buffer profile** command. To apply the buffer profile to PG or queue, use **config interface buffer bind priority-group** command and **config interface buffer bind queue** command respectively.

Configuration Guide

1. Configuring the buffer pool.

The buffer amounts vary by switch platform. The following table shows the buffer amounts per switch platform.

Table 1: Switch buffer size.

Switch	Total ingress buffer size (bytes)	Total egress buffer size (bytes)
DCS204(AS7726-32X), DCS203(AS7326-56X), DCS201(AS5835)	32,677,120	32,379,648
EPS202(AS4630-54PE)	8,370,944	8,071,424
DCS208(AS5812)	16,628,352	16,122,080
DCS800(Wedge100BF)	12,766,208	12,766,208
DCS810(AS9516-32D)	61,680,000	61,680,000

The buffer amounts can be divided by the individual buffer pools. In a buffer pool, you can specify the total shared buffer size in the pool and headroom size if the pool is used for lossless PG. The reserved size (or called minimum guaranteed buffer) of a PG or queue is not included in the shared buffer size of the pool. The following equations can be used to indicate the relation.

Total ingress buffer size (shared buffer size + headroom size for all ingress buffer pool) + (reserved size for all PGs)

Total egress buffer size (shared buffer size for all egress buffer pool) + (reserved size for all queues)

Headroom Size Calculation

Headroom buffer is used to absorb inflight packets while the upstream is responding to the PFC PAUSE. The headroom size requirement depends on processing and queuing delay on downstream switch, transmission time based on port speed and cable length, response time on upstream switch. In general, the size can be calculated approximately as below.

headroom size of a PG = 2 × (latency + media delay + cable length × 5) × pps × packets to cells × cell size ÷ 10⁹

packets to cells conversion = ceiling[(packet size + M) ÷ cell size], M is 64 for Broadcom switches, 1 for Intel switches

where latency is switch latency for queuing or responding in nanoseconds, using respectively **1,000** (1μs) and **2,000** (2μs) for Broadcom and Intel switches. Media delay is PHY latency in nanoseconds. Cable length is the cable length in meters. 5 is the link propagation delay in nanoseconds calculated by the propagation speed of 200,000,000 meters/sec. pps is packet per second at the current port speed. Cell size, which varies across switch platforms, is shown in Table 2. For example, to calculate the headroom size with the media delay of 100 ns, cable length of 300 meters, pps of 1,369,713 (using the size of 9,126 bytes and the port speed of 100 Gb/s), cell size of 256 bytes on Broadcom Trident 3 switch, we get:

$$65 \text{ KB} \quad 2 \times (1,000 + 100 + 300 \times 5) \times 1,369,713 \times 36 \times 256 \div 10^9$$

The size of headroom pool is the sum of the size of the headroom requirement for the all lossless PGs. E.g., the size for 32 ports 100Gb/s and enabling 2 priorities per port is $65\text{KB} \times 32 \times 2 = 4,160 \text{ KB}$. All ports and all priorities can be paused simultaneously.

Table 2: Cell size.

Switch	bytes per cell
DCS204(AS7726-32X), DCS203(AS7326-56X), DCS201(AS5835), EPS202(AS4630-54PE)	256
DCS208(AS5812)	208
DCS800(Wedge100BF)	80
DCS810(AS9516-32D)	176

The following example configures the ingress buffer pool "ingress_lossless_pool". This example uses shared buffer size of 2,940,941 bytes and headroom size of 14,704,704 bytes.

Example

```
admin@sonic:~$ sudo config buffer pool add ingress_lossless_pool --type ingress --size 2940941 --xoff-size 14704704
```

On Broadcom Trident 2 plus switches only, the size of the buffer pool contains the total shared buffer size, the headroom size and the reserved size. `--xoff-size` is not available on these switches. The following command can be used instead (assuming the reserved size is 0).

Example

```
admin@sonic:~$ sudo config buffer pool add ingress_lossless_pool --type ingress --size 17645645
```

2. Configuring the buffer profile.

For each buffer profile, there are:

- pool name: the name of the pool which contains the maximum shared size and the maximum headroom size that can be used in this profile.
- reserved size: the minimum guaranteed size for the priority-group or queue on a port.
- shared size: the maximum shared size for the priority-group or queue on a port. When the reserved size is insufficient, the shared size will be used. It's possible to specify an absolute value or a percentage of the buffer pool size.
- (optional) headroom size (xoff): the headroom size for absorbing inflight packets to avoid packet loss for PFC.
- (optional) xon and xon-offset: used to determine the XON limit, below which the XOFF state will be released when the buffer limit at the ingress port falls. The XON limit is the larger value of (*xon*) and (*the overall PG size - xon-offset*).

The following example configures a buffer profile "ingress_lossless_profile". The example uses the buffer pool "ingress_lossless_pool", reserved size of 0 bytes, shared buffer percentage of 66%, headroom size of 14,704,704 bytes, xon of 1,746,919 bytes, and xon-offset of 18,432 bytes.

Example

```
admin@sonic:~$ sudo config buffer profile add ingress_lossless_profile --pool ingress_lossless_pool --size 0
--shared-percent 66 --xoff 14704704 --xon 1746919 --xon-offset 18432
```

On Intel switches, it's not supported to specify the xon-offset value. Use the following example instead.

Example

```
admin@sonic:~$ sudo config buffer profile add ingress_lossless_profile --pool ingress_lossless_pool --size
0 --shared-percent 66 --xoff 14704704 --xon 18432
```

3. Configuring the buffer profile to PG or queue.

For example, to configure the buffer profile "ingress_lossless_profile" to PG3 of Ethernet60:

Example

```
admin@sonic:~$ sudo config interface buffer bind priority-group Ethernet60 3 ingress_lossless_profile
```

For example, to configure the buffer profile "egress_lossless_profile" to queue3 of Ethernet60:

Example

```
admin@sonic:~$ sudo config interface buffer bind queue Ethernet60 3 egress_lossless_profile
```

For more information, see the (202111.2) Shared Buffer section in the CLI reference guide.

(202111.2) ACL-based CoS

- Match SIP/DIP and remark DSCP value
 - Create ACL table
 - Apply rules
 - Configure DSCP remarking
- Match vlan id and remark PCP value
 - Create ACL table
 - Apply rules
 - Configure PCP remarking

Match SIP/DIP and remark DSCP value

The following example matches packets based on source IP address and destination IP address at the ingress port "Ethernet0" and remarks their DSCP value at the egress port "Ethernet4".

Create ACL table

Create an ACL table "DATA_L3" of type "L3" (for IPv4) and apply it to the interface "Ethernet0" at the "ingress" stage.

Example

```
admin@sonic:~$ sudo config acl add table DATA_L3 L3 --ports Ethernet0 --stage ingress
```

Apply rules

Apply an ACL rule that matches packets with source IP address "1.1.1.1/32" and destination IP address "2.2.2.2/32", sets their TCs (internal priority) to 3 and forwards them.

Example

```
admin@sonic:~$ sudo config acl update full match_sip_dip.json
```

match_sip_dip.json

```
{
  "acl": {
    "acl-sets": {
      "acl-set": {
        "DATA_L3": {
          "acl-entries": {
            "acl-entry": {
              "1": {
                "ip": {
                  "config": {
                    "source-ip-address": "1.1.1.1/32",
                    "destination-ip-address": "2.2.2.2/32"
                  }
                },
                "config": {
                  "sequence-id": 1
                },
                "actions": {
                  "config": {
                    "forwarding-action": "ACCEPT",
                    "set-traffic-class-action": 3
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Configure DSCP remarking

Create a DSCP remarking profile "remark_dscp" to map TC(3) to DSCP value(8) for rewriting.

```
admin@sonic:~$ sudo config qos remark dscp add remark_dscp --tc 3 --dscp 8
```

Apply the DSCP remarking profile "remark_dscp" to the interface "Ethernet4".

```
admin@sonic:~$ sudo config interface qos remark dscp bind Ethernet4 remark_dscp
```

Match vlan id and remark PCP value

The following example matches packets based on vlan id at the ingress port "Ethernet0" and remarks their PCP values at the egress port "Ethernet4".

Create ACL table

Create an ACL table "DATA_L3" of type "L3" (for IPv4) and apply it to the interface "Ethernet0" at the "ingress" stage.

Example

```
admin@sonic:~$ sudo config acl add table DATA_L3 L3 --ports Ethernet0 --stage ingress
```

Apply rules

Apply an ACL rule that matches packets with vlan id "100" and sets their TCs (internal priority) to 3 and forward them.

Example

```
admin@sonic:~$ sudo config acl update full match_vlan.json
```

match_vlan.json

```
{
  "acl": {
    "acl-sets": {
      "acl-set": {
        "DATA_L3": {
          "acl-entries": {
            "acl-entry": {
              "1": {
                "12": {
                  "config": {
                    "vlan-id": "100"
                  }
                },
                "config": {
                  "sequence-id": 1
                },
                "actions": {
                  "config": {
                    "forwarding-action": "ACCEPT",
                    "set-traffic-class-action": 3
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Configure PCP remarking

Create a PCP remarking profile "remark_pcp" to map TC(3) to PCP value(7) for rewriting.

```
admin@sonic:~$ sudo config qos remark dot1p add remark_pcp --tc 3 --dot1p 7
```

Apply the PCP remarking profile "remark_pcp" to the interface "Ethernet4".

```
admin@sonic:~$ sudo config interface qos remark dot1p bind Ethernet4 remark_pcp
```

(202111.2) Security

(202111.2) AAA and TACACS+

The authentication, authorization, and accounting (AAA) feature provides the main framework for configuring access control on the switch. The three security functions can be summarized as follows:

- Authentication — Identifies users that request access to the network.
- Authorization — Determines if users can access specific services.
- Accounting — Provides reports, auditing, and billing for services that users have accessed on the network.

The AAA functions require the use of configured Terminal Access Controller Access Control System Plus (TACACS+) servers in the network. The security servers can be defined as sequential groups that are applied as a method for controlling user access to specified services. For example, when the switch attempts to authenticate a user, a request is sent to the first server in the defined group, if there is no response the second server will be tried, and so on. If at any point a pass or fail is returned, the process stops.

Use the **aaa authentication login** command to configure whether AAA should use the local database or a remote TACACS+ server database for user authentication. By default, AAA uses a local database for authentication. Local authentication restricts management access based on user names and passwords manually configured on the switch. New users can be added/deleted using Linux commands (note that configuration using Linux commands is not preserved during reboot).

You can enable remote TACACS+ server authentication by selecting "tacacs+" in the **aaa authentication login** command. Administrators need to configure the TACACS+ server accordingly and ensure that connectivity to the TACACS+ server is available. Once TACACS+ server remote authentication is selected, all user logins will be authenticated by the TACACS+ server. If the authentication fails, AAA will check the "failthrough" configuration, and if enabled, authenticate the user based on the local database.

Example

```
admin@sonic:~$ sudo config aaa authentication login tacacs+
```

Use the **aaa authentication failthrough** command to enable the failthrough option.

When failthrough is enabled and multiple TACACS+ server have been configured (up to a maximum of seven), the authentication process will continue through all configured servers, even when an authentication request to the first server fails. If all remote authentication fails, the process will then proceed to local authentication.

When failthrough is disabled and an authentication request fails on the first server, the authentication process will stop and the login will be disallowed.

Example

```
admin@sonic:~$ sudo config aaa authentication failthrough enable
```

```
admin@sonic:~$ show aaa
AAA authentication login local
AAA authentication failthrough False
```

For more information on AAA, see the (202111.2) AAA section in the CLI reference guide.

(202111.2) Configuring TACACS+ Server Parameters

When a remote authentication server is used, you must specify the message exchange parameters for the TACACS+ protocol. With multiple servers specified (maximum of seven), some parameters are global and can be applied to all servers, or the parameters can be configured separately for each server.

Use the **config tacacs add** command to specify a TACACS+ server. Specify the server IP address and optionally other parameters if they are not set at the global level. The **authtype**, **passkey** and **timeout** parameters must be defined, either using the **config tacacs add** command or by using the global setting commands **config tacacs authtype**, **config tacacs passkey**, and **config tacacs timeout**.

Example

```
admin@sonic:~$ sudo config tacacs add 10.11.12.13 -t 10 -k testing789 -a pap -o 50 -p 9
```

To remove a TACACS+ server, use the **config tacacs delete** command and just specify the IP address.

Example

```
admin@sonic:~$ sudo config tacacs delete 10.11.12.13
```

For more information on configuring TACACS+ servers, see the (202111.2) TACACS+ section in the CLI reference guide.

(202111.2) Configuring RADIUS Server Parameters

When a remote authentication server is used, you must specify the message exchange parameters for the RADIUS protocol. With multiple servers specified (maximum of 8), some parameters are global and can be applied to all servers, or the parameters can be configured separately for each server.

Use the **config radius add** command to specify a RADIUS server. Specify the server IP address and optionally other parameters if they are not set at the global level. The authtype, passkey and timeout parameters must be defined, either using the **config radius add** command or by using the global setting commands **config radius authtype**, **config radius passkey**, and **config radius timeout**.

Example

```
admin@sonic:~$ sudo config radius add 10.11.12.13 -k test1234
```

To remove a RADIUS server, use the **config radius delete** command and just specify the IP address.

Example

```
admin@sonic:~$ sudo config radius delete 10.11.12.13
```

Example: configuring FreeRADIUS

To configure the RADIUS server edit the RADIUS configuration file (/etc/raddb/clients.conf). Below shows an example to configure the client IP address of 192.168.0/24, secret passkey of "test1234".

Example

```
client 192.168.8.0/24 {
    secret = test1234
}
```

To add user for authentication edit the RADIUS user configuration file (/ect/raddb/users). Below shows an examble to add a user, name of "ruser", password of "mypassword", the privilege of 7.

Example

```
ruser    Cleartext-Password := "mypassword"
         Management-Privilege-Level = 7
```

RADIUS support the privilege level of 1 ~ 15. Privilege level 15 is for the administrator user.



The user authenticated by RADIUS will obtain the privilege level without needing to enable authorization by "aaa authorization" command.

Start FreeRADIUS server:

Example

```
admin@spine-2:~/freeradius$ sbin/radiusd -f
```

For more information on configuring RADIUS servers, see the (202111.2) RADIUS section in the CLI reference guide.

(202111.2) Access Control Lists (ACL)

An Access Control Lists (ACL) is a sequential list of matching criteria with permit or deny action that applied to IPv4/IPv6 addresses or other specific criteria. SONiC tests ingress or egress packets against the criteria on ACL, one by one. The packet is accepted if it matched a permit rule, or dropped if it matched a deny rule. If no rules matched, the packet is matched the default rule an implicit DENY ALL rule.

ACL creation then Rules provisioning

Create ACL table

ACL is created with a name, type, and interface which binds the ACL to (Refer config acl add table command). Before creating the ACL table, user needs to have (a) matching criteria which is examined, (b) applied interfaces, (c) direction, ingress or egress.

For example, create an ACL table "DATA_L3", tyoe "L3" (for IPv4), and apply to interfaces "Ethernet0, Ethernet4" in "ingress" stage, the command is

Example

```
admin@sonic:~$ sudo config acl add table DATA_L3 L3 --description "L3 table" --ports "Ethernet0,Ethernet4" --stage "ingress"
```

Apply rules

Before provisioning ACL rules, user needs to configure the matching criteria and action in configuration file (refer (202111.2) ACL Rule Configuration File), then using config acl update full (or config acl update incremental) command to load (or replace) the rules in the configuration file. There is default rule an implicit DENY ALL rule be inserted at the end.

For example, update those rules specified in configuration file, acl_rule_L3_scrip_dstip.json.

Example

```
admin@sonic:~$ sudo config acl update incremental acl_rule_L3_scrip_dstip.json
```

Sample of ACL configuration

acl_rule_L3_scrip_dstip.json

```
{
  "acl":
  {
    "acl-sets":
    {
      "acl-set":
      {
        "DATA_L3":
        {
          "acl-entries":
          {
            "acl-entry":
            {
              "RULE_1":
              {
                "ip":
                {
                  "config":
                  {
                    "destination-ip-address": "4.4.4.4/32",
                    "source-ip-address": "3.3.3.3/32"
                  }
                },
                "config":
                {
                  "sequence-id": 1
                },
                "actions":
                {
                  "config":
                  {
                    "forwarding-action": "ACCEPT"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

The following shows how to configure a "blacklist" as the below example using ACL configuration file.

- 1) drop sip=3.3.3.3 and dip=4.4.4.4
- 2) forward any
- 3) drop any (default rule, auto inserted)

The second rule in the configuration file is used to disable the default rule. The configuration file of the example shows as below:

acl_rule_blacklist.json

sudo config acl add table DATA_L3 L3

cat > acl_rule.json << EEE

```
{
  "acl":
  {
    "acl-sets":
    {
      "acl-set":
      {
        "DATA_L3":
        {
          "acl-entries":
          {
            "acl-entry":
            {
              "1":
              {
                "ip":
                {
                  "config":
                  {
                    "destination-ip-address": "4.4.4.4/32",
                    "source-ip-address": "3.3.3.3/32"
                  }
                },
                "config":
                {
                  "sequence-id": 1
                },
                "actions":
                {
                  "config":
                  {
                    "forwarding-action": "DROP"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

IP tables rules

The `caclmgrd` process control the ACL related rule's update after the related database change, it also initialize the default iptables rules when the process start.

It will check interface configured on Ethernet/PortChannel/VLAN/Management interface add block IP2ME rule into iptables by default.

The following example blocks the IP address (DROP all -- 0.0.0.0/0 192.168.0.0) configured on the management interface (eth0).

acl_rule_blacklist.json

```
admin@leaf-1:~$ show ip in
Interface      Master      IPv4 address/mask  Admin/Oper  BGP Neighbor  Neighbor IP
-----
docker0                240.127.1.1/24    up/down    N/A         N/A
eth0                  192.168.8.111/20  up/up      N/A         N/A
lo                    127.0.0.1/16     up/up      N/A         N/A
admin@leaf-1:~$ sudo iptables -n -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
ACCEPT     all  --  127.0.0.1              0.0.0.0/0
ACCEPT     all  --  0.0.0.0/0              0.0.0.0/0          ctstate RELATED,ESTABLISHED
ACCEPT     icmp --  0.0.0.0/0              0.0.0.0/0          icmptype 8
ACCEPT     icmp --  0.0.0.0/0              0.0.0.0/0          icmptype 0
ACCEPT     icmp --  0.0.0.0/0              0.0.0.0/0          icmptype 3
ACCEPT     icmp --  0.0.0.0/0              0.0.0.0/0          icmptype 11
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp dpts:67:68
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp dpts:546:547
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp dpt:4789
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp spts:49152:65535 dpt:3784
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp dpt:3785
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0          udp spts:49152:65535 dpt:4784
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0          tcp dpt:179
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0          tcp spt:179
DROP       all  --  0.0.0.0/0              192.168.0.0
ACCEPT     all  --  0.0.0.0/0              0.0.0.0/0          TTL match TTL < 2

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

If user configure IP addresses on other interfaces after system startup, `caclmgrd` doesn't reflect the changes automatically.

It needs to trigger it to re-initialize the default iptables rules manually (by `sudo systemctl restart caclmgrd`) or ACL related configuration change.

(202111.2) Configuring a Control Plane ACL

A Control Plane ACL is an enhancement of ACLs and is applied to packets that originate from CoPP to filter and protect the main CPU.

In SONiC you can configure CoPP to forward and limit the packet flow to the CPU, but you cannot apply ACLs to filter out some traffic as they do in the data plane. For example, to filter out (or grant) SNMP, NTP, SSH access from some hosts (or IP address). In this enhancement, a reserved keyword "CTRLPLANE" is used for the ACL rules that apply to the management flow (or control plane flow). The current supported services are SNMP, NTP, and SSH.

Here is an example of how to apply Control Plane ACLs to permit packets from address (3.3.3.3) to access management services and deny access from other IP addresses.

First, use ACL commands to create an ACL table, for example "DATA_CTRL", with the keyword CTRLPLANE, and then specify the applied services, e.g. SNMP and SSH.

```
create control plane acl table
```

```
admin@sonic:~$ sudo config acl add table DATA_CTRL CTRLPLANE --description "CTRLPLANE table" --services "SSH, SNMP"
```

Second, create a configuration file (e.g. service_rules.json) and add the rules that you want to apply for the services. The following example needs to specify:

- packets from (source ip address) 3.3.3.3
- action is accept

configuration file (service_rules.json)

```
{
  "acl":
  {
    "acl-sets":
    {
      "acl-set":
      {
        "DATA_CTRL":
        {
          "acl-entries":
          {
            "acl-entry":
            {
              "1":
              {
                "ip":
                {
                  "config":
                  {
                    "source-ip-address": "3.3.3.3/32"
                  }
                },
                "config":
                {
                  "sequence-id": 1
                },
                "actions":
                {
                  "config":
                  {
                    "forwarding-action": "ACCEPT"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Third, use the **config acl update** command to add the rules.

add the rules to ACL

```
admin@sonic:~$ sudo config acl update incremental service_rules.json
```

You can use the commands **show acl table** or **show acl rule** to examine the configuration.

- ```

 }
 }
}

```
- **acl-entries**: A collection of access entries.
    - **acl-entry**: A collection of rules that are composed of (<rule name>, <configuration>). The key is the <rule name> in the following structure:
 

```

{
 "rule_name_1": {
 <access-list-entries-config>,
 <l2-fields-common-config> (optional),
 <ip-protocol-fields-common-config> (optional),
 <transport-fields-config> (optional),
 <action-config>
 },
 "rule_name_2": {
 <access-list-entries-config>,
 <l2-fields-common-config> (optional),
 <ip-protocol-fields-common-config> (optional),
 <transport-fields-config> (optional),
 <action-config>
 }
}

```
  - <access-list-entries-config>: Access Control list Entries (ACE) config that specifies the matching rule.
    - **sequence-id**: The sequence ID determines the order in which ACL entries are applied. The sequence ID must be unique for each entry in an ACL set. Target devices should apply the ACL entry rules in ascending order determined by sequence ID (low to high), rather than the relying only on order in the list. Minimum: 1. Maximum: 5000. For example: "sequence-id": 1.
  - <l2-fields-common-config>: Ethernet header fields.
    - **vlan-id**: VLAN identifier number. Minimum: "1". Maximum: "4095".
  - <ip-protocol-fields-common-config>: IP protocol fields common to IPv4 and IPv6.
    - **protocol**: The protocol carried in the IP packet, expressed either as its IP protocol number (e.g. "1000"), or by a defined identity:
      - "IP\_ICMP": Internet Control Message Protocol (1).
      - "IP\_IGMP": Internet Group Membership Protocol (2).
      - "IP\_TCP": Transmission Control Protocol (6).
      - "IP\_UDP": User Datagram Protocol (17).
      - "IP\_RSVP": Resource Reservation Protocol (46).
      - "IP\_GRE": Generic Routing Encapsulation (47).
      - "IP\_AUTH": Authentication header, e.g., for IPSEC (51).
      - "IP\_PIM": Protocol Independent Multicast (103).
      - "IP\_L2TP": Layer Two Tunneling Protocol v.3 (115).
 For example: "protocol": "IP\_ICMP".
    - **source-ip-address**: The source IP address of a packet. Valid values are :
      - The string of IPv4 prefix (e.g. "192.168.1.1/32").
      - The string of IPv6 prefix (e.g. "2001::db:1/128").
    - **destination-ip-address**: The destination IP address of a packet. Valid values are :
      - The string of IPv4 prefix (e.g. "192.168.1.2/32").
      - The string of IPv6 prefix (e.g. "2001::db:2/128").
    - **dscp**: The number of the diffserv codepoint. Minimum: 0. Maximum: 63. For example: "dscp": 8.
  - <icmp-fields-config>: Configure data of ICMP and ICMPv6.
    - **type**: ICMP type. Minimum: "0". Maximum: "255".
    - **code**: ICMP code. Minimum: "0". Maximum: "255".
  - <transport-fields-config>: Configuration data of transport-layer packet fields.
    - **source-port**: Source port (e.g. "1000") or range (e.g. "1000..2000"). Minimum: "0". Maximum: "65535".
    - **destination-port**: Destination port (e.g. "1000") or range (e.g. "1000..2000"). Minimum: "0". Maximum: "65535".
    - **tcp\_flags**: List of TCP flags to match. Valid values are :
      - "TCP\_FIN": TCP FIN flag.
      - "TCP\_SYN": TCP SYN flag.
      - "TCP\_RST": TCP RST flag.
      - "TCP\_PSH": TCP PSH flag.
      - "TCP\_ACK": TCP ACK flag.
      - "TCP\_URG": TCP URG flag.
 For example: "tcp\_flags": ["TCP\_FIN", "TCP\_SYN"].
  - <action-config>: Config of the action type.
    - **forwarding-action**: Specifies the forwarding action. One forwarding action must be specified for each ACL entry. One of :
      - "ACCEPT": Accept the packet.
      - "DROP" or "REJECT": Drop the packet without sending any ICMP error message.
        - For example: "forwarding-action": "ACCEPT".
    - **set-traffic-class-action**: Specifies the traffic class. Minimum: 0. Maximum: 7.
      - For example: "set-traffic-class-action": 0.

## Matching Criteria and Actions

### Data Plane ACL

Due to chipset capability, the (ingress, egress)<stage> cannot apply to all (L3, L3V6, MIRROR, MIRRORV6, MIRROR\_DSCP) matching criteria type. Following matrix shows the valid matching criteria <type> and <stage>. After the criteria matched, apply the action.

#### Matching Criteria

| Match criteria         | L3              | L3                         | L3V6            | L3V6                       | MIRROR          | MIRROR | MIRRORV6        | MIRRORV6 | MIRROR_DSCP | MIRROR_DSCP |
|------------------------|-----------------|----------------------------|-----------------|----------------------------|-----------------|--------|-----------------|----------|-------------|-------------|
|                        | Ingress         | Egress                     | Ingress         | Egress                     | Ingress         | Egress | Ingress         | Egress   | Ingress     | Egress      |
| vlan-id                | V               | V                          | V               | V                          | V               | X      | V               | X        | X           | X           |
| protocol               | V               | V                          | V               | V                          | V               |        | V               |          | X           |             |
| source-ip-address      | V (ipv4-prefix) | V (ipv4-prefix)            | V (ipv6-prefix) | V (ipv6-prefix)            | V (ipv4-prefix) |        | V (ipv6-prefix) |          | X           |             |
| destination-ip-address | V (ipv4-prefix) | V (ipv4-prefix)            | V (ipv6-prefix) | V (ipv6-prefix)            | V (ipv4-prefix) |        | V (ipv6-prefix) |          | X           |             |
| tcp_flags              | V               | V                          | V               | V                          | V               |        | V               |          | X           |             |
| source-port            | V               | V (Not support port range) | V               | V (Not support port range) | V               |        | V               |          | X           |             |
| destination-port       | V               | V (Not support port range) | V               | V (Not support port range) | V               |        | V               |          | X           |             |
| dscp                   | X               | X                          | X               | X                          | V               |        | V               |          | V           |             |
| (ICMP) type            | V               | V                          | V               | V                          | V               |        | V               |          | X           |             |
| (ICMP) code            | V               | V                          | V               | V                          | V               |        | V               |          | X           |             |

Note,

"V" is valid matching criteria

"X" is invalid matching criteria.

#### Allowed actions for matched criteria in data plane

| Actions                  | L3 | L3V6 | MIRROR | MIRRORV6 | MIRROR_DSCP |
|--------------------------|----|------|--------|----------|-------------|
| ACCEPT                   | V  | V    | V      | V        | V           |
| DROP / REJECT            | V  | V    | X      | X        | X           |
| set-traffic-class-action | V  | V    | V      | V        | V           |

Note,

- Mirroring is a forwarding action, which does not support DROP or REJECT
- "V": valid action to apply when criteria matched, "X": not allowed, due to chipset capacity

### Control Plane ACL

Control Plane ACL applies to the packet in software protocol stack, which inspects source IP address and tcp flags, and takes action in (Accept, Drop/Reject).

#### Matching Criteria

|                   | CTRLPLANE |
|-------------------|-----------|
| protocol          | X         |
| source-ip-address | V         |

|                        |          |
|------------------------|----------|
| destination-ip-address | <b>X</b> |
| tcp_flags              | V        |
| source-port            | <b>X</b> |
| destination-port       | <b>X</b> |
| dscp                   | <b>X</b> |
| (ICMP) type            | <b>X</b> |
| (ICMP) code            | <b>X</b> |

Note,

"V" is valid matching critieria

"X" is invalid matching criteria.

#### Allowed actions for matched criteria in control plane

| Actions                  | CTRLPLANE |
|--------------------------|-----------|
| ACCEPT                   | V         |
| DROP / REJECT            | V         |
| set-traffic-class-action | <b>X</b>  |

## (202111.2) PINS

This user guide introduces the fundamental operation to configure P4 target which is supported by PINS(P4 Integrated Network Stack, a SONiC-based switch stack allowing switches to be programmed remotely via P4Runtime).

The content of this document includes using p4runtime shell as a P4 client to configure route entry, verifying the configuration in redis database and compiling P4 source code.

**Note** that the configuration of ACL is not supported in this revision and there is a plan to support it in the future release.

- [Introduction](#)
- [Setup the P4 Client on Your Local Environment](#)
  - [Check if the Docker Image of P4runtime-sh is Available in Your Local Repository](#)
- [Start the Connection with P4Runtime Sever and Push a Forwarding Pipeline Configuration with the Shell](#)
- [An Example to Set a Route Entry on P4 Target through P4runtime Shell](#)
  - [Configure Route Interface Table](#)
  - [Configure Neighbor Table](#)
  - [Configure Nexthop Table](#)
  - [Configure Ipv4 Route Entry Table](#)
  - [Verify the Table Entry in Redis Database](#)
- [Troubleshooting & Limitation](#)
- [Appendix A: Compiling P4 Source Code](#)
- [Reference](#)

## Introduction

PINS revealed that the device which is operated with SONiC can be programmable by the remote SDN controllers over P4RT API.

A new container named "**p4rt**" has been added into SONiC system and a program running as a P4runtime Server is listening on TCP port **955** by default inside this new container.

Therefore, you can check with the following command to ensure that the SONiC image on your device has supported PINS function.

```
$ docker ps
$ sudo netstat -lpnt | grep p4rt
```

```
admin@sonic:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
61c12577871c docker-snmp:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes snmp
890bf9bc77d8 docker-sonic-mgmt-framework:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes mgmt-framework
5174af41c9ea docker-sonic-telemetry:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes telemetry
58f45c6cde1e docker-router-advertiser:latest "/usr/bin/docker-ini..." 3 hours ago Up 2 minutes radv
af36fe2f4696 docker-stp:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes stp
c001cca5b656 docker-lldp:latest "/usr/bin/docker-ll..." 3 hours ago Up 2 minutes lldp
d09c7b4aee83 docker-syncd-brcm:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes syncd
1ae07594bcf5 docker-teamd:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes teamd
1aae95e0568a docker-iccpd:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes iccpd
e60d873956db docker-sonic-p4rt:latest "/usr/local/bin/supe..." 3 hours ago Up 2 minutes p4rt
d4fcdcefaefa docker-orchagent:latest "/usr/bin/docker-ini..." 3 hours ago Up 2 minutes swss
a53408347866 docker-fpm-frr:latest "/usr/bin/docker_ini..." 3 hours ago Up 2 minutes bgp
171b53b90f29 docker-platform-monitor:latest "/usr/bin/docker_ini..." 3 hours ago Up 2 minutes pmon
3d5349d7ac26 docker-database:latest "/usr/local/bin/dock..." 3 hours ago Up 31 minutes database
admin@sonic:~$ sudo netstat -lpnt | grep p4rt
tcp6 0 0 :::9559 :::* LISTEN 33125/p4rt
admin@sonic:~$
```

## Setup the P4 Client on Your Local Environment

There are various tools acted as a P4 client to communicate with P4runtime server in SONiC. For example, "[p4runtime-shell](#)", "[p4rt-client](#)" and "[onos](#)"...etc.

This guide focuses on using "[p4runtime-shell](#)" as a P4 client to configure SONiC switch in the following section. And we adopt the method "[Run with Docker](#)", which is recommended in the "[p4runtime-shell README.md](#)".

### Check if the Docker Image of P4runtime-sh is Available in Your Local Repository

A docker image should be located in your repository after following the [step](#) in the README.md.

```
$ docker images | grep p4runtime
p4lang/p4runtime-sh latest 1beb21dedae0 2 months ago 152MB
```

## Start the Connection with P4Runtime Sever and Push a Forwarding Pipeline Configuration with the Shell

### Usage

```
$ docker run -ti -v /tmp:/tmp/ p4lang/p4runtime-sh --grpc-addr <server IP>:<server port> --device-id 0 --election-id 0,1 --config /tmp/p4info.txt,/tmp/bmv2.json
```

<server IP> : The IP address of your P4 target's management port.  
 <server port>: The tcp port of your P4 target, default should be "9559".  
 p4info.txt : The output of compiling P4 source code, it is also called P4Info file.  
 bmv2.json : The output of compiling P4 source code.

In the following example, we take the instantiation example(i.e. [middleblock.p4](#)) from google in [sonic-pins](#) but not activate the ACL related source code([middleblock\\_without\\_acl.p4](#)). Therefore, put the following files([middleblock\\_without\\_acl.p4info.txt](#), [middleblock\\_without\\_acl.json](#)) which are the output of compiling "middleblock\_without\_acl.p4" into /tmp/ directory on your server.

### Example

```
$ docker run -ti -v /tmp:/tmp/ p4lang/p4runtime-sh --grpc-addr 10.250.0.196:9559 --device-id 0 --election-id 0,1 --config /tmp/p4info.txt,/tmp/bmv2.json
```

An interaction interface will be opened when the connection has been established successfully.

```
sonic@sonic-server-02:~$ docker run -ti -v /tmp:/tmp/ p4lang/p4runtime-sh --grpc-addr 10.250.0.196:9559 --device-id 0 --election-id 0,1 --config /tmp/p4info.txt,/tmp/bmv2.json
*** Welcome to the IPython shell for P4Runtime ***
P4Runtime sh >>> tables
ingress.routing.ipv4_table
ingress.routing.ipv6_table
ingress.routing.neighbor_table
ingress.routing.next_hop_table
ingress.routing.router_interface_table
ingress.routing.wcmp_group_table
P4Runtime sh >>> |
```

The available commands in the P4Runtime shell can be referenced to the [section](#) in "p4runtime-shell README.md".

## An Example to Set a Route Entry on P4 Target through P4runtime Shell

The example in this section is going to configure a route entry on P4 target which takes [middleblock\\_without\\_acl.p4info.txt](#) as its forwarding pipeline configuration step by step.

### Configure Route Interface Table

#### p4runtime shell command

```
te = table_entry["ingress.routing.router_interface_table"](action="ingress.routing.set_port_and_src_mac")
te.match["router_interface_id"]="1000"
te.action["port"]="Ethernet0"
te.action["src_mac"]="a8:2b:b5:f0:f0:f5"

//publish the configuration to P4 target
te.insert
```

## The Output Result on P4runtime Shell:

```
P4Runtime sh >>> te = table_entry["ingress.routing.router_interface_table"](action="ingress.routing.set_port_and_src_mac")
...: te.match["router_interface_id"]="1000"
...: te.action["port"]="Ethernet0"
...: te.action["src_mac"]="a8:2b:b5:f0:f0:f5"
field_id: 1
exact {
 value: "1000"
}

param_id: 1
value: "Ethernet0"

param_id: 2
value: "\250+\265\360\360\365"

P4Runtime sh >>> te.insert
P4Runtime sh >>> █
```

## Configure Neighbor Table

### p4runtime shell command

```
te = table_entry["ingress.routing.neighbor_table"](action="ingress.routing.set_dst_mac")
te.match["router_interface_id"]="1000"
te.match["neighbor_id"]="fe80::aa2b:b5ff:fef0:ede3"
te.action["dst_mac"]="a8:2b:b5:f0:ed:e3"

//publish the configuration to P4 target
te.insert
```

## The Output Result on P4runtime Shell:

```
P4Runtime sh >>> te = table_entry["ingress.routing.neighbor_table"](action="ingress.routing.set_dst_mac")
...: te.match["router_interface_id"]="1000"
...: te.match["neighbor_id"]="fe80::aa2b:b5ff:fef0:ede3"
...: te.action["dst_mac"]="a8:2b:b5:f0:ed:e3"
field_id: 1
exact {
 value: "1000"
}

field_id: 2
exact {
 value: "fe80::aa2b:b5ff:fef0:ede3"
}

param_id: 1
value: "\250+\265\360\355\343"

P4Runtime sh >>> te.insert
P4Runtime sh >>> █
```

## Configure Nexthop Table

p4runtime shell command

```
te = table_entry["ingress.routing.nexthop_table"](action="ingress.routing.set_nexthop")
te.match["nexthop_id"]="nexthop1"
te.action["router_interface_id"]="1000"
te.action["neighbor_id"]="fe80::aa2b:b5ff:fef0:ede3"

//publish the configuration to P4 target
te.insert
```

The Output Result on P4runtime Shell:

```
P4Runtime sh >>> te = table_entry["ingress.routing.nexthop_table"](action="ingress.routing.set_nexthop")
...: te.match["nexthop_id"]="nexthop1"
...: te.action["router_interface_id"]="1000"
...: te.action["neighbor_id"]="fe80::aa2b:b5ff:fef0:ede3"
field_id: 1
exact {
 value: "nexthop1"
}

param_id: 1
value: "1000"

param_id: 2
value: "fe80::aa2b:b5ff:fef0:ede3"

P4Runtime sh >>> te.insert
P4Runtime sh >>> █
```

Configure Ipv4 Route Entry Table

p4runtime shell command

```
te=table_entry["ingress.routing.ipv4_table"](action="ingress.routing.set_nexthop_id")
te.match["vrf_id"]=" "
te.match["ipv4_dst"]="10.2.2.0/24"
te.action["nexthop_id"]="nexthop1"

//publish the configuration to P4 target
te.insert
```

```
P4Runtime sh >>> te=table_entry["ingress.routing.ipv4_table"](action="ingress.routing.set_nexthop_id")
...: te.match["vrf_id"]=" "
...: te.match["ipv4_dst"]="10.2.2.0/24"
...: te.action["nexthop_id"]="nexthop1"

field_id: 1
exact {
}

field_id: 2
lpm {
 value: "\n\002\002\000"
 prefix_len: 24
}

param_id: 1
value: "nexthop1"

P4Runtime sh >>> te.insert

P4Runtime sh >>> █
```

## Verify the Table Entry in Redis Database

In this section, we assume that the above configuration steps are executed without error. If there is any error occurred during the configuration, please refer to Troubleshooting part for assistance.

In PINS, after the request is handled by the P4runtime server, the relative P4 tables with prefix "P4RT\_TABLE" will be added in APP DB.

The tables shown below are the mapping between the table defined in P4Info and the table added in APP DB.

| Tables defined in P4Info               | Tables defined in APP DB                |
|----------------------------------------|-----------------------------------------|
| ingress.routing.router_interface_table | P4RT_TABLE:FIXED_ROUTER_INTERFACE_TABLE |
| ingress.routing.neighbor_table         | P4RT_TABLE:FIXED_NEIGHBOR_TABLE         |
| ingress.routing.nexthop_table          | P4RT_TABLE:FIXED_NEXTHOP_TABLE          |
| ingress.routing.ipv4_table             | P4RT_TABLE:FIXED_IPV4_TABLE             |
| ingress.routing.ipv6_table             | P4RT_TABLE:FIXED_IPV6_TABLE             |
| ingress.routing.wcmp_group_table       | P4RT_TABLE:FIXED_WCMP_GROUP_TABLE       |

The following command can extract the whole table entries configured by PINS. The dumped entries below are the expected output when PINS configuration operations are successful.

```
$ redis-cli KEYS "P4*"
```

```
admin@sonic:~$ redis-cli KEYS "P4*"
1) "P4RT_TABLE:FIXED_IPV4_TABLE:{\"match/ipv4_dst\": \"10.2.2.0/24\", \"match/vrf_id\": \"\"}"
2) "P4RT_TABLE:FIXED_ROUTER_INTERFACE_TABLE:{\"match/router_interface_id\": \"1000\"}"
3) "P4RT_TABLE:FIXED_NEIGHBOR_TABLE:{\"match/neighbor_id\": \"fe80::aa2b:b5ff:fe0:ede3\", \"match/router_interface_id\": \"1000\"}"
4) "P4RT_TABLE:FIXED_NEXTHOP_TABLE:{\"match/nexthop_id\": \"nexthop1\"}"
```

## Troubleshooting & Limitation

- A request which is aimed at the specified table like "P4RT\_TABLE:FIXED\_IPV4\_TABLE" is not allowed and will be forbidden by the P4Runtime Server.  
PINS only supports to handle the "read request" which is going to extract the **whole** table entries in P4 table.
- The P4 Client used in this document(i.e. P4runtime shell) has a limitation on reading the P4 tables entries. It only supports to get a specified table from P4Runtime Server.
- PINS does not support to replace the previous forwarding pipeline configuration during system runtime. Once a forwarding pipeline configuration had been pushed into P4 target, it needs to reboot the device before you push another configuration.

## Appendix A: Compiling P4 Source Code

The tool used to compile P4 source code is [p4c](#). It can be installed from [package](#) or [source code](#). Because the file [middleblock\\_without\\_acl.p4](#) is based on the header definition in [sai\\_p4](#), it should be put into the directory(`sai_p4/instantiations/google/`) in sonic-pins project before starting to compile.

### Usage

```
$ p4c --target <target> --arch <arch> --p4runtime-files=<filename> <source file>
```

`<target>` : Specify the target behavior model.  
`<arch>` : Specify the target architecture.  
`<filename>`: The file name of output P4Info file.  
`<source>` : The source file of p4 program.

### Example

```
$ p4c --target bmv2 --arch v1model --p4runtime-files=middleblock_without_acl.p4info.txt
middleblock_without_acl.p4
```

## Reference

- <https://github.com/p4lang/p4runtime-shell>
- <https://github.com/p4lang/p4c>
- <https://github.com/Azure/sonic-pins>
- <https://github.com/pins/tutorials>